

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 11.09.2026, 20:37:42
Файлов исходного кода: 82
Суммарный объём кода: 1.14 МВ
Глав/билетов в пособии: 30

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: tree-seg]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/82: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - main
```

```
 # Позволяет проверить Pages прямо из рабочей ветки
```

```
 - "arena/**"
```

```
 workflow_dispatch:
```

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
```


- ```
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
 done
 convert -version
```
- name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.density}
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit
continue-on-error: true
  - name: Summary  
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \public/export/full\_code\_all\_pages"
 echo "| \public/export/full\_code\_all\_pages"
 echo "| PNG-страниц | \$(ls tmp/export-pages
 } >> "\$GITHUB\_STEP\_SUMMARY"
  - name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7

```
 steps:
 - name: Checkout
 - uses: actions/checkout@v4
 + uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 - uses: actions/setup-node@v4
 + uses: actions/setup-node@v7
 with:
 - node-version: "22"
 + node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
 } >> "$GITHUB_STEP_SUMMARY"

 - name: Upload artifacts (PDF + TXT)
 - uses: actions/upload-artifact@v4
 + uses: actions/upload-artifact@v7
 with:
 name: full-code-export
 path: |
@@ -107,7 +107,7 @@ jobs:
 retention-days: 30

 - name: Upload artifacts (PNG-страницы)
 - uses: actions/upload-artifact@v4
 + uses: actions/upload-artifact@v7
 with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png

`index.html`
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
  <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

### Patch относительно `main`

````diff

diff --git a/public/favicon.svg b/public/favicon.svg

new file mode 100644

index 00000000..3bbbba0

--- /dev/null

+++ b/public/favicon.svg

@@ -0,0 +1,5 @@

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

````

## `src/App.tsx`

### Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
```



```

<div className="min-h-screen bg-slate-950 text-slate-50" >
  <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

  {activeTab === "guide" ? (
    <>
      <MobileToc
        selectedChapterId={activeChapterId}
        setSelectedChapterId={goTo}
        currentTitle={activeChapter.title}
        index={index}
        total={chapters.length}
      />

      <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
        {/* Сайдбар только на десктопе – на мобильном скрывается */}
        <div className="hidden lg:block lg:w-80 shrink-0" >
          <Sidebar selectedChapterId={activeChapterId} />
        </div>

        <main id="main-content" className="flex-1 m-0 p-4" >
          {/* Шапка главы с быстрыми переходами */}
          <div className="flex items-center justify-between" >
            <div className="min-w-0" >
              <div className="text-[10px] uppercase" >
                {activeChapter.category || "Универсальное пособие"}
              </div>
              <h2 className="text-base sm:text-xl font-medium" >
                {activeChapter.title}
              </h2>
            </div>
            <ChapterNav prev={prev} next={next} index={index} />
          </div>

          <div className="h-1 w-full bg-slate-800 rounded" >
            <div
              className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
              style={{ width: `${progress}%` }}
            />

```

Patch относительно `main`

```diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/Sim

import { SimulatorHub } from "../components/SimulatorHub

+import { PythonCompiler } from "../components/PythonComp

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" |

+ const [activeTab, setActiveTab] = useState<"guide" |

const [activeChapterId, setActiveChapterId] = useSta

const [modalVizId, setModalVizId] = useState<string

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

- ) : (

+ ) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

/>

</main>

+ ) : (

+ <PythonCompiler />

}}

<SimulatorModal vizId={modalVizId} onClose={() =>

```
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-slate-900">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-extrabold">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-slate-900 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-slate-900 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-compiler-btn"
 onClick={() => setActiveTab('compiler')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-slate-900 duration-200 whitespace-nowrap ${
 activeTab === 'compiler'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Компилятор
 </button>
 </div>
</div>
```

```

 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
</header>
);
};
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
    >
      <Cpu className="w-4 h-4" /> Тренажёры
```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index-gz.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index-gz.js`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL, {
      module as PyodideModule,
    }).then((module) => module.loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}
```

```

    }
  }, [code, stdin, running, runtimeReady]);

const reset = () => {
  setCode(STARTER_CODE);
  setStdin("");
  setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
  <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
    <div className="max-w-6xl mx-auto">
      <div className="flex flex-col sm:flex-row sm:items-center">
        <div>
          <div className="inline-flex items-center gap-2">
            <Terminal className="w-4 h-4" /> Боковая панель
          </div>
          <h2 className="text-2xl sm:text-3xl font-extrabold">
            <p className="text-slate-400 mt-2 max-w-2xl">
              Пишите небольшой код и запускайте его прямо в браузере,
              который переводит CPython в WebAssembly.
            </p>
          </div>
          <a
            href="https://pyodide.org/"
            target="_blank"
            rel="noreferrer"
            className="inline-flex items-center gap-1.5"
          >
            Как это работает <ExternalLink className="w-4 h-4" />
          </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
          <section className="rounded-2xl border border-slate-200 p-4">
            <div className="flex flex-wrap items-center">
              <div className="flex items-center gap-2">
                <span className="w-2.5 h-2.5 rounded-full bg-slate-200">

```

```

        placeholder="Напишите Python-код..."
    />

    <div className="border-t border-slate-800 border"
      <label className="block px-4 pt-3 text-slate-500"
        Ввод для input() • по одной строке на клавишу
      </label>
      <textarea
        id="python-stdin"
        value={stdin}
        onChange={(event) => setStdin(event.target.value)}
        spellCheck={false}
        rows={2}
        placeholder={'Например:\nАлиса'}
        className="block w-full resize-y bg-transparent border"
        style={{border: '1px solid #333'}}
      />
    </div>
  </section>

  <section className="rounded-2xl border border-slate-800 border"
    <div className="flex items-center justify-between"
      <div className="flex items-center gap-2 text-slate-500"
        <Terminal className="w-4 h-4 text-emerald-500"
        />
      <span className={`inline-flex items-center gap-2`}
        {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500"
        />
        {runtimeReady ? "готов" : "не загружен"}
      </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px] border border-slate-800 p-4"
      x] leading-6 text-slate-300">
      {output}
    </pre>
    <div className="border-t border-slate-800 border"
  </section>
</div>

```

```

+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
+  const [runtimeReady, setRuntimeReady] = useState(fal

```



```

+   };
+
+   return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+       <div className="max-w-6xl mx-auto">
+         <div className="flex flex-col sm:flex-row sm:items-center">
+           <div>
+             <div className="inline-flex items-center gap-2">
+               <Terminal className="w-4 h-4" /> Боковая панель
+             </div>
+             <h2 className="text-2xl sm:text-3xl font-extrabold">
+               <p className="text-slate-400 mt-2 max-w-2xl">
+                 Пишите небольшой код и запускайте его прямо в браузере,
+                 который переводит CPython в WebAssembly.
+               </p>
+             </div>
+             <a
+               href="https://pyodide.org/"
+               target="_blank"
+               rel="noreferrer"
+               className="inline-flex items-center gap-1.5"
+             >
+               Как это работает <ExternalLink className="text-slate-400" />
+             </a>
+           </div>
+
+           <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+             <section className="rounded-2xl border border-slate-200 p-4">
+               <div className="flex flex-wrap items-center gap-2">
+                 <div className="flex items-center gap-2">
+                   <span className="w-2.5 h-2.5 rounded-full bg-slate-200">
+                     <span className="text-sm font-bold text-slate-400">
+                       <span className="text-[11px] text-slate-400">
+                         </div>
+                   <div className="flex items-center gap-2">
+                     <button
+                       type="button"
+                       onClick={reset}

```

```

+         <textarea
+             id="python-stdin"
+             value={stdin}
+             onChange={(event) => setStdin(event.target.value)}
+             spellCheck={false}
+             rows={2}
+             placeholder={'Например:\nАлиса'}
+             className="block w-full resize-y bg-transparent border-slate-800 p-2"
+         />
+     </div>
+ </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4">
+         <div className="flex items-center justify-between">
+             <div className="flex items-center gap-2">
+                 <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+             </div>
+             <span className={`inline-flex items-center gap-2`}
+                 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+                 {runtimeReady ? "готов" : "не загружен"}}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-2">
+         </div>
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3">
+     <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+         <Info className="w-4 h-4 shrink-0 text-indigo-500 font-mono" />
+         <span>Ctrl или Cmd + Enter запускает код. L</span>
+     </div>
+     <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+         <Info className="w-4 h-4 shrink-0 text-amber-500 font-mono" />

```

```

    },
    server: {
      host: "0.0.0.0",
      port: 5173,
      strictPort: true,
      // предпросмотр может открываться через внешний про
      allowedHosts: true,
    },
    preview: {
      host: "0.0.0.0",
      port: 4173,
      strictPort: true,
      allowedHosts: true,
    },
  },
});

```

Универсальное пособие • полный исходный код

- Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и классы элементов. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/82: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <title> Дискретка для тупых – Интерактивный гид по дискретной математике
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/82: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "description": "Удобная web-читалка для подготовки к экзаменам",
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

Универсальное пособие • полный исходный код

```
-----  
    "@vitejs/plugin-react": "5.1.1",  
    "esbuild": "^0.28.2",  
    "tailwindcss": "4.1.17",  
    "typescript": "5.9.3",  
    "vite": "7.3.2",  
    "vite-plugin-singlefile": "2.3.0"  
  },  
  "description": "Интерактивное пособие по алгоритмам и  
}
```

ФАЙЛ 7/82: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с
и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash  
npm install
npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|--------------------|
| --- | --- | --- |
| `npm run export:txt` | код → txt | `public/export/ful |
| `npm run export:png` | txt → png | `tmp/export-pages/ |
| `npm run export:pdf` | png → pdf | `public/export/ful |

Универсальное пособие • полный исходный код

известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор).
Словарь — `src/data/glossary.ts`, мини-анимации — `src/anim`
* **Реестр интерактивов** — `src/components/vizRegistry`
и для панели под главой, и для модальки из подсказки.

Структура

...

| | |
|--------------------|---|
| src/ | |
| App.tsx | — оболочка, привязка глав к визуализации |
| components/ | — интерактивные симуляторы (Действия) |
| data/ | — контент пособия (главы/билеты/карты) |
| scripts/export/ | — пайплайн код → txt → png → pdf |
| public/export/ | — сгенерированные TXT и PDF (обновляются) |
| .github/workflows/ | — автоматизация |

...

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и классы.
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика — параграфы `

`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| | | |
|--------------------|-----------|----------------------|
| Глава | id | Что делать |
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| | | |
|--------|--------------------------------|-----------------------------|
| Билет | Тема | Статус |
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. К |
| | ит вхолостую. | |
| **7** | Планарность и раскраска графов | Номерной глав |
| | огии**. | |
| **11** | Мосты в графах | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- * `stack-dfs` → `StackViz.tsx` – **Стек и DFS**
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **Очередь
- * `heap-beam-search` → `HeapViz.tsx` – **Куча и лучевой
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – **Дер
- * `dynamic-programming` → `DPVisualizer.tsx` – **Динами

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

| | | |
|--|--------------|-------------|
| Глава | id | Комментарий |
| --- | --- | --- |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e | |

Универсальное пособие • полный исходный код

| | | | | |
|----|--------------------------------------|---|---|---|
| 17 | 17. Графы. Алгоритм Джонсона | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - | - |
| 22 | 22. Z-функция | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| - | Обзор: Кратчайшие пути и Графы | - | - | - |
| - | Алгоритмы на карте | - | - | - |
| - | Эйлер: Путь ≠ Цикл | - | - | - |
| - | Мосты в графах: код + визуализация | 1 | - | - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)
это 5 готовых компонентов, которые сейчас просто не...
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш...
3. ****Добавить 2D**** к 4 главам «Уровня 4» – минимум inli...
4. ****Решить судьбу `alg-map`**** – раскрыть или удалить.

ФАЙЛ 9/82: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],
```



```
-----
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

=====

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

ФАЙЛ 11/82: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 KB

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate-
```

```
    <Navbar activeTab={activeTab} setActiveTab={setAc
```

```
    {activeTab === "guide" ? (
```

```
      <>
```

```
        <MobileToc
```

```
          selectedChapterId={activeChapterId}
```

```
          setSelectedChapterId={goTo}
```

```
          currentTitle={activeChapter.title}
```

```
          index={index}
```

```
          total={chapters.length}
```

```
        />
```

```
      <div className="flex flex-col lg:flex-row max
```

```
        {/* Сайдбар только на десктопе – на мобильном
```

```
        <div className="hidden lg:block lg:w-80 shr
```

```
          <Sidebar selectedChapterId={activeChapter
```

```
        </div>
```

```
      <main id="main-content" className="flex-1 m
```

```
        {/* Шапка главы с быстрыми переходами */}
```

```
        <div className="flex items-center justify
```

```
          <div className="min-w-0">
```

```
            <div className="text-[10px] uppercase
```

```
              {activeChapter.category || "Универс
```

```
            </div>
```

```
            <h2 className="text-base sm:text-xl f
```

```
              {activeChapter.title}
```

```
            </h2>
```

```
          </div>
```

```
          <ChapterNav prev={prev} next={next} ind
```

```
        </div>
```

```
      <div className="h-1 w-full bg-slate-800 r
```

```
        <div
```

ФАЙЛ 12/82: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
  const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l))
  // границы «не буква/цифра» – \b не работает с кириллицей
  return new RegExp(`(?<![\\p{L}\\p{N}]_~)(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * <span class="term-hint" data-term-id="...">, чтобы на них
 * повесить всплывающую карточку (как превью ссылок в Википедии)
```

```

    const usedTerm = perTerm.get(id) ?? 0;
    const usedSurface = perSurface.get(surface) ?? 0;
    if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
      perTerm.set(id, usedTerm + 1);
      perSurface.set(surface, usedSurface + 1);
    }

    if (m.index > last) frag.appendChild(document.createElement("span"));
    const span = document.createElement("span");
    span.className = "term-hint";
    span.dataset.termId = id;
    span.tabIndex = 0;
    span.setAttribute("role", "button");
    span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
    span.textContent = m[1];
    frag.appendChild(span);
    last = m.index + m[1].length;
  }

  if (last === 0) continue;
  if (last < text.length) frag.appendChild(document.createElement("span"));
  textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
  const run = () => {
    const el = ref.current;
    if (!el) return;
    try {
      annotateTerms(el);
    }
  };
  useLayoutEffect(run, []);
}

```

```
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
  from {
    transform: translateX(-100%);
  }
  to {
    transform: translateX(0);
  }
}

@keyframes hintIn {
  from {
    opacity: 0;
  }
}
```

```
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
    white-space: pre-wrap;
    word-break: break-word;
}

.chapter-body pre {
    overflow-x: auto;
    max-width: 100%;
    font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
    display: block;
    width: 100%;
    overflow-x: auto;
    border-collapse: collapse;
    font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
    cursor: pointer;
    padding: 0.35rem 0;
}

@media (max-width: 767px) {
    .chapter-body :where(.grid) {
        grid-template-columns: minmax(0, 1fr) !important;
    }
    .chapter-body :where(h2) {
        font-size: 1.25rem !important;
    }
}
```

```
    from {  
      width: 0%;  
    }  
    to {  
      width: 100%;  
    }  
  }  
}
```

ФАЙЛ 14/82: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";  
import { createRoot } from "react-dom/client";  
import "./index.css";  
import App from "./App";  
  
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

ФАЙЛ 15/82: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {  
  id: string;  
  title: string;  
  type: "html" | "markdown";  
  content: string;  
  description?: string;  
  category?: string;
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
          ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные заготовки
            ь отрезок длины 7 (перекрывая друг друга по 1 единицу)
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
              Аналогия 2: Ступени
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
`
  }
]
```



```

        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
метода: Split и Merge.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Split – таможня
        </div>
        <p class="text-slate-300 text-sm mb
> &gt; х». Спуск от корня: узел, который вме
ез вскрытия</b> – дальше досматривается тол
    </div>
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
            Аналогия 2: Merge – стыковка
        </div>
        <p class="text-slate-300 text-sm mb
дый шаг – одно сравнение: чей у у корня бол
что сохраняет порядок х). Итого O(h) сравн
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {

```

на каждом шаге спуска от корня.

«!» держит всё слово: уберечь его от
ево поиска, которое на неудачном вводе выро-
– идут влево по уставу. В демо-наборе все

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg
<div class="absolute -top-3 left-4
border-rose-500 shadow-md">

Ты пытаешь

</div>

<p class="text-slate-300 text-sm mb
не быстрее $n \cdot \log n$.</p>

<p class="text-slate-300 text-sm mb
</p>

<p class="text-slate-300 text-sm">•
сшиваются merge за $O(h)$.</p>

</div>

</div>

<p class="text-slate-400 text-sm mt-6">Симу-
uicksort против counting → линии в любом по-
Миф $2k/2k+1$ и Поиск ≠ сортировка

<p class="text-slate-300 text-sm mt-4">с
"bg-slate-950 px-2 py-0.5 rounded text-xs f-
оение – <code class="bg-slate-950 px-2 py-0.5
<code> (сортировка + n спусков; counting по n
2 py-0.5 rounded text-xs font-mono text-rose-500
x-2 py-0.5 rounded text-xs font-mono text-rose-500

<details class="bg-slate-900/40 rounded-lg l-
<summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">

Код: вставка и удаление (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
```

```
<!-- 0. Определение -->
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border"
  <h3 class="text-xl font-bold text-blue-400">
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-blue-300 italic">
    <p class="text-lg sm:text-xl font-mono">
поиска <span class="text-slate-500">без</span>
: серия поворотов, которая поднимает v в ко
    <p class="text-sm text-slate-400 text-c">
т – поворот.</p>
  </div>
```

```
<div class="grid grid-cols-1 lg:grid-cols-3"
  <div class="bg-slate-900 rounded-lg p-4"
    <p class="text-xs uppercase tracking">
    <p class="text-white font-bold">мож
    <p class="text-slate-400 text-xs mt">
  </div>
  <div class="bg-slate-900 rounded-lg p-4"
    <p class="text-xs uppercase tracking">
    <p class="text-emerald-300 font-bol">
    <p class="text-slate-400 text-xs mt">
  </div>
  <div class="bg-slate-900 rounded-lg p-4"
    <p class="text-xs uppercase tracking">
    <p class="text-white font-bold">0(n
    <p class="text-slate-400 text-xs mt">
  </div>
</div>
</div>
```

ет ровно это: «взял → положил наверх».

```
<div class="bg-slate-900 p-6 rounded-lg border-2 border-rose-500">
  <div class="absolute -top-3 left-4 bg-rose-500 border-rose-500 shadow-md">
```

Аналогия 2: тропинка через газон

`<p class="text-slate-300 text-sm">Ты ид
ода дорожка сама укорачивалась вдвое: чем ч
оплачивает все следующие. Пойдѐшь один раз
</div>`

<!-- 3. Поворот -->

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">
  <h3 class="text-lg font-bold text-white mb-4">Почему это важно?
  <p class="text-slate-300 text-sm mb-4">Поворот, движение, три перевешенные ссылки</span> и
  <pre class="bg-slate-950 p-4 rounded-lg overflow-hidden">
    adding-relaxed">
```

40

```

      /  \
    20    C
   /  \
  A    B

```

ПОВОРОТ 20

-----&at;

<-----

ПОВОРОТ 40

/ \\
 A 40
 / \\
 B C

обход слева направо ДО: А 20 В 40 С

обход слева направо ПОСЛЕ: A 20 B 40 C ← тот же шаг

```
<div class="grid grid-cols-1 md:grid-cols-2
```

```
<div class="bg-slate-900 rounded-lg p-4">
  <p class="text-emerald-400 font-bold">
    <p class="text-slate-300 text-xs">П
сла поворотов – сломать его поворотами нево
```

```
</div>
<div class="bg-slate-900 rounded-lg p-4">
  <p class="text-amber-400 font-bold">
  <p class="text-slate-300 text-xs">П
одно и то же место, поэтому В просто перев
```

```

<td class="py-3">р и г стали
</tr>
</tbody>
</table>
</div>

```

```

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>
</div>

```

<!-- 5. Главная ловушка -->

```

<div class="bg-rose-950/30 p-6 rounded-xl borde
<h3 class="text-lg font-bold text-rose-300 m
<p class="text-slate-300 text-sm mb-4">Это
ми.</p>
<div class="grid grid-cols-1 lg:grid-cols-2
  <div class="bg-slate-950 rounded-lg p-4
    <p class="text-rose-400 font-bold t
    <pre class="overflow-x-auto text-[1

```

```

      20
    /
  40 /
 /
20

→

      20
    /
  20 \
    \
  40

→

      \
    60
    /
  40

```

было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!</pre>

```

  <p class="text-slate-400 text-xs mt
мортизации не получится.</p>
</div>

```

```

<div class="bg-slate-950 rounded-lg p-4
  <p class="text-emerald-400 font-bol
  <pre class="overflow-x-auto text-[1

```

```

      20
    /
  /  \
 /    \
/      \

```

```

<!-- 7. Амортизация -->
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">
  <h3 class="text-lg font-bold text-white mb-3">7. Амортизация
  <p class="text-slate-300 text-sm mb-3">Строим дерево (сplay-
оддеревя) по всем узлам, и Zig-Zig/Zig-Zag. Идея такая:
  <div class="grid grid-cols-1 md:grid-cols-3 gap-4">
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-white font-bold mb-1">1. Узел
      <p class="text-slate-400 text-xs">ссылка на узел
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-white font-bold mb-1">2. Узел
      <p class="text-slate-400 text-xs">на узле
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-white font-bold mb-1">3. Узел
      <p class="text-slate-400 text-xs">дерева
    </div>
  </div>
  <p class="text-slate-400 text-xs mt-4">Отсюда видно, что количество запросов splay-дерева не хуже (с точностью до константы) чем в бинарном дереве. Оно как бы само подстраивается под
</div>

```

```

<!-- 8. Операции -->
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">
  <h3 class="text-lg font-bold text-white mb-3">8. Операции
  <p class="text-slate-300 text-sm mb-4">Красная операция:
  <div class="space-y-2 text-sm">
    <div class="bg-slate-900 rounded-lg p-3">
      <p class="text-white font-bold mb-1">1. Узел
      <p class="text-slate-400 text-xs">ссылка на узел
    </div>
    <div class="bg-slate-900 rounded-lg p-3">
      <p class="text-white font-bold mb-1">2. Узел
      <p class="text-slate-400 text-xs">на узле
    </div>
    <div class="bg-slate-900 rounded-lg p-3">
      <p class="text-white font-bold mb-1">3. Узел
      <p class="text-slate-400 text-xs">дерева
    </div>
  </div>
  <p class="text-slate-400 text-xs mt-4">Отсюда видно, что количество запросов splay-дерева не хуже (с точностью до константы) чем в бинарном дереве. Оно как бы само подстраивается под
</div>

```

```

function splay(x) {
  while (x.parent) {
    const p = x.parent, g = p.parent;

    if (!g) { // ZIG: деда не нашли
      rotate(x);
    } else if ((g.left === p) === (p.left === x)) {
      rotate(p); // ZIG-ZIG: сносим вправо
      rotate(x);
    } else {
      rotate(x); // ZIG-ZAG: сносим влево
      rotate(x);
    }
  }
  return x; // теперь x — корень
}

// поиск: спустились и подняли найденное наверх
function find(root, key) {
  let cur = root, last = null;
  while (cur) {
    last = cur;
    if (key === cur.key) break;
    cur = key < cur.key ? cur.left : cur.right;
  }
  return last ? splay(last) : null; // splay делаем обязательно
}

```

<p class="text-xs text-slate-400">Вся работа в функции splay(x);

 против того, что в бинарном дереве, где мы не можем сразу найти элемент по ключу, а только по его оценке.</p>

</div>

</details>

<!-- 10. Плюсы/минусы -->

<div class="grid grid-cols-1 md:grid-cols-2 gap-4">

<div class="bg-emerald-950/30 rounded-xl p-4">

<p class="text-emerald-300 font-bold mb-2">

<ul class="list-disc list-inside text-slate-400">

```

        вает один из них в корень, превращая дерево
        ступает только для <em>одной</em> конкретно
        </div>
        <div>
            <strong class="text-white block mb-6">
                <p class="text-slate-400">Treap держит
                play не хранит вообще ничего и держит баланс
            <p>
        </div>
    </div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сжатие суммы подматрицы за  $O(1)$ ",
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">
                <p class="text-xl font-mono text-white">
                , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4">

```



```

        <p class="text-slate-300 text-sm mb-6">
            . Границы не пересекаются в одной точке по-
            ы не сливались цветами, Всегда можно всего
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
        id: "top-sort",
        title: "9. Графы. Топологическая сортировка",
        type: "html",
        description: "Разложение задач по порядку выполнения",
        category: "Графы",
        content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-6">
                <p class="text-lg text-blue-300 italic">
                <p class="text-xl font-mono text-white">
перед.</p>
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
                        Аналогия 1: Учеба в вузе
                    </div>
                    <p class="text-slate-300 text-sm mb-6">
                        еская сортировка – это расписание, которое
                    >

```

Аналогия 1: Улицы с односторонним движением

```

    </div>
    <p class="text-slate-300 text-sm mb-6">
      В некоторых направлениях по односторонним улицам. Кликните по
      у на карте.</p>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Рёбра, удаление которых раскхреничивает граф",
    category: "Графы. Продвинутые",
    content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
          <h3 class="text-xl font-bold text-rose-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
              <p class="text-lg text-rose-300 italic">
                <p class="text-xl font-mono text-white">
                  и fup[v] > tin[u].</p>
              </div>
            <div class="grid grid-cols-1 gap-6">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border
                  rder border-emerald-500 shadow-md">
                  Аналогия 1: Единственный мост
                </div>
              <p class="text-slate-300 text-sm mb-6">

```

```

    </div>
  </section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Эйлеров цикл
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Что такое граф?
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Граф — это набор вершин, соединенных ребрами.
        <p class="text-xl font-mono text-white">Пример:
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрывая
          </div>
          <p class="text-slate-300 text-sm mb-4">Если у каждой вершины сходится нечетное число линий, значит, из каждой вершины
          а везде должно быть четное число (зашел-вышел).
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-coloring",
    title: "14. Графы. Раскраска графа",
    type: "html",
    description: "Раскрасить граф так, чтобы смежные вершины имели разные цвета.",
    category: "Графы",
    content: `
<section id="graph-coloring" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Раскраска графа
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Что такое раскраска графа?
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Раскраска графа — это присвоение цвета каждой вершине так, чтобы смежные вершины имели разные цвета.
        <p class="text-xl font-mono text-white">Пример:
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 2: Раскраска карт
          </div>
          <p class="text-slate-300 text-sm mb-4">Если две смежные вершины имеют один и тот же цвет, то это неверно.
          а везде должно быть четное число (зашел-вышел).
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-shortest-path",
    title: "15. Графы. Кратчайший путь",
    type: "html",
    description: "Найти кратчайший путь между двумя вершинами графа.",
    category: "Графы",
    content: `
<section id="graph-shortest-path" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Кратчайший путь
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Что такое кратчайший путь?
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Кратчайший путь — это путь между двумя вершинами графа, состоящий из минимального количества ребер.
        <p class="text-xl font-mono text-white">Пример:
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 3: Кратчайший путь в городе
          </div>
          <p class="text-slate-300 text-sm mb-4">Если есть несколько путей между двумя вершинами, то кратчайший — это тот, который имеет минимальную длину.
          а везде должно быть четное число (зашел-вышел).
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-traveling-salesman",
    title: "16. Графы. Задача коммивояжера",
    type: "html",
    description: "Найти кратчайший путь, проходящий по всем вершинам графа.",
    category: "Графы",
    content: `
<section id="graph-traveling-salesman" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Задача коммивояжера
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Что такое задача коммивояжера?
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Задача коммивояжера — это задача нахождения кратчайшего пути, проходящего по всем вершинам графа.
        <p class="text-xl font-mono text-white">Пример:
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 4: Задача коммивояжера
          </div>
          <p class="text-slate-300 text-sm mb-4">Если есть несколько путей, проходящих по всем вершинам графа, то кратчайший — это тот, который имеет минимальную длину.
          а везде должно быть четное число (зашел-вышел).
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-network-flow",
    title: "17. Графы. Поток в сети",
    type: "html",
    description: "Найти максимальный поток в сети.",
    category: "Графы",
    content: `
<section id="graph-network-flow" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Поток в сети
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Что такое поток в сети?
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Поток в сети — это количество вещества, проходящего по ребрам графа.
        <p class="text-xl font-mono text-white">Пример:
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 5: Поток в сети
          </div>
          <p class="text-slate-300 text-sm mb-4">Если есть несколько путей, проходящих по ребрам графа, то максимальный — это тот, который имеет максимальное количество вещества.
          а везде должно быть четное число (зашел-вышел).
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-matching",
    title: "18. Графы. Паросочетание",
    type: "html",
    description: "Найти максимальное паросочетание в графе.",
    category: "Графы",
    content: `
<section id="graph-matching" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Паросочетание
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Что такое паросочетание?
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Паросочетание — это набор ребер графа, не имеющих общих вершин.
        <p class="text-xl font-mono text-white">Пример:
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 6: Паросочетание
          </div>
          <p class="text-slate-300 text-sm mb-4">Если есть несколько наборов ребер, не имеющих общих вершин, то максимальное — это тот, который имеет максимальное количество ребер.
          а везде должно быть четное число (зашел-вышел).
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-planarity",
    title: "19. Графы. Плоскостность графа",
    type: "html",
    description: "Проверить, является ли граф плоскостным.",
    category: "Графы",
    content: `
<section id="graph-planarity" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Плоскостность графа
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Что такое плоскостность графа?
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Плоскостность графа — это свойство графа, позволяющее его нарисовать на плоскости без пересечений ребер.
        <p class="text-xl font-mono text-white">Пример:
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 7: Плоскостность графа
          </div>
          <p class="text-slate-300 text-sm mb-4">Если граф можно нарисовать на плоскости без пересечений ребер, то он плоскостной.
          а везде должно быть четное число (зашел-вышел).
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-isomorphism",
    title: "20. Графы. Изоморфизм графов",
    type: "html",
    description: "Проверить, являются ли два графа изоморфными.",
    category: "Графы",
    content: `
<section id="graph-isomorphism" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-60
```

```

        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Прокладка интернет
          </div>
          <p class="text-slate-300 text-sm mb-4">
            с самых дешевых дорог в стране". Он строит
            единую сеть.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
      },
      {
        id: "mst-prima",
        title: "19. Графы. Остовное дерево. Прима",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `

```

```

        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
            <h3 class="text-xl font-bold text-emerald-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-emerald-300 italic">
                        <p class="text-xl font-mono text-white">
                            ается с соседом.</p>
                        </div>
                        <div class="grid grid-cols-1 gap-6">
                            <div class="bg-slate-800 p-6 rounded-lg">
                                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                                    Аналогия: Племена
                                </div>
                                <p class="text-slate-300 text-sm mb-4">
                                    изкому племени для объединения. К вечеру пле
                                    ства шлют гонцов к ближайшему чужому царству.
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </section>`,
        },
        {
            id: "string-kmp",
            title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
            type: "html",
            description: "",
            category: "Строки",
            content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    </div>
    <div class="space-y-8">

```

```

</section>`,
  },
  {
    id: "string-z-func",
    title: "22. Z-функция",
    type: "html",
    description: "",
    category: "Строки",
    content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-blue-300 italic">
          <p class="text-xl font-mono text-white">
щегоря с i.</p>
        </div>
        <div class="grid grid-cols-1 gap-6">
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
              Аналогия 1: Копипаста
            </div>
            <p class="text-slate-300 text-sm mb">
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной коп
          </div>
        </div>
        <details class="bg-slate-900/40 rounded-lg">
          <summary class="p-4 font-bold text-slate">
            -b border-slate-700/50">
              Код (Скрыто)
            </summary>

```

```

        order border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
        </div>
        <p class="text-slate-300 text-sm mb-2">
            пример сортировка чисел). Класс <b>NP</b> – это не-
            енную сетку (ответ), он БЫСТРО (за класс P)
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border
            order border-purple-500 shadow-md">
                Аналогия 2: Машина-переводчик
            </div>
            <p class="text-slate-300 text-sm mb-2">
                ь отличный переводчик на английский (Сведения
                аче В, то В – это <b>NP-Полная</b> (сосредоточен
            </div>
        </div>
    </div>
</section>`,
    },
];

```

ФАЙЛ 18/82: src/data/content_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМЕР: 1.5 КБ

```

export interface Chapter {
    id: string;
    title: string;
    type: "html";
    content: string;
}

```

```

export const chapters: Chapter[] = [

```

```
<div class="bg-slate-800 p-6 rounded-lg border-2 border-green-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-slate-900 text-white p-2">
    Путь: Нормальная прогулка
  </div>
  <p class="text-slate-300 text-sm mb-4">
    Ты вышел из дома <strong>(нечётная)</strong>.
    Твой друг, который был с тобой в прошлый раз,
    такий пришёл в бар <strong>(вторая нечётная)</strong>.
    Домой вернуться не смог, потому что
  </p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2 border-rose-500 shadow-md bg-slate-950">
  <div class="absolute -top-3 left-4 bg-slate-900 text-white p-2">
    Цикл: Гамильтонов синдром (болезнь)
  </div>
  <p class="text-slate-300 text-sm mb-4">
    <strong>Гамильтон – это болезнь.</strong>
    Ты обходишь все бары (вершины) ровно
    по одному разу и вернуться домой.
    Ты обходишь все бары (вершины) ровно
    по одному разу и вернуться домой.
    Главное – зайти и выйти, не заходя
    дважды в одну и ту же вершину.
    Никто не знает, как решать быстро.
  <br><span class="text-xs text-rose-400">
    (это называется Гамильтоновым синдромом).</span>
  </p>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-2 border-rose-500 shadow-md">
  <summary class="p-4 font-bold text-slate-400">
    Душный режим: Формальное доказательство
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400">Теорема Эйлера
    <p>
      Связный граф содержит эйлеров цикл,
      если и только если у каждой вершины
      степени чётно.
    </p>
  </div>
</details>
```



```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
    Если из сына  $u$  и всех его потомков  $\text{strong}(u)$ 
    Единственная ниточка. Порвётся — граф развалится.
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
```

```
  <div class="bg-slate-800 p-6 rounded-lg border-rose-500">
```

```
    <div class="absolute -top-3 left-4 bg-slate-800
    border-emerald-500 shadow-md">
```

```
      Мост: Единственная веревка
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb-4">
```

```
      Представь, что ты альпинист. Ты спустился в пещеру.
      Из пещеры и нет других выходов — только вверх.
      Если верёвка (ребро  $v-u$ ) оборвётся — ты застрянешь.
      Если бы из пещеры и был чёрный ход — ты бы ушёл.
```

```
    </p>
```

```
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500">
```

```
  <div class="absolute -top-3 left-4 bg-slate-900
  der-rose-500 shadow-md">
```

```
    Аналогия: Кровеносный сосуд
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb-4">
```

```
    Граф — это кровеносная система. Ребра (артерии и вены) — это не мост, живём.
```

```
    Если же перерезать единственную артерию — организм умрёт.
    Алгоритм DFS ищет такие «критические» ребра.
```

```
  </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500">
```

```
  <summary class="p-4 font-bold text-slate-400">
```

```

        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}
}

```

```

</div>
<p class="text-xs text-slate-400">Занес
</div>
</details>
</div>
</section>`,
},
{
    id: "planarity-euler-formula",
    title: "Планарность: K5, K3,3 и формула Эйлера",
    type: "html",
    content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
        <h3 class="text-xl font-bold text-blue-400 r

        <div class="bg-slate-900 p-4 rounded-lg mb-1
            <p class="text-lg text-blue-300 italic r

```

```

        У него <code class="text-white">V=6
        xt-white">9 ≤ 12</code> – проходит, но он в
        Почему? Потому что у двудольного графа.
        Отсюда более строгое ограничение: <code class="text-white">9 > 8</code>
    </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border
    <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
    Доказательство непланарности K5 (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
    <p class="text-blue-400">Доказательство непланарности K5</p>
    <p>
        Пусть K5 планарен. V = 5, E = 10.
        <br>F = E - V + 2 = 10 - 5 + 2 = 7
        <br>Каждая грань ограничена минимумом 3 ребрами.
        <br>Сумма длин граней = 2E = 20.
        <br>Отсюда: 2E ≥ 3F ⇒ 20 ≥ 21 – противоречие.
        <br>Значит, K5 не может быть планарным.
    </p>
</div>
</details>
</div>
</section>`,
    },
    {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        content: `<section id="graph-articulation" class="mb-6">
            <div class="flex items-center mb-6">
                <span class="bg-rose-600 text-white px-4 py-1 rounded">12. Графы. Точки сочленения
                <h2 class="text-3xl font-bold text-white">Точки сочленения
            </div>
        `
    }
]

```

```

<div class="bg-slate-900 p-6 rounded-lg border-2 border-emerald-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-emerald-500 text-white font-weight-bold text-sm">
    Телеком: Центральный свитч
  </div>
  <p class="text-slate-300 text-sm mb-4">
    У тебя несколько компьютеров в одной комнате (абелем (мостом). Сами свитчи — это точки со
  </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border-2 border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400 font-bold">Особый случай (если нет отцов, то выше неё прыгать некуда). Поэтому для него <strong>больше 1 независимого ребенка</strong>
    </p>
    <div class="bg-slate-950 p-4 rounded-lg">
      <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);

```

```
import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges"];

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
  if (!toRemove.includes(c.id)) {
    // We prefer the newer versions (like tickets14_20)
    dedup.set(c.id, c);
  }
});

// Add the new ones
if (newChapters) {
  newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
```

```

    formal?: string;
    complexity?: string;
    /** Какая мини-анимация показывается в карточке. */
    demo?: DemoKind;
    /** id полного симулятора (см. vizRegistry) – кнопка
    simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
  {
    id: "bfs",
    labels: ["BFS", "обход в ширину", "поиск в ширину",
    title: "BFS – обход в ширину",
    short:
      "Волна от источника: сначала все соседи, потом со
    formal:
      "Очередь (FIFO). Кладём старт, достаём вершину –
        у рёбер.",
    complexity: "O(V + E)",
    demo: "bfs",
    simulator: "queue-bfs",
  },
  {
    id: "dfs",
    labels: ["DFS", "обход в глубину", "поиск в глубину",
    title: "DFS – обход в глубину",
    short:
      "Прёшь вперёд до упора, упёрся – откатываешься на
    formal:
      "Стек (или рекурсия). Из DFS растут: времена вход
    complexity: "O(V + E)",
    demo: "dfs",
    simulator: "stack-dfs",
  },
  {
    id: "binsect",
    labels: ["binsect", "бинсект", "бинарный поиск", "д
    title: "Бинарный поиск (и его самозванец binsect)",

```

```

    },
    {
      id: "dsu",
      labels: ["DSU", "СНМ", "система непересекающихся множеств"],
      title: "DSU – система непересекающихся множеств",
      short:
        "«Кто твой староста?» Каждый элемент знает своего старосту",
      formal:
        "find со сжатием путей + union по рангу/размеру. Обратная функция Аккермана",
      complexity: "почти  $O(1)$  – обратная функция Аккермана",
      demo: "dsu",
      simulator: "mst-kruskal",
    },
    {
      id: "trie",
      labels: ["бор", "бора", "боре", "бору", "бором", "trie"],
      title: "Бор (trie, префиксное дерево)",
      short:
        "Дерево, где путь от корня по буквам и есть слово",
      formal: "Из корня по символам; в вершине хранится ф. слово",
      complexity: "поиск слова –  $O(|\text{слова}|)$ ",
      demo: "trie",
      simulator: "aho-corasick",
    },
    {
      id: "bfs-on-trie",
      labels: [
        "обход дерева",
        "обход бора",
        "обход в ширину по бору",
        "какой-то обход",
        "обходом дерева",
        "конечный автомат",
        "конечного автомата",
        "автомат",
        "автомата",
      ],
      title: "Обход бора в ширину (тот самый «какой-то обход»)"
    }
  ]
}

```

"«А через соседа не быстрее?» Если известный путь

formal: "if $d[v] > d[u] + w(u,v) \rightarrow d[v] = d[u] + w(u,v)$

ан и Флойд.",

demo: "relax",

simulator: "dijkstra",

},

{

id: "prefix-sum",

labels: ["префиксные суммы", "префиксная сумма", "префикс"],

title: "Префиксные суммы",

short:

"Заранее посчитанные «сколько всего набежало от начальной точки»

formal: " $P[0]=0, P[i]=P[i-1]+a[i-1]$. Сумма $a[l..r]$ = $P[r]-P[l-1]$."

complexity: "препроцессинг $O(n)$, запрос $O(1)$ ",

demo: "prefix-sum",

simulator: "prefix-sums-2d",

},

{

id: "sparse-table",

labels: [

"разреженная таблица", "разреженная таблица", "sparse-table",

"разреженная", "двоичные подьёмы", "двоичный подьём"],

],

title: "Разреженная таблица (метод двоичных подьёмов)",

short:

"Заранее считаем ответ для всех отрезков длины 1, 2, 4, 8, ...,

м. ",

formal: " $ST[i][j] = op(ST[i][j-1], ST[i+2^{(j-1)}][j-1])$ "

complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ "

demo: "sparse-table",

simulator: "sparse-table",

},

{

id: "segment-tree",

labels: ["дерево отрезков", "дерево отрезков", "дерево отрезков"],

title: "Дерево отрезков",

short:


```

        no).",
        demo: "mst",
        simulator: "mst-kruskal",
    },
    {
        id: "amortized",
        labels: [
            "амортизированная", "амортизированный", "амортизи",
            "амортизированное", "амортизированного", "амортизи",
        ],
        title: "Амортизированная сложность",
        short:
            "Средняя цена операции «по абонементу»: одна опер",
        formal: "Метод потенциалов: дорогая операция оплачи",
            "ассив.",
        demo: "amortized",
    },
    {
        id: "np",
        labels: [
            "NP-полная", "NP-полнота", "NP-трудная", "NP-полн",
            "класс P", "полином", "полиномиальное",
        ],
        title: "Класс NP и NP-полнота",
        short:
            "NP — «решение проверить легко, найти трудно» (су",
            "е в NP.",
        formal: "Задача NP-полна, если она в NP и к ней полн",
        demo: "np",
    },
    {
        id: "potential",
        labels: ["потенциал", "потенциалы", "потенциалов",
        title: "Потенциалы (перевзвешивание Джонсона)",
        short:
            "Трюк «поднять все дороги на одинаковую высоту»: ",
        formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  — ра",
            "к путей сохраняется.",
    }

```

```

        аждой букве.",
    formal:
        "z[i] = длина наибольшего общего префикса s и s[i]",
    complexity: "O(n)",
    demo: "prefix-function",
    simulator: "string-z-func",
},
{
    id: "dp",
    labels: [
        "динамическое программирование", "динамики", "дин.",
        "мемоизация", "мемоизацию", "подзадач", "подзадач",
    ],
    title: "Динамическое программирование",
    short:
        "Решаем задачу через ответы на её меньшие копии и",
    formal:
        "Нужны: оптимальная подструктура + перекрывающиеся",
        "ция).",
    demo: "dp",
    simulator: "dynamic-programming",
},
{
    id: "shortest-path",
    labels: ["кратчайший путь", "кратчайшие пути", "кра",
    title: "Кратчайший путь",
    short:
        "Самый дешёвый маршрут из A в B. «Дешёвый» – по с",
    formal:
        "Дейкстра – неотрицательные веса,  $O(E \log V)$ . Фор",
        "ы,  $O(V^3)$ .",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "negative-cycle",
    labels: ["отрицательный цикл", "отрицательного цикла",
    title: "Отрицательный цикл",

```

```

    formal:
      "V - E + F = 2 для связного планарного графа. Кур",
    simulator: "planarity-euler-formula",
  },
  {
    id: "splay",
    labels: [
      "splay", "splay-дерево", "AVL", "AVL-дерево", "вс",
    ],
    title: "Splay и повороты",
    short:
      "Каждый раз, когда к узлу обратились, его «вытягив",
    formal:
      "Три случая: zig (родитель - корень), zig-zig (уз",
      "рацию.",
    demo: "amortized",
    simulator: "splay-tree",
  },
  {
    id: "adjacency",
    labels: ["матрица смежности", "список смежности", "с",
    ],
    title: "Способы хранить граф",
    short:
      "Матрица смежности - таблица «есть ли ребро», быс",
      "й, экономно для разреженных графов.",
    formal: "Матрица: проверка O(1), память O(V^2). Спис",
  },
  {
    id: "dijkstra",
    labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс",
    ],
    title: "Алгоритм Дейкстры",
    short:
      "Жадный «навигатор»: всегда раскрываем ближайший",
      "льные.",
    formal:
      "Куча хранит пары (расстояние, вершина). Достали",
    complexity: "O(E log V) с бинарной кучей",
  },

```

```

    short:
        "Остов растёт как плесень из одной точки: на каждой
formal:
        "Множество посещённых + куча рёбер на границе. До
        усок.",
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-prim",
},
{
    id: "boruvka",
    labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
    title: "Алгоритм Борувки",
    short:
        "Все компоненты одновременно шлют «гонца» к ближайш
formal:
        "Фаза: для каждой компоненты ищем минимальное исх
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-boruvka",
},
{
    id: "kruskal",
    labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
    title: "Алгоритм Краскала",
    short:
        "Сортируем все рёбра по весу и берём подряд те, ч
formal:
        "Сортировка O(E log E) + E операций union/find. С
    complexity: "O(E log E)",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "

```

```
{
  id: "treap",
  labels: ["Treap", "декартово дерево", "декартова де",
  title: "Treap = Tree + Heap",
  short:
    "Одно дерево живёт по двум правилам сразу: по ключу и по случайности. Случайность и держит его сбалансированным.",
  formal:
    "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить два дерева через них. Ожидаемая высота  $O(\log n)$ ).",
  complexity: "ожидаемо  $O(\log n)$ ",
  demo: "heap",
},
{
  id: "bst",
  labels: ["бинарное дерево поиска", "дерево поиска",
  title: "Бинарное дерево поиска",
  short:
    "Слева всё меньше корня, справа — всё больше. Поиск по ключу.",
  formal:
    "Без балансировки дерево может вырождаться в список. Ожидаемая высота  $O(\log n)$ .",
  complexity: " $O(\text{высоты})$ : от  $O(\log n)$  до  $O(n)$ ",
  demo: "segment-tree",
},
{
  id: "inclusion-exclusion",
  labels: [
    "включений-исключений", "включения-исключения", "принцип включения-исключения",
    "вырезание прямоугольников",
  ],
  title: "Включения-исключения на префиксных суммах",
  short:
    "Чтобы получить сумму прямоугольника, берём большую сумму, из которой вырежем лишнее.",
  formal:
    "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
  complexity: "запрос  $O(1)$  после  $O(nm)$  предподсчёта",
  demo: "prefix-sum",
}
```

```

    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворо",
    title: "Поворот (rotation) в дереве",
    short:
        "Локальная перевеска двух узлов: ребёнок встаёт н
        дившемуся месту. Порядок ключей при этом не лом
    formal:
        "Правый поворот вокруг p: x = p.left; p.left = x.
        g и Zig-Zag.",
    complexity: "O(1)",
    demo: "zig",
    simulator: "splay-rotations",
},
{
    id: "zig",
    labels: ["Zig", "Zag", "зиг"],
    title: "Zig – одиночный поворот",
    short:
        "Самый простой случай: родитель x уже сидит в корн
        канчивается, если глубина была нечётной.",
    formal: "Применяется ровно один раз за всю операцию",
    demo: "zig",
    simulator: "splay-rotations",
},
{
    id: "zig-zig",
    labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
    title: "Zig-Zig – x и родитель с одной стороны",
    short:
        "Дерево вытянулось в «бамбук»: g → p → x идут в о
        м (p, x). Именно это вдвое сокращает глубину вс
    formal:
        "Если крутить наивно (два раза поднимать x), бамб
    demo: "zig-zig",
    simulator: "splay-rotations",
},
{
    id: "zig-zag",

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <p class="text-slate-300 text-sm mb-4">Пред
    оезда). Алгоритмы поиска кратчайшего пути п
    <div class="mt-8 mb-4">
      <div id="slot-mnemonic-cards"></div>
    </div>
  </div>
</div>
</section>`
},
{
  id: "dijkstra",
  title: "Билет 14. Дейкстра",
  type: "html",
  category: "Графы",
  content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 r
  <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-emerald-4

    <div class="bg-slate-900 p-4 rounded-lg mb-
      <p class="text-lg text-emerald-300 ital
      <p class="text-xl font-mono text-white"
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
        rder border-emerald-500 shadow-md">
        Аналогия 1: Позитивное плани

```

```
content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 r
  <h2 class="text-3xl font-bold text-white">Форд-Л
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-rose-400 m

    <div class="bg-slate-900 p-4 rounded-lg mb-4
      <p class="text-lg text-rose-300 italic m
      <p class="text-xl font-mono text-white"
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
        rder border-emerald-500 shadow-md">
          ♂ Аналогия 1: Параноик
        </div>
        <p class="text-slate-300 text-sm mb
        еряет ВСЕ возможные дороги V-1 раз подряд.
        <div id="slot-chapter-image-bellman
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
        border-rose-500 shadow-md">
          Отрицательный цикл
        </div>
        <p class="text-slate-300 text-sm mb
        г станет ЕЩЕ короче — значит мы попали во в
        </div>
      </div>
```



```

        <div class="bg-slate-900 p-6 rounded-lg"
          <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
            Время работы
          </div>
          <p class="text-slate-300 text-sm mb-2">
три вложенных цикла. Сложность  $O(V^3)$ . Если
          </p>
        </div>

        <div id="slot-floyd-viz" class="mt-8"></div>
      </div>
    </div>
  </section>`
  },
  {
    id: "aho-corasick",
    title: "Билет 23. Алгоритм Ахо-Корасик",
    type: "html",
    category: "Строки",
    content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1 rounded"
  <h2 class="text-2xl sm:text-3xl font-bold text-white"
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l
    <h3 class="text-xl font-bold text-blue-400 mb-4">Время работы

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text
      <p class="text-sm text-blue-300 italic mb-2">Осложнение
      <p class="text-lg font-mono text-white">Бор (Тр
    </div>
      <p class="text-slate-300 text-sm">Позволяет найти
        -mono text-amber-300 bg-black/30 px-1 rounded">Время
        y.</p>
    </div>
  `
  }
]

```

```

    bool is_terminal = false;
    vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {
        nodes[child].link = 0;
        q.push(child);
    }

    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto const& [ch, child] : nodes[v].trie) {
            // Суффиксная ссылка ребёнка – это переход к
            nodes[child].link = nodes[nodes[v].link].go

            // Если суффиксная ссылка сама является сло
            // Иначе наследуем terminal_link
            if (nodes[nodes[child].link].is_terminal) {
                nodes[child].term_link = nodes[child].l
            } else {
                nodes[child].term_link = nodes[nodes[ch
            }

            q.push(child);
        }
    }
}

```

```

    </div>
</div>
</details>
</div>
</section>`

```

```

"euler-path-vs-cycle": [
  {
    question: "В связном графе ровно 2 вершины с нечётными степенями. Что можно сказать о графе?",
    options: [
      "Есть эйлеров цикл (вернусь домой!)",
      "Есть эйлеров путь (из одной нечётной в другую)",
      "Ни пути, ни цикла нет — это полный провал"
    ],
    correctIndex: 1,
    explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если больше 2 — эйлеров пути нет, заканчиваем в другой."
  },
  {
    question: "Почему гамильтонов цикл — это NP-полная задача?",
    options: [
      "Потому что Гамильтон — болезнь, и лечить её тяжело",
      "Потому что для эйлерова цикла есть простой критерий",
      "Потому что Эйлер был умнее Гамильтона"
    ],
    correctIndex: 1,
    explanation: "Для эйлерова цикла работает критерий на чётности степеней. Для гамильтонова цикла — нет. Это NP-полная задача, которую не известно, как решить за полиномиальное время."
  },
  {
    question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
    options: [
      "Всё ещё можно пройти по всем рёбрам ровно раз",
      "Такого графа не существует",
      "Эйлерова пути нет — нужно удалять лишние рёбра"
    ],
    correctIndex: 2,
    explanation: "Если нечётных вершин больше 2, эйлерова пути нет. Можно удалить лишние рёбра, чтобы осталось 2 нечётные вершины."
  }
],
"bridges-code": [
  {
    question: "После DFS у нас low[u] = 4, а tin[v] = 5. Что это значит?"
  }
]

```

```

        correctIndex: 1,
        explanation: "E ≤ 3V - 6 ⇒ 12 ≤ 12 – на грани пр
            Так что такой граф непланарен."
    },
    {
        question: "Сколько граней у планарного графа с V=
        options: [
            "3 грани",
            "5 граней",
            "10 граней"
        ],
        correctIndex: 1,
        explanation: "F = E - V + 2 = 10 - 7 + 2 = 5. Не
    },
    {
        question: "Почему K5 непланарен?",
        options: [
            "Потому что 10 > 3*5 - 6 = 9 – нарушение",
            "Потому что Эйлер был против",
            "Потому что 5 вершин – несчастливое число"
        ],
        correctIndex: 0,
        explanation: "У K5: V=5, E=10, 3V - 6 = 9. 10 > 9
    }
}
];
};

```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 23/82: src/data/tickets/ticket1_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

```
import { Chapter } from '../../../types';
```


Код (Скрыто)

```

</summary>
<div class="p-5 text-sm text-slate-300">
  <pre class="bg-slate-950 p-4 rounded">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
  </div>
</details>
</div>
</div>
</section>`
},
{
  id: "treap",
  title: "3. Декартово дерево (Treap)",
  type: "html",
  description: "Дерево поиска по ключу и Куча по приоритету",
  category: "Продвинутые структуры",
  content: `
<section id="treap" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">Treap</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white">Декартово дерево</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">Описание</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Дерево поиска по ключу и Куча по приоритету</p>
        <p class="text-xl font-mono text-white">метода: Split и Merge.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Удар катаной (Sp
          </div>

```

```

{
  id: "splay-tree",
  title: "4. Splay-дерево",
  type: "html",
  description: "Дерево поиска, которое самобалансируется",
  category: "Продвинутые структуры",
  content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: VIP-лифт
          </div>
          <p class="text-slate-300 text-sm mb-4">
            вится корнем дерева (VIP-статус). Если ты член клуба,
            ка почти не требуется времени!</p>
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
              Аналогия 2: Балансировка черешка
            </div>
            <p class="text-slate-300 text-sm mb-4">
              и становиться кривым как бамбук. Но как толк
            </p>
          </div>
        </div>
      </div>
    </div>
  </div>
`

```

```

        находящимися на разных концах дерева, его
        ащая дерево в вытянутый бамбук для другого.
        . Постоянное свинг-переключение превратит пр
        </div>

        <div>
            <strong class="text-white block">
                <p>Хотя один поиск может стоять
                емах обладают прекрасным свойством: они не
                по пути. "Палочное" дерево прессуется в сб
                осов.</p>
            </div>
        </div>
    </div>
</div>
</section>`
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сжатие суммы подматрицы за  $O(1)$ ",
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-emerald-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-emerald-300 italic">
                    <p class="text-xl font-mono text-white">
                        , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
                </div>
            </div>
        </div>
    </div>
`
    }
}

```



```

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
  <h3 class="text-xl font-bold text-emerald-400">Аналогия 1
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-emerald-300 italic">Аналогия 1: Позитивное планирование
    <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
  </div>
  <div class="grid grid-cols-1 xl:grid-cols-2">
    <!-- Аналогия 1 -->
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
        Аналогия 1: Позитивное планирование
      </div>
      <p class="text-slate-300 text-sm mb-4">
        (нельзя поехать и заработать). Он всегда в
      </div>
    <!-- Аналогия 2 -->
    <div class="bg-slate-900 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
        Ограничения
      </div>
      <p class="text-slate-300 text-sm mb-4">
        дный и не умеет пересматривать уже "зафиксир
      </div>
    </div>
    <div id="slot-dijkstra-viz" class="mt-8"></div>
  </div>
</div>
</section>`
  },
  {
    id: "bellman-ford",
    title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="bellman-ford-content" class="mb-12 scroll-l

```

```

category: "Графы. Пути",
content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>

      <div class="bg-slate-800 p-6 rounded-lg border">
        <div class="absolute -top-3 left-4 bg-slate-900
          border-emerald-500 shadow-md">
          Суть фокуса (По шагам)
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Главный герой – <b>внешний цикл</b>
          шаг <code class="bg-slate-900 text-pink-400">
          шу себе проходить через вершину k?</i>
        </p>
        <ul class="text-sm text-slate-300 space-y-2">
          <li><b>Шаг k=0:</b> Ты знаешь только
          <li><b>Шаг k=1:</b> Разрешаем пересадки
          е) через путь <code class="text-indigo-300">
          <li><b>Шаг k=2:</b> Разрешаем пересадки
          нутри этих путей уже могут быть пересадки ч
          <li>...</li>
          <li><b>Шаг k=N:</b> Ты проверил все
        </li>
      </ul>
    </div>
  </div>
</div>
`

```

```

        </ol>
      </div>
    </div>
    <div id="slot-johnson-viz" class="mt-8"></div>
  </div>
</section>`
},
{
  id: "mst-kruskal",
  title: "18. Графы. Остовное дерево. Краскал",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Прокладка интернет
          </div>
          <p class="text-slate-300 text-sm mb-4">
            с самых дешевых дорог в стране". Он строит
            единую сеть.</p>
        </div>
      </div>
    </div>
  </div>
`

```

```

{
  id: "mst-boruvka",
  title: "20. Графы. Остовное дерево. Борувка",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-4">
            изкому племени для объединения. К вечеру пле
            ства шлют гонцов к ближайшему чужому царству.
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`
}
];

```

```

x</code>. Такой алгоритм начнет искать заголовок
</code>, а это начало нашего слова! Не надо

```

```

</p>

```

```

</div>

```

```

<!-- Аналогия 2 -->

```

```

<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; padding: 10px;">

```

```

  <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">
    Аналогия 2: LLM стриминг
  </div>

```

```

  <p class="text-slate-300 text-sm mb-0">
    Для LLM, которая стримит токены
  </p>

```

```

  <p class="text-slate-300 text-sm mb-0">
    Для LLM, которая стримит токены
    на каждом шаге. Если мы частично совпали с заголовком
    какого-либо места этого слова мы можем продолжить
  </p>

```

```

  </div>

```

```

</div>

```

```

</div>

```

```

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; padding: 10px;">

```

```

  <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #000; padding: 5px; margin-bottom: 5px;">
    Пример вычисления (Скрыто)
  </summary>

```

```

  <div class="p-5 text-sm text-slate-300" style="border: 1px solid #000; padding: 5px;">

```

```

    <p>Тот же пример: <b>abcabcd</b></p>

```

```

    <ul class="list-disc pl-5 space-y-2" style="list-style-type: none; padding-left: 20px;">

```

```

      <li><b>a</b>: Префиксов нет. <code></code></li>

```

```

      <li><b>ab</b>: Из начала ("a") не совпало. <code></code></li>

```

```

    </ul>

```

```

      <li><b>abc</b>: Опять мимо. <code></code></li>

```

```

      <li><b>abca</b>: О! Начало ("a") совпало. <code></code></li>

```

```

    </div>

```

```

  </div>

```

```

</code></li>

```

```

      <li><b>abcab</b>: Начало ("ab") совпало. <code></code></li>

```

```

    </li>

```

```

      <li><b>abcabc</b>: Начало ("abc") совпало. <code></code></li>

```

```

    </li>

```

```

      <li><b>abcabcd</b>: "d" всё испробовано. <code></code></li>

```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-emerald-300 italic">
    <p class="text-xl font-mono text-white">
      ca 0) и её суффиксом, начинающимся с i.</p>
    </p>
  </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Линейка-шаблон
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Тут мы берём всю строку и «приклеиваем» шаблон.
      Я начну читать строку отсюда, насколько длинна строка.
      Это самый быстрый способ найти искомую строку.
      >, считаешь Z-функцию, и там, где <code>Z[i]</code>
    </p>
  </div>
```

```
  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Аналогия 2: LLM Самокопираст
    </div>
    <p class="text-slate-300 text-sm mb-4">
      В LLM Z-функция может применяться для поиска
      <code>[i]</code> (где <i>i</i> указывает назад на предыдущий индекс)
      сама себя.
    </p>
  </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg mb-4">
  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50">
```

```
    type: "html",
    category: "Строки",
    content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-l">
      <h3 class="text-xl font-bold text-blue-400 mb-4">

      <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
        <p class="text-sm text-blue-300 italic mb-2">0с
        <p class="text-lg font-mono text-white">Бор (Тр
      </div>
      <p class="text-slate-300 text-sm">Позволяет найти
        го один проход.</p>
    </div>

    <!-- Аналогии -->
    <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg border bo">
        <div class="absolute -top-3 left-4 bg-slate-700
          emerald-500 shadow-md">
          Аналогия 1: Паутина (Бор)
        </div>
        <p class="text-slate-300 text-sm mb-4">Представ
          ет – мы падаем по <b>суффиксной ссылке</b> ("
        </div>

      <div class="bg-slate-900 p-6 rounded-lg border-2">
        <div class="absolute -top-3 left-4 bg-rose-900
          -500 shadow-md">
          Аналогия 2: Матёшка (Терминальные)
        </div>
        <p class="text-slate-300 text-sm mb-4">Представ
```

```

<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-purple-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-purple-300 italic">
      <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Судoku (P vs NP)
        </div>
        <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
      </div>
      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
          Аналогия 2: Машина-переводчик
        </div>
        <p class="text-slate-300 text-sm mb-4">
ь отличный переводчик на английский (Сведенн
аче B, то B – это <b>NP-Полная</b> (сосредо
      </div>
    </div>
  </div>
</div>
</section>`
}

```



```

        <p class="text-slate-300 text-sm mb-4">
            глупо, но проверка "знает ли Вася Петю" мегаломан.
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 border border-rose-500 shadow-md">
                Аналогия: Списки = Контакты
            </div>
            <p class="text-slate-300 text-sm mb-4">
                т. Оптимально для почти всех задач.</p>
        </div>
    </div>

    { /* DFS vs BFS */ }
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
        <h3 class="text-xl font-bold text-indigo-400">DFS vs BFS</h3>

        <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
            <!-- Аналогия DFS -->
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 right-4 border border-indigo-500 shadow-md">
                    ♂ DFS (Поиск в глубину) = Жадный алгоритм
                </div>
                <p class="text-slate-300 text-sm mb-4">
                    <b>Что делает:</b> Жадный алгоритм ищет первую доступную
                    вершину (например, минимальную по номеру) и пробует другой путь.
                </p>
                <ul class="text-xs text-indigo-300">
                    <li><b>Где используется:</b></li>
                    <li>Топологическая сортировка (линейное время)</li>
                    <li>Поиск мостов и точек сочленения</li>
                    <li>Сильно связанные компоненты Киркпатрика</li>
                    <li>Поиск циклов и просто проверка связности</li>
                </ul>
            </div>
        </div>
    </div>

```

```

<pre class="bg-slate-950 p-3 rounded-
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
}</pre>

</div>
</details>
</div>
</div>
</section>`,
},
{
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
<div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
</div>
<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
        <h3 class="text-xl font-bold text-blue-400 m
        <div class="bg-slate-900 p-4 rounded-lg mb-4
            <p class="text-lg text-blue-300 italic m

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Острова
    </div>
    <p class="text-slate-300 text-sm mb
карту — остались ли серые (неисследованные)
ь через океан — столько и компонент.</p>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-blue-400 m
      <div class="bg-slate-900 p-4 rounded-lg mb-1
        <p class="text-lg text-blue-300 italic m
        <p class="text-xl font-mono text-white"
перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2"
        <div class="bg-slate-800 p-6 rounded-lg"
          <div class="absolute -top-3 left-4 l

```

```
void topological_sort(int n) {
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
    reverse(ans.begin(), ans.end()); // Разворачиваем с
}
```

</details>

</section>`

4.

f

```
id: "scc-kosaraju",
```

```
title: "10. Графы. Компоненты сильной связности (SC)"
```

```
type: "html",
```

description: "Разбиение орграфа на макро-вершины. А

category: "Графы. Продвинутое",

content: `

<section id="scc-kosaraju" class="mb-12 scroll-mt-10">

```
<div class="flex items-center mb-6 flex-wrap gap-3":
```

```
<span class="bg-blue-600 text-white px-4 py-1 rounded" data-bbox="115 100 400 115">
```

class="text-2xl sm:text-3xl font-bold text-violet-600"> 1. Introduction

<div class="space-y-8">

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
```

class="text-xl font-bold text-emerald-400">

<div class="bg-slate-900 p-4 rounded-lg mb-

Text-300

<div class="text-left font-mono text-sm

Kompon

и *ориентированного* графа, в котором

```

        <details class="bg-slate-900/40 rounded-lg border-1
            <summary class="p-4 font-bold text-slate-300
                -b border-slate-700/50">
                    Душный мод: Код Алгоритм Косарайю
            </summary>
            <div class="p-5 text-sm text-slate-300
                <p class="text-emerald-400 font-bold
                <ol class="list-decimal list-inside
                    <li>Запускаем DFS на исходном графе
                </li>
                    <li>Разворачиваем этот список (только для не-мостов)
                    <li>Переворачиваем все ребра в обратном направлении
                    <li>Идем по <code>order</code>:
                </li>
            </ol>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "graph-bridges",
        title: "11. Графы. Мосты",
        type: "html",
        description: "Рёбра, удаление которых расхреначивает граф",
        category: "Графы. Продвинутое",
        content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border-1">
                <h3 class="text-xl font-bold text-rose-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

        <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</strong>
        <div class="p-3 bg-slate-950 rounded-lg border-rose-500 shadow-md">
            <span class="text-rose-400 font-weight-bold">Точка
            <p class="text-xl font-bold text-white">Точка
            <p class="text-xs text-slate-400">Точка
        </div>
    </div>
</div>

<p class="text-slate-300 text-sm">
    <strong>Важно:</strong> Это работает только для точек, не являющихся
    епята к точкам сочленения. То есть, <strong>Точка сочленения</strong>
    ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
    <div class="bg-slate-800 p-6 rounded-lg border-rose-500 shadow-md">
        <div class="absolute -top-3 left-4 bg-slate-900 border-rose-500 shadow-md">
            Аналогия: Главный перекресток
        </div>
        <p class="text-slate-300 text-sm mb-4">
            Представь город, где два района соединены мостом. Если
            её перекроют (удалят вершину) – районы будут изолированы.
            <strong>Хрупкость</strong> системы.
        </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-emerald-500 shadow-md">
        <div class="absolute -top-3 left-4 bg-slate-950 border-emerald-500 shadow-md">
            Телеком: Центральный свитч
        </div>
        <p class="text-slate-300 text-sm mb-4">
            У тебя несколько компьютеров в одной сети. Если один из них
            абелем (мостом). Сами свитчи – это точки сочленения.
        </p>
    </div>
</div>
```

```

        </div>
      </div>
    </details>

    <div class="bg-indigo-950/60 p-6 rounded-xl border"
      <h4 class="text-lg font-bold text-indigo-400">
      <p class="text-slate-300 text-sm mb-4">
        Это глубокий дуальный мост между ориентиром
      </p>
      <ul class="list-disc list-inside text-sm text-slate-300">
        <li><strong class="text-rose-400">Точки
        <strong> и показывают физическую уязвимость связности.
        <li><strong class="text-emerald-400">SCC
        зависимые "конгломераты".</li>
        <li><strong>Как точка сочленения рушит связность?
        Косарайю, свернув каждую SCC в одну супер-вершину
        (сделаем неориентированным), то любая <strong>
        о и есть критическая SCC! Если удалить эту SCC,
        . Одно слабое звено изолирует целые касты SCC.
      </ul>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">

```

```

    y: number;
    label: string;
}

interface Edge {
    u: number;
    v: number;
}

const nodes: Node[] = [
    { id: 0, x: 250, y: 50, label: "0" },
    { id: 1, x: 100, y: 150, label: "1" },
    { id: 2, x: 250, y: 250, label: "2" },
    { id: 3, x: 400, y: 150, label: "3" },
    { id: 4, x: 550, y: 150, label: "4" },
    { id: 5, x: 700, y: 50, label: "5" },
    { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
    { u: 0, v: 1 },
    { u: 1, v: 2 },
    { u: 2, v: 0 },
    { u: 2, v: 3 },
    { u: 3, v: 4 }, // 3 and 4 are articulation points (3
    { u: 4, v: 5 },
    { u: 5, v: 6 },
    { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
    { length: Object.keys(nodes).length },
    () => [],
);

edges.forEach((e) => {
    adj[e.u].push(e.v);
    adj[e.v].push(e.u);
});

```



```

    visited.add(v);
    currentTin[v] = currentLow[v] = timer++;
    let children = 0;

    recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);

    for (const to of adj[v]) {
        if (to === p) continue;

        if (visited.has(to)) {
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);
            recordStep(
                `Обратное ребро ${v}-${to}. Обновляем low[${v}]`
            );
        } else {
            children++;
            dfs(to, v);
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);

            recordStep(
                `Возврат в ${v} из ${to}. Обновляем low[${v}]`
            );

            if (currentLow[to] >= currentTin[v] && p !== -1) {
                ap.add(v);
                recordStep(
                    `Найдена точка сочленения: вершина ${v}`
                );
            }
        }
    }
}

if (p === -1 && children > 1) {
    ap.add(v);
    recordStep(

```

```

    }
    return () => clearInterval(interval);
  }, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIndex(0);
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
      >
        <RotateCcw className="w-4 h-4" />

```

```

        : isAp
        ? "#e11d48"
        : isVisited
        ? "#10b981"
        : "#1e293b";
const stroke = isCurrent
? "#60a5fa"
: isAp
? "#f43f5e"
: isVisited
? "#34d399"
: "#334155";
const r = isAp ? 24 : 20;

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r={r}
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-all duration-100"
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events-none"
    >
      {node.label}
    </text>
  </g>
)

```

```

<div className="bg-slate-900 border border-slate-800" data-bbox="233 118 1000 235">
  <p className="text-white font-medium min-h-[100px]" data-bbox="273 148 1000 193">
    {currentStep.desc}
  </p>
</div>

```

```

<div className="space-y-4 flex-1 overflow-y-auto" data-bbox="233 261 1000 734">
  <div className="mb-4" data-bbox="273 291 1000 734">
    <h5 className="text-xs text-slate-500 font-medium" data-bbox="303 311 1000 331">
      <div className="flex items-center gap-2" data-bbox="303 336 1000 426">
        <div className="w-3 h-3 rounded-full bg-rose-500" data-bbox="338 361 608 401">
          Точка сочленения
        </div>
        <div className="flex items-center gap-2" data-bbox="303 431 1000 521">
          <div className="w-3 h-3 rounded-full bg-rose-500" data-bbox="338 456 608 496">
            Посещена
          </div>
          <div className="flex items-center gap-2" data-bbox="303 526 1000 616">
            <div className="w-3 h-3 rounded-full bg-rose-500" data-bbox="338 551 608 591">
              Текущая
            </div>
            <div className="flex items-center gap-2" data-bbox="303 621 1000 711">
              <div className="w-8 h-1 bg-rose-500 border border-rose-400" data-bbox="338 646 1000 686">
                Мост
              </div>
            </div>
          </div>
        </div>
      </div>

```

```

<div>
  <h5 className="text-xs text-slate-500 font-medium" data-bbox="303 786 1000 856">
    ЛОГ ДЕЙСТВИЙ:
  </h5>
  <div className="space-y-2" data-bbox="303 861 1000 996">
    {steps
      .slice(0, currentStepIndex + 1)
      .reverse()
      .map((step, idx) => (
        <div

```

```

checkingEdge: { u: string; v: string; w: number } | null;
distances: { [key: string]: number };
previous: { [key: string]: string | null };
log: string;
hasNegativeCycle?: boolean;
activeCodeLine: number;
}

```

```

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

```

```

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

```

```

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
      { u: 'C', v: 'E', w: 4 },
      { u: 'D', v: 'F', w: 2 },
      { u: 'E', v: 'F', w: 1 },
    ];
    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
    } else {

```

```

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
  let anyUpdate = false;
  for (const edge of edges) {
    const uDist = dist[edge.u];
    const vDist = dist[edge.v];

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
      dist = { ...dist, [edge.v]: uDist + edge.w };
      prev = { ...prev, [edge.v]: edge.u };
      updated = true;
      anyUpdate = true;
    }

    simulationSteps.push({
      iteration: iter, checkingEdge: { u: edge.u, v
      distances: { ...dist }, previous: { ...prev }
      log: `Итерация ${iter}: Проверяем ${edge.u} →
        updated ? ` Релаксация успешна! dist[${edge
      }`,
      activeCodeLine: updated ? 6 : 5,
    });
  }

  if (!anyUpdate && iter < vCount - 1) {
    simulationSteps.push({
      iteration: iter, checkingEdge: null, distance
      log: ` Ранний выход после итерации ${iter}.
      activeCodeLine: 10
    });
    break;
  }
}

if (simulationSteps[simulationSteps.length - 1].ite
  let cycleFound = false;

```

```

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border"
    <div className="flex flex-wrap items-center justify"
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-blue-600 text-white"
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white"
            <p className="text-sm text-blue-300">Полный
          </p>
        </div>
      </div>

      <div className="flex items-center gap-3">
        <div className="bg-slate-900 p-1 rounded-lg border"
          <button onClick={() => setHasCycle(false)}
            bg-blue-600 text-white' : 'text-slate-400'
          <button onClick={() => setHasCycle(true)}
            r gap-1 ${hasCycle ? 'bg-rose-600 text-white'
        </div>
        <button onClick={() => { setIsPlaying(false);
          g-slate-600 text-white px-3 py-2 rounded-lg
        <button onClick={() => setIsPlaying(!isPlaying
          sition-colors text-white ${isPlaying ? 'bg-
        <button onClick={() => { setIsPlaying(false);
          ; }} className="flex items-center gap-1 bg-l
          s">⏏ </button>
        </div>
      </div>

    <div className="flex flex-col lg:flex-row gap-6 m
      {/* Граф */}
      <div className="w-full lg:w-2/3 bg-blue-950/20
        y-center min-h-[400px]">
        <div className="absolute top-3 left-3 bg-blue

```

```

asNegativeCycle && isCyclePart) ? 4 : 2} ma
g ? '4,4' : 'none'} />
    <circle cx={({uNode.x + vNode.x) / 2}
Checking ? '#f59e0b' : isNeg ? '#f43f5e' :
    <text x={({uNode.x + vNode.x) / 2} y={
8fafc'} fontSize="11" fontWeight="bold" tex
    </g>
  });
}})

{nodes.map((node) => {
  const dist = currentStep.distances[node.id]
  const isCheckingNode = currentStep.checkin
node.id);

  return (
    <g key={node.id} className="transition-
      <circle cx={node.x} cy={node.y} r={is
ckingNode ? '#7dd3fc' : '#64748b'} strokeWi
      <text x={node.x} y={node.y + 5} fill=
      <text x={node.x} y={node.y - 28} fill
ight="bold" textAnchor="middle" className="
      <text x={node.x} y={node.y + 35} fill
split(' ')[0]}</text>
    </g>
  );
}})
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
  <p className="font-semibold text-amber-400
  <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
  <h4 className="text-lg font-bold text-emerald

```



```
        </div>
      </div>
    </div>

    {/* Код алгоритма */}
    <div className="w-full lg:w-1/2 bg-slate-900/60" >
      <div>
        <h4 className="text-lg font-bold text-slate-100">Алгоритм
        <div className="bg-slate-950/80 rounded-lg p-4" >
          {codeLines.map(item => (
            <div key={item.line} className={`py-1.5 px-3
            e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'}`>
              <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
              <span className="whitespace-pre flex-grow-1">{item.code}</span>
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
</div>
);
}
```

ФАЙЛ 29/82: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useCallback } from 'react';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';
```

```
const NODES = [
  { id: 0, label: '0', x: 300, y: 50 },
  { id: 1, label: '1', x: 150, y: 150 },
  { id: 2, label: '2', x: 450, y: 150 },
  { id: 3, label: '3', x: 75, y: 250 },
```

```

visited.add(0);
steps.push({
  activeLine: 2,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Добавляем стартовую вершину 0 в очередь и по
});

while (queue.length > 0) {
  steps.push({
    activeLine: 4,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: `Очередь не пуста, элементов: ${queue.length}
  });

  const curr = queue.shift()!;
  steps.push({
    activeLine: 5,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: curr,
    logs: `Достаем из очереди (${curr}).`
  });

  const neighbors = ADJ[curr];
  if (neighbors.length > 0) {
    for (const n of neighbors) {
      if (!visited.has(n)) {
        steps.push({
          activeLine: 7,
          queue: [...queue],
          visited: Array.from(visited),
          currentNode: curr,
          logs: `Рассматриваем соседа: ${n}. Он еще не
        });
      }
    }
  }
}

```

```

const [stepIdx, setStepIdx] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

const step = STEPS[stepIdx];

const handleNext = useCallback(() => {
  setStepIdx(prev => Math.min(prev + 1, STEPS.length
  }, []));

// @ts-expect-error зарезервировано под кнопку «назад»
const handlePrev = useCallback(() => {
  setStepIdx(prev => Math.max(prev - 1, 0));
}, []);

useEffect(() => {
  if (!isPlaying) return;
  if (stepIdx >= STEPS.length - 1) {
    setIsPlaying(false);
    return;
  }
  const timer = setTimeout(handleNext, 1200);
  return () => clearTimeout(timer);
}, [isPlaying, stepIdx, handleNext]);

return (
  <div className="bg-slate-900 border-2 border-slate-
    <div className="flex flex-col md:flex-row items-c
      <div>
        <h3 className="text-2xl font-bold text-white
          <span className="text-blue-400"> </span> An
        </h3>
        <p className="text-slate-400 text-sm mt-1">Вн
      </div>
      <div className="flex bg-slate-800 p-1.5 rounded
        <button
          onClick={() => { setIsPlaying(false); setSt
          className="p-2 text-slate-400 hover:text-wh
          title="C6poc"

```

```

        />
    });
  }}}

  { /* Вершины */
  { NODES.map(n => {
    const isCurrent = step.currentNode === n.id;
    const isInQueue = step.queue.includes(n.id);
    const isVisited = step.visited.includes(n.id);

    let fill = '#1e293b'; // slate-800
    let stroke = '#475569'; // slate-600
    let scale = 1;
    void scale;

    if (isCurrent) {
      fill = '#3b82f6'; // blue-500
      stroke = '#60a5fa'; // blue-400
      scale = 1.3;
    } else if (isInQueue) {
      fill = '#059669'; // emerald-600
      stroke = '#34d399'; // emerald-400
      scale = 1.1;
    } else if (isVisited) {
      fill = '#0f172a'; // slate-900
      stroke = '#3b82f6'; // blue-500
    }

    return (
      <g key={n.id} className="transition-transform"
        {isCurrent && {
          <circle cx={n.x} cy={n.y} r="30" fill={fill} stroke={stroke}
        }}
        <circle
          cx={n.x} cy={n.y}
          r="20"
          fill={fill}
          stroke={stroke}

```

```

        const isActive = step.activeLine === cl
        return (
          <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5' : 'te
          <span className="opacity-40 w-6 inl
          <span className="whitespace-pre">{c
          </div>
        );
      }}}
    </div>
  </div>

  <div className="bg-slate-800/80 border border
    <div className="text-sm font-bold text-eme
      <span>Состояние Очереди (Queue):</span>
      <span className="text-xs text-slate-500
    </div>
    <div className="flex items-center gap-2 mi
      {step.queue.length === 0 ? (
        <span className="text-slate-500 text-x
      ) : (
        step.queue.map(({qId, i}) => (
          <span key={`_${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
            {qId}
          </span>
        ))
      )}
    </div>
    <div className="mt-2 text-sm text-amber-300
ms-center justify-center">
      "{step.logs}"
    </div>
  </div>
</div>
</div>
</div>
);

```

```
    if (!info) return null;

    return (
      <div>
        <div className={`rounded-2xl overflow-hidden border`>
          <img src={info.src} alt={info.alt} className="w-full h-40" />
        </div>
        <p className={`text-center text-xs mt-2 font-bold`}>{info.title}</p>
      </div>
    );
  }
}
```

ФАЙЛ 31/82: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  compact?: boolean;
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» — как на Codecademy
```

```
 * Компактный вариант живёт в шапке главы, крупный — по центру
```

```
 */
```

```
export const ChapterNav: React.FC<Props> = ({ prev, next, onGo, compact }) => {
  if (compact) {
    return (

```

```

        <ChevronLeft className="w-3.5 h-3.5" /> Пре
      </div>
      <div className="text-sm font-bold text-slate-1
    </button>
  ) : (
    <div className="hidden sm:block" />
  ))

  {next && (
    <button
      onClick={() => onGo(next.id)}
      className="group text-right p-4 rounded-xl bg
        transition-all sm:col-start-2"
    >
      <div className="flex items-center justify-end
        text-indigo-400 mb-1">
        Следующая тема <ChevronRight className="w-3
      </div>
      <div className="text-sm font-bold text-slate-1
    </button>
  )}
</nav>
);
};

```

ФАЙЛ 32/82: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИ

```

import React, { useCallback, useEffect, useRef, useState
import { Check, Download, Loader2, Maximize2, MousePoin
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/T
import { getViz } from "../vizRegistry";
import { ChapterNav } from "../ChapterNav";

```

```

const saveHtml = useCallback(async () => {
  setSaving("work");
  try {
    await downloadChapterHtml(chapter);
    setSaving("done");
    window.setTimeout(() => setSaving("idle"), 2500);
  } catch {
    setSaving("idle");
  }
}, [chapter]);

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
    cancelHide();
    setAnchor({ termId, rect: el.getBoundingClientRect() });
  },
  [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
  if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
  if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
  if (!el) return;
  e.preventDefault();
};

```



```

        <Printer className="w-3.5 h-3.5" /> Печать
      </button>
      <span className="text-[11px] text-slate-500">
    </div>

```

```

<div
  ref={contentRef}
  className="prose prose-invert max-w-none"
  onMouseOver={handlePointerOver}
  onMouseOut={handlePointerOut}
  onClick={handleClick}
  onFocus={handleFocus}
  dangerouslySetInnerHTML={{ __html: chapter.content }}
/>

```

```

<p className="mt-6 flex items-start gap-2 text-slate-500">
  <MousePointerClick className="w-4 h-4 shrink-0" />
  <span>
    Подчёркнутые термины – интерактивные: наведение,
    объяснение на пальцах, мини-анимацию и кнопки.
  </span>
</p>

```

```

{viz && (
  <section id="chapter-viz" className="mt-8 scroll-mt-16">
    <div className="flex items-center justify-between">
      <h3 className="flex items-center gap-2 text-slate-500">
        <Sparkles className="w-4 h-4 text-indigo-500" />
        Демонстрация: {viz.title}
      </h3>
      <button
        onClick={() => onOpenSimulator(chapter.title)}
        className="flex items-center gap-1.5 text-slate-300 hover:text-white hover:border-indigo-500"
      >
        <Maximize2 className="w-3.5 h-3.5" /> P
      </button>
    </div>
  </section>
)
}

```

```

{ line: "void dfs(int v, int p = -1) {", highlight: "normal" },
{ line: "    visited[v] = true;", highlight: "normal" },
{ line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
{ line: "    for (int to : adj[v]) {", highlight: "normal" },
{ line: "        if (to == p) continue;", highlight: "normal" },
{ line: "        if (visited[to]) {", highlight: "normal" },
{ line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
{ line: "        } else {", highlight: "normal" },
{ line: "            dfs(to, v);", highlight: "normal" },
{ line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
{ line: "        }", highlight: "normal" },
{ line: "        if (low[to] > tin[v]) {", highlight: "normal" },
{ line: "            // ЭТО МОСТ!", highlight: "normal" },
{ line: "        }", highlight: "normal" },
{ line: "    }", highlight: "normal" },
{ line: "}", highlight: "normal" },
};

```

```

interface DfsStep {
    currentNode: number | null;
    visitedIndices: number[];
    tin: Record<number, number>;
    low: Record<number, number>;
    stack: number[];
    bridges: string[];
    log: string;
    activeEdge: [number, number] | null;
    backEdge: [number, number] | null;
    activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
    { id: 0, label: "A", x: 100, y: 120 },
    { id: 1, label: "B", x: 260, y: 120 },
    { id: 2, label: "C", x: 180, y: 200 },

```

```

    },
    {
        currentNode: 1, visitedIndices: [0,1,6],
        tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
        activeEdge: null, backEdge: null,
        log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
        activeCodeLine: [13,14,15]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
        activeEdge: [1,2], backEdge: null,
        log: "Из B идём в C. tin[C]=4, low[C]=4.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
        activeEdge: null, backEdge: [2,0],
        log: "Видим соседа A (уже посещён)! Обратное ребро ",
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3],
        tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
        activeEdge: [2,3], backEdge: null,
        log: "Идём C→D. tin[D]=5, low[D]=5.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:2,4:3},
        activeEdge: [3,4], backEdge: null,
        log: "D→E. tin[E]=6, low[E]=6.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],

```

```

        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1},
        activeEdge: null, backEdge: null,
        log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм завершён!",
        activeCodeLine: [11,13,16,17,18]
    },
    {
        currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1},
        activeEdge: null, backEdge: null,
        log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм завершён!",
        activeCodeLine: []
    },
    ],
];

export function DfsBridgesSimulator() {
    const [dfsIdx, setDfsIdx] = useState(0);
    const [dfsAuto, setDfsAuto] = useState(false);
    const dfsTimer = useRef<NodeJS.Timeout | null>(null);

    useEffect(() => {
        if (dfsAuto) {
            dfsTimer.current = setInterval(() => {
                setDfsIdx(prev => {
                    if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
                    return prev + 1;
                });
            }, 2500);
        }
        return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
    }, [dfsAuto]);

    const step = DFS_STEPS[dfsIdx];

    return {

```

```

        (step.activeEdge[0]===e.u && step.active
        (step.activeEdge[0]===e.v && step.active
    );
    const activeBack = step.backEdge && {
        (step.backEdge[0]===e.u && step.backEdge
        (step.backEdge[0]===e.v && step.backEdge
    );
    let sc = "#475569", sw = 2, dash = "none"
    if (bridge) { sc = "#eab308"; sw = 4; }
    else if (activeTree) { sc = "#10b981"; sw
    else if (activeBack) { sc = "#f43f5e"; sw
    else if (step.visitedIndices.includes(e.u
    return <line key={e.key} x1={u.x} y1={u.y
        stroke={sc} strokeWidth={sw} strokeDash
        className="transition-all duration-500"
    }}}
    {step.currentNode !== null && step.currentN
        const n = GRAPH_NODES.find(x => x.id ===
        if (!n) return null;
        return <circle cx={n.x} cy={n.y} r={18} f
    >;
    })()}
    {GRAPH_NODES.map(n => {
        const cur = step.currentNode === n.id;
        const vis = step.visitedIndices.includes(n
        const st = step.stack.includes(n.id);
        return <g key={n.id}>
            <circle cx={n.x} cy={n.y} r={12}
                fill={cur ? "#10b981" : st ? "#064e3b
                stroke={cur ? "#fff" : st ? "#34d399"
                strokeWidth={2.5}
                className="transition-all duration-300
            <text x={n.x} y={n.y+3} textAnchor="mid
        label}</text>
            {vis && (
                <
                <rect x={n.x-18} y={n.y-28} width={
            "transition-all duration-300" />

```



```

    { u: 'B', v: 'D', w: 5 },
    { u: 'G', v: 'D', w: 1 },
    { u: 'G', v: 'E', w: 2 },
    { u: 'D', v: 'F', w: 3 },
    { u: 'E', v: 'F', w: 1 },
  ];

```

```

const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
  adjList[e.u].push({ v: e.v, w: e.w });
});

```

```

const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '  dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '  pq = PriorityQueue(); pq.push(start, 0)' },
  { line: 4, text: '  while not pq.empty():' },
  { line: 5, text: '    d, u = pq.pop() // Извлеките элемент с минимальным расстоянием' },
  { line: 6, text: '    if d > dist[u]: continue' },
  { line: 7, text: '    for v, weight in graph.neighbours(u):' },
  { line: 8, text: '      if dist[u] + weight < dist[v]:' },
  { line: 9, text: '        dist[v] = dist[u] + weight' },
  { line: 10, text: '        pq.push(v, dist[v])' },
  { line: 11, text: '  return dist // Все кратчайшие пути найдены' },
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false, B: false };
  const prev: Record<string, string | null> = { A: null, B: null };

  simulationSteps.push({

```

```

        if (dist[u] + edge.w < dist[v]) {
            dist[v] = dist[u] + edge.w;
            prev[v] = u;
            simulationSteps.push({
                currentNode: u, checkingEdge: { u, v },
                distances: { ...dist }, visited: { ...visited },
                log: `    Улучшение (Релаксация)! Новое кратчайшее расстояние: ${dist[v]},
                activeCodeLine: 9,
            });
        }
    }
}

simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: `    Дейкстра всё доставил! Все возможные кратчайшие расстояния найдены!
    activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => {
            setCurrentStepIndex((prev) => prev + 1);
        }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

```



```

        Шаг
      </button>
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 md:gap-8" >
    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-indigo-950/20 text-white font-
      stify-center min-h-[400px]" >
      <div className="absolute top-3 left-3 bg-indigo-950/50 text-white font-
        ont-bold shadow-sm" >
        Шаг {currentStepIndex + 1} из {steps.length}
      </div>

      <svg className="w-full h-auto max-h-[500px]" >
        <defs>
          <marker id="arrow-dh-normal" markerWidth="10" markerHeight="10"
            <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
          </marker>
          <marker id="arrow-dh-active" markerWidth="10" markerHeight="10"
            <path d="M0,0 L6,3 L0,6 z" fill="#f59e00" />
          </marker>
          <marker id="arrow-dh-opt" markerWidth="6" markerHeight="6"
            <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
          </marker>
        </defs>
        { /* Рёбра */ }
        {edges.map((edge, idx) => {
          const uNode = nodes.find((n) => n.id === edge.u);
          const vNode = nodes.find((n) => n.id === edge.v);
          const isChecking =
            currentStep.checkingEdge === edge.id &&
            (currentStep.checkingEdge.u === edge.u && currentStep.checkingEdge.v === edge.v);
          const isOptimal = currentStep.previous[edge.u] === vNode;

          let marker = 'url(#arrow-dh-normal)';
          if (isChecking) marker = 'url(#arrow-dh-active)';
          if (isOptimal) marker = 'url(#arrow-dh-opt)';
          return (
            <g key={idx} >
              <rect x={uNode.x} y={uNode.y} x2={vNode.x} y2={vNode.y}
                className={isChecking ? 'stroke-indigo-950' : 'stroke-white'}
                stroke-width={1} />
              <path d={`M${uNode.x},${uNode.y} L${vNode.x},${vNode.y}`}
                fill="none" stroke={isChecking ? 'white' : 'white'} stroke-width={1} />
              <marker
                className={isChecking ? 'fill-indigo-950' : 'fill-white'}
                href={marker}
                x={vNode.x} y={vNode.y} />
            </g>
          );
        })}
      </svg>
    </div>
  </div>

```

```

        cx={node.x} cy={node.y}
        r={isCurrent ? 22 : 18}
        fill={isCurrent ? '#4f46e5' : isVis
        stroke={isCurrent ? '#a5b4fc' : isV
        strokeWidth={isCurrent ? 4 : 2}
        className="transition-all duration-1
    />
    <text x={node.x} y={node.y + 5} fill=
        {node.id}
    </text>
    <text x={node.x} y={node.y - 28} fill=
    "middle" className="bg-indigo-950 px-1 py-0
        {dist === 999 ? '∞' : `d=${dist}`}
    </text>
    <text x={node.x} y={node.y + 35} fill=
        {node.name.split(' ')[0]}
    </text>
    </g>
    );
    }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 ro
    <p className="font-semibold text-amber-400
    <p className="min-h-[40px] flex items-cente
    </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <div className="space-y-2.5 flex-1 overflow-y
        {Object.keys(adjList).map((u) => {
            const isCurrentNode = currentStep.current
            return (
                <div key={u} className={`p-3 rounded-lg
                    isCurrentNode ? 'bg-emerald-900/60
                t-slate-300'
                }`>
            )
        })}
    </div>

```

```

        <th className="py-2 px-3">Вершина</th>
        <th className="py-2 px-3">Расстояние</th>
        <th className="py-2 px-3">Из (пред. шаг)</th>
      </tr>
    </thead>
    <tbody className="text-sm">
      {nodes.map((n) => {
        const dist = currentStep.distances[n.id]
        const prev = currentStep.previous[n.id]
        const vis = currentStep.visited[n.id]
        const isCurr = currentStep.currentNode === n

        return (
          <tr key={n.id} className={`border-1
            isCurr ? 'bg-indigo-950/60 font-medium' : ''}>
            <td className="py-2.5 px-3 flex items-center">
              <span className={`w-2 h-2 rounded-full ${vis ? 'bg-indigo-950/60' : 'bg-gray-300'} mr-2`}></span>
              {n.name}
            </td>
            <td className="py-2.5 px-3 font-medium">
              {dist === 999 ? '∞' : dist}
            </td>
            <td className="py-2.5 px-3 text-sm">
              {prev || '—'}
            </td>
          </tr>
        );
      })}
    </tbody>
  </table>
</div>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 p-10 rounded-2xl">
  <div>

```

```

-----
  memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "    if (n in memo) return memo[n];",
  /* 3 */ "    if (n <= 2) return 1;",
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo",
  /* 5 */ "    return memo[n];",
  /* 6 */ "}"
];

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |

  useEffect(() => {
    const generatedSteps: DPStep[] = [];
    let stepId = 0;
    const memoMap: { [key: number]: number } = {};

    function simulateFib(n: number): number {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вызов fibMemo(${n}). Проверяем наличие в
        codeLine: 2,
        memoState: { ...memoMap }
      });

      if (n in memoMap) {
        generatedSteps.push({
          id: stepId++,

```

```

        desc: `Сохраняем в кэш: мемо[${n}] = ${res1} + ${res2}`,
        codeLine: 5,
        memoState: { ...memoMap }
    });

    return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
    id: stepId++,
    n: targetN,
    action: 'done',
    desc: `Расчет завершен! fibMemo(${targetN}) = ${finalRes}`,
    codeLine: 6,
    memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => {
            setCurrentStepIndex(prev => prev + 1);
        }, speed);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

```

```

        >
          <RotateCcw className="w-4 h-4" />
        </button>
        <button
          onClick={() => { setIsPlaying(false); set
            disabled={currentStepIndex === 0}
            className="p-2 text-slate-400 hover:text-
            title="Шаг назад"
          >
            <ChevronLeft className="w-5 h-5" />
          </button>
          <button
            onClick={() => setIsPlaying(!isPlaying)}
            className={`flex items-center gap-1.5 px-
              isPlaying
                ? 'bg-amber-500 text-slate-950 hover:
                : 'bg-emerald-600 text-white hover:
              }`
          >
            {isPlaying ? <Pause className="w-4 h-4" />
              {isPlaying ? 'Пауза' : 'Пуск'}}
          </button>
          <button
            onClick={() => { setIsPlaying(false); set
              disabled={currentStepIndex === steps.leng
              className="p-2 text-slate-400 hover:text-
              title="Шаг вперед"
            >
              <ChevronRight className="w-5 h-5" />
            </button>
          </div>
        </div>
      </div>
    </div>

    {/* DP Table View */}
    <div className="mb-8 bg-slate-950 p-4 md:p-6 round
      <h4 className="text-xs font-bold uppercase trac
        <div className="flex flex-wrap items-center gap

```

```

{ /* Tabs */
<div className="flex items-center gap-2 border-b
  <button
    onClick={() => setActiveTab('table')}
    className={`flex items-center gap-2 px-6 py-3
      activeTab === 'table'
        ? 'border-emerald-500 text-emerald-400 bg
        : 'border-transparent text-slate-400 hove
    }`}
  >
    <Eye className="w-4 h-4" />
    Стен-бай-стен состояние
  </button>
  <button
    onClick={() => setActiveTab('code')}
    className={`flex items-center gap-2 px-6 py-3
      activeTab === 'code'
        ? 'border-emerald-500 text-emerald-400 bg
        : 'border-transparent text-slate-400 hove
    }`}
  >
    <Code className="w-4 h-4" />
    Интерактивный код
  </button>
</div>

```

```

{ /* Tab Content */
<div className="min-h-[320px] flex flex-col justifi
  {activeTab === 'table' && {
    <div className="bg-slate-950 p-6 rounded-2xl
      <h5 className="font-bold text-white text-sm
        <span className="text-emerald-400"> </span>
      </h5>
      <p className="text-sm text-slate-400">
        В классической рекурсии для N={targetN} п
        вычисленное значение в таблице, мы момента
      </p>

```

```

    })
  </div>

  {/* Progress Bar */}
  <div className="mt-8 pt-6 border-t border-slate-800" >
    <div className="flex items-center justify-between" >
      <span>Прогресс выполнения</span>
      <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
    </div>
    <div className="h-2 bg-slate-950 rounded-full overflow-hidden" >
      <div
        className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
        style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
      ></div>
    </div>
  </div>
);
}

```

ФАЙЛ 36/82: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },
  { id: 3, label: "D", x: 280, y: 250 },
  { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [

```



```

if (nextEdges.length === C1_EDGES.length) {
  const startDegree = C1_EDGES.filter(e => e.u === vId);
  const endDegree = C1_EDGES.filter(e => e.v === vId);
  const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
  if (isCycle && nextPath[0] === vId) {
    setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
  } else {
    setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С`);
  }
  setC1Done(true);
} else {
  setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $`);
}
};

```

```

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
          <Undo className="h-3.5 w-3.5" /> Сброс
        </button>
      </div>

      <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
        <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate_1px)] border border-slate-800">
          <svg className="w-full h-[260px] z-10 relative">
            {C1_EDGES.map((e, i) => {
              const u = C1_NODES.find(n => n.id === e.u);
              const v = C1_NODES.find(n => n.id === e.v);
              const key = `${Math.min(e.u, e.v)}-${Math.max(e.u, e.v)}`;
              const used = c1VisitedEdges.includes(key);
              return <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
                stroke={used ? "#6366f1" : "#475569"}
                strokeWidth={used ? 4 : 2}
              />
            })}
          </svg>
        </div>
      </div>
    </div>
  )

```

```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
  id: number;
  question: string;
  options: string[];
  correctAnswer: number;
  explanation: string;
}

const quizQuestions: Question[] = [
  {
    id: 1,
    question: 'Что мы делаем в алгоритме Краскала, если ребро уже в дереве (или оба конца уже в одном компоненте связности, красный) цвет?',
    options: [
      'Мы радуемся и добавляем его в минимальное остовное дерево.',
      'Мы перекрашиваем их в синий цвет и делим граф по этому ребру.',
      'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл.',
      'Мы вызываем алгоритм Дейкстры для поиска нового минимального ребра.'
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане, добавление ребра создаст цикл. Это нарушит структуру дерева.'
  },
  {
    id: 2,
    question: 'Почему алгоритм Дейкстры впадает в ступор на графе с отрицательными ребрами?',
    options: [
      'Потому что он написан только для положительных весов.',
      'Он жадный и считает, что каждый следующий шаг даст минимальный путь, не учитывая, что отрицательные ребра могут создать более короткий путь.',
      'Отрицательные ребра создают гравитационный коллапс в алгоритме.',
      'Дейкстра просто не любил отрицательные балансы на графах.'
    ]
  }
];
```

```

    ],
    correctAnswer: 0,
    explanation: 'Блестяще! CHM (DSU) позволяет за прак
    .',
  }
];

```

```

export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<num
  const [selectedOption, setSelectedOption] = useState<
  const [isAnswered, setIsAnswered] = useState<boolean>
  const [score, setScore] = useState<number>(0);
  const [showResults, setShowResults] = useState<boolean>

```

```

  const currentQ = quizQuestions[currentQIndex];

```

```

  const handleSelectOption = (index: number) => {
    if (isAnswered) return;
    setSelectedOption(index);
    setIsAnswered(true);
    if (index === currentQ.correctAnswer) {
      setScore(prev => prev + 1);
    }
  };

```

```

  const handleNext = () => {
    if (currentQIndex + 1 < quizQuestions.length) {
      setCurrentQIndex(prev => prev + 1);
      setSelectedOption(null);
      setIsAnswered(false);
    } else {
      setShowResults(true);
    }
  };

```

```

  const handleRestart = () => {
    setCurrentQIndex(0);
    setSelectedOption(null);
  };

```

```

    }

    return (
      <div className="bg-slate-900/90 border border-slate-
        >
        <div className="flex items-center justify-between
          <div className="flex items-center gap-3">
            <span className="bg-purple-600 text-white px-3
              Экспресс-тест
            </span>
            <h3 className="text-xl font-bold text-white">
          </div>
          <span className="text-sm font-mono text-slate-400
            Bonpoc {currentQIndex + 1} / {quizQuestions.length}
          </span>
        </div>

        <div className="mb-8">
          <h4 className="text-xl md:text-2xl font-bold text-white">
            {currentQ.question}
          </h4>

          <div className="space-y-4">
            {currentQ.options.map((option, idx) => {
              const isSelected = selectedOption === idx;
              const isCorrect = idx === currentQ.correctAnswer;

              let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
              if (isAnswered) {
                if (isCorrect) {
                  optionStyle = "bg-emerald-950/80 border border-emerald-500";
                } else if (isSelected) {
                  optionStyle = "bg-rose-950/80 border-rose-500";
                } else {
                  optionStyle = "bg-slate-800/40 border-slate-700";
                }
              }
            })}
          </div>
        </div>
      </div>
    );
  }
}

```

```

        <div className="flex justify-end pt-4 border-t border-gray-200" >
          <button
            onClick={handleNext}
            disabled={!isAnswered}
            className={
              "px-8 py-3.5 font-bold rounded-xl transition-colors duration-200"
              (!isAnswered
                ? "bg-slate-800 text-slate-500 border border-slate-900"
                : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700")
            }
          >
            {currentQIndex + 1 < quizQuestions.length ? 'Следующий' : 'Завершить'}
          </button>
        </div>
      </div>
    );
  };
};

```

ФАЙЛ 38/82: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect } from 'react';

interface Step {
  k: number; i: number; j: number;
  matrix: number[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

```

```

    { line: 7, text: '                                dist[i][j] = 0' },
    { line: 8, text: '    return dist' },
  ];

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist = initialMatrix.map(row => [...row]);

  simulationSteps.push({
    k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: '    Инициализация матрицы прямых путей, ∞ (999)',
    activeCodeLine: 2,
  });

  const N = 7;
  for (let k = 0; k < N; k++) {
    simulationSteps.push({
      k, i: -1, j: -1, matrix: dist.map(r => [...r]),
      log: `+ Внешний цикл. Разрешаем пути через прогнанные`,
      activeCodeLine: 3,
    });

    for (let i = 0; i < N; i++) {
      for (let j = 0; j < N; j++) {
        if (i === j || i === k || j === k) continue;

        const oldVal = dist[i][j];
        const viaVal = dist[i][k] + dist[k][j];
        if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
          dist[i][j] = viaVal;
          simulationSteps.push({
            k, i, j, matrix: dist.map(r => [...r]),
            log: `    Улучшение пути [${i}]→[${j}] через [${k}]`,
            activeCodeLine: 7,
          });
        }
      }
    }
  }
}

```

```

</div>

<div className="flex items-center gap-2">
  <button onClick={() => { setIsPlaying(false);
    text-sm">  Сброс</button>
  <button onClick={() => setIsPlaying(!isPlaying
    -amber-600' : 'bg-emerald-600'}`}>{isPlaying
  <button onClick={() => { setIsPlaying(false);
    ; }} className="px-3 py-2 bg-emerald-600 te
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 m
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-indigo-950/2
    stify-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-indi
      ont-bold shadow-sm">
        {currentStep.k >= 0 && currentStep.k < 7 ?
    </div>

    <svg className="w-full h-auto max-h-[500px]"
      <defs>
        <marker id="arrow-fw-normal" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
        <marker id="arrow-fw-active" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
        <marker id="arrow-fw-via" markerWidth="6"
        th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
      </defs>
      {Object.keys(adjList).flatMap((uStr) => {
        const u = parseInt(uStr);
        return adjList[u].map((edge) => {
          const uNode = nodesSvg.find((n) => n.id
          const vNode = nodesSvg.find((n) => n.id
          const isTarget = currentStep.i === u &&
          const isVia = (currentStep.i === u && c
          let strokeColor = '#475569'; let marker

```

```

</div>

  { /* Список смежности */ }
  <div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <div className="space-y-2 text-sm font-mono t
      {Object.keys(adjList).map((uStr) => {
        const u = parseInt(uStr);
        const isI = currentStep.i === u;
        const isK = currentStep.k === u;
        return {
          <div key={u} className={`flex flex-wrap
            isI ? 'bg-amber-900/40 border-ambe
            isK ? 'bg-indigo-900/40 border-ind
          }`}>
            <span className="font-bold text-whi
e-400">→</span>
            {adjList[u].map((edge, idx) => {
              const isUpd = currentStep.i === u
              const isVia = currentStep.k !== c
              const isDown = currentStep.j === edge.v);
              return {
                <span key={idx} className={`px-1
                dow' : isVia ? 'bg-indigo-500 text-white bo
                  {edge.v} ({edge.w})
                </span>
              }
            }
          }
        </div>
      )};
    }
  </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  { /* Матрица расстояний */ }
  <div className="w-full lg:w-1/2 bg-rose-950/10

```



```

    <h4 className="text-lg font-bold text-slate-600">
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map(item => (
        <div key={item.line} className={`py-1.5 px-4
e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'}`>
        <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
        <span className="whitespace-pre flex-grow-1">{item.code}</span>
      </div>
    )
    )
    </div>
  </div>
</div>
</div>
</div>
);
}
```

ФАЙЛ 39/82: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```

        steps.push({ type: 'process', node: v, desc: `Действие: ${v}`,
            .stack], visitedNodes: Array.from(vis) });
        dfs(u);
    }
}

stack.pop();
steps.push({ type: 'backtrack', node: v, desc: `Действие: ${v}`,
    ueOrStack: [...stack], visitedNodes: Array.from(vis) });
}

dfs(start);
return steps;
}

```

```

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начальное состояние`,
        ray.from(vis)] });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Действие: ${v}`,
            q], visitedNodes: [...Array.from(vis)] });
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, desc: `Посещение узла ${u}`,
                    tack: [...q], visitedNodes: [...Array.from(vis)] });
            }
        }
    }
}

```

```
const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
  <div className="space-y-4">
    <div className="flex bg-slate-900 border border-slate-700 p-4">
      <button onClick={() => setMode("dfs")}
        className={`px-4 py-2 text-xs font-medium rounded-md transition-colors duration-200`}
        text-slate-400 hover:text-slate-300">>
        DFS (В глубину)
      </button>
      <button onClick={() => setMode("bfs")}
        className={`px-4 py-2 text-xs font-medium rounded-md transition-colors duration-200`}
        "text-slate-400 hover:text-slate-300">>
        BFS (В ширину / Волна)
      </button>
    </div>

    <div className="flex flex-col md:flex-row gap-4">
      { /* SVG Graph View */ }
      <div className="bg-slate-900 border border-slate-700 p-4 min-h-[300px]">
        <svg className="w-full h-full max-w-[600px] max-h-[300px]">
          { /* Edges */ }
          {edges.map((e, i) => {
            const u = nodes.find(n => n.id === e.u);
            const v = nodes.find(n => n.id === e.v);
            // Check if edge is part of current path
            const edgeTraversed = isVisited(e.u) & isVisited(e.v);

            return (
              <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
                className={`${edgeTraversed ? 'stroke-emerald-500' : 'stroke-slate-400'} stroke-width=
500' : 'stroke-emerald-500')} : 'stroke-slate-400'`>
            );
          })}
        </svg>
      </div>
    </div>
  )
)
```



```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
  id: string;
  name: string;
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  let idx = a.length - 1;
  while (idx > 0) {
    const parent = Math.floor((idx - 1) / 2);
    if (a[parent].priority < a[idx].priority) {
      [a[parent], a[idx]] = [a[idx], a[parent]];
      idx = parent;
    } else break;
  }
  return a;
}
```

```
function siftDown(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  const n = a.length;
  let idx = 0;
  while (true) {
    let largest = idx;
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < n && a[l].priority > a[largest].priority) largest = l;
    if (r < n && a[r].priority > a[largest].priority) largest = r;
    if (largest !== idx) {
      [a[idx], a[largest]] = [a[largest], a[idx]];
      idx = largest;
    }
  }
  return a;
}
```

```

    { name: 'Оля (Порез)', symptom: ' Глубокий порез',
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emo
  ];
  start.forEach((p, i) => {
    arr = heapInsert(arr, { id: `init${i}`, ...p, animS
  });
  return arr;
})());

```

```
let uid = 300;
```

```

export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>([]);
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(0);
  const [log, setLog] = useState<string>('');
  const [shakeEmpty, setShake] = useState(false);
  const [busy, setBusy] = useState(false);
  const [healingPatient, setHealing] = useState<Patient>({

```

```
const addLog = (msg: string) => setLog(prev => [msg,
```

```
// --- INSERT: Новый пациент поступает в приемный покр
```

```

const insertPatient = useCallback((name: string, prio
  if (busy || heap.length >= 15) {
    addLog('⚠ Все палаты переполнены! Дождитесь выпис
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);

```

```

const preset = PATIENT_PRESETS[Math.floor(Math.rand
const finalName = name || preset.name.split(' ')[0]
const finalEmoji = preset.emoji;
const finalSymptom = preset.symptom;

```

```
const newPatient: Patient = {
```

```

    if (busy || heap.length === 0) {
      setShake(true);
      addLog(`⚠ Нет пациентов для экстренной госпитализации`);
      setTimeout(() => setShake(false), 400);
      return;
    }
    setBusy(true);
    const topPatient = heap[0];
    setHealing(topPatient);

    addLog(`СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

    // Step 1: Root lights up as winner
    setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'winning' } : x));

    setTimeout(() => {
      // Step 2: Root exits the tree, tree reorganizes
      setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'exiting' } : x));

      setTimeout(() => {
        // Run binary heap extraction
        setHeap(prev => {
          if (prev.length === 0) return [];
          const a = prev.map(x => ({ ...x }));
          a[0] = { ...a[a.length - 1] };
          a.pop();
          return siftDown(a).map(x => ({ ...x, animState: 'sifting' }));
        });

        // Patient gets healed in the bed
        setTimeout(() => {
          setHealing(prev => prev ? { ...prev, animState: 'healed' } : null);
          addLog(`Успех! Пациент ${topPatient.name} выписан`);
          setTimeout(() => {
            setHealing(null);
            setBusy(false);
          }, 600);
        }, 800);
      });
    }, 400);
  }
}

```

```

    return res;
  });

  return (
    <div className="flex flex-col gap-6">
      {/* МНЕМОНИКА / ПРАВИЛО */}
      <div className="bg-emerald-950/40 border border-emerald-500">
        <div className="text-3xl"> </div>
        <div>
          <h3 className="font-black text-emerald-400 text-center">
            Типы Queue</h3>
          <p className="text-xs text-slate-300 mt-1 leading-relaxed">
            В реанимации не важно, кто пришел раньше, а кто
            остояние</b> (максимальный приоритет).
            Куча (Heap) автоматически перестраивает пацки,
            оказывался на самом верху дерева (в корне).
          </p>
        </div>
      </div>

      {/* УПРАВЛЕНИЕ */}
      <div className="flex flex-col lg:flex-row items-stretch gap-6">
        {/* Добавить случайного */}
        <button
          onClick={handleRandomInsert}
          disabled={busy || heap.length >= 15}
          className="flex-1 min-w-[150px] bg-gradient-to-r from-emerald-500 to-emerald-700
            opacity-40 disabled:cursor-not-allowed text-emerald-400 font-medium
            ve:scale-95 transition-all cursor-pointer focus:outline-none"
        >
          <span> Привезти по Скорой (INSERT)</span>
        </button>

        {/* Добавить кастомного */}
        <form onSubmit={handleCustomInsert} className="flex-1">
          <input
            type="text"

```



```

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800
        r-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

```

```

{/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
<div className="grid grid-cols-1 lg:grid-cols-12" >

```

```

  {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}

```

```

  <div className={`lg:col-span-8 bg-slate-900 rounded-2xl
    -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}` >
    <div className="absolute top-4 left-4 text-[14px] font-medium" >
      Сортировка по тяжести (Двоичная куча / Max-Heap)
    </div>

```

```

    <div className="relative w-full h-full min-h-[400px]" >
      <svg className="absolute inset-0 w-full h-full" >
        {edges}
      </svg>

```

```

    {heap.map((patient, idx) => {
      const pos = nodePos(idx);
      const isRoot = idx === 0;
      return (

```

```

        <div
          key={patient.id}
          className={`
            absolute flex flex-col items-center justify-center
            -translate-x-1/2 -translate-y-1/2 transition-all duration-300
            ${patient.animState === 'in' ? 'animate-in' : ''}
            ${patient.animState === 'winner' ? 'animate-winner' : ''}
            ${patient.animState === 'out' ? 'animate-out' : ''}
            ${isRoot ? 'bg-rose-950/80 border-rose-500' : ''}

```

```

        <div className="relative">
          <span className="text-6xl block anim-bounce">
            <span className="absolute -top-4 -right-4">
              <span className="absolute -bottom-2 -right-2">
            </span>
          </div>
        <div>
          <h4 className="text-white font-black">
            <span>Пациент: {healingPatient.name}</span>
          </h4>
          <p className="text-emerald-400 text-x-small">
            Состояние: Стабилизация ({healingPatient.status})
          </p>
          <div className="flex justify-center gap-2">
            <span className="w-2 h-2 rounded-full bg-emerald-400">
              <Heart className="w-4 h-4 text-rose-500">
            </span>
          </div>
        </div>
      </div>
    ) : (
      <div className="text-center text-slate-600">
        <span className="text-5xl mb-2 block">
          <p className="text-xs font-bold font-mono">
            <p className="text-[10px] mt-1">Ждём самовосстановления
          </p>
        </div>
      </div>
    )}
  </div>

  <div className="w-full h-3 bg-gradient-to-r from-emerald-400 to-emerald-600">
  </div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4">
  <div className="text-[10px] font-mono text-slate-400">
    {log.map((entry, idx) => (
      <div
        key={idx}

```

```

const [step, setStep] = useState(0);
const paused = useContext(DemoPauseContext);
const period = Math.round(ms * SPEED);

useEffect(() => {
  if (steps <= 1 || paused) return;
  const t = setInterval(() => setStep((s) => (s + 1) % 2), period);
  return () => clearInterval(t);
}, [steps, period, paused]);

return step;
}

export const C = {
  idle: "#334155",
  idleStroke: "#475569",
  text: "#e2e8f0",
  dim: "#94a3b8",
  active: "#6366f1",
  done: "#10b981",
  warn: "#f59e0b",
  bad: "#f43f5e",
  edge: "#475569",
};

export const Svg: React.FC<{ children: React.ReactNode;
const paused = useContext(DemoPauseContext);
return (
  <div className="w-full">
    <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
      {children}
    </svg>
    {caption && (
      <div className="text-[10px] text-slate-400 text-center">
        {caption}
      </div>
    )}
    <div className="text-[9px] text-slate-600 text-center">

```

```

strokeDasharray={dashed ? "4 3" : undefined}
style={{ transition: MOVE }}
/>
);

```

ФАЙЛ 42/82: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7
  const pos: Record<string, [number, number]> = { i: [4, 1], j: [4, 9], k: [3, 4] };
  const viaOpen = step >= 1;
  const relaxed = step >= 2;
  return (
    <Svg
      caption={
        step === 0
          ? "k ещё запрещена: путь i → j = 9"
          : step === 1
            ? "Разрешаем пересадку через k: 3 + 4 = 7"
            : "7 < 9 → D[i][j] = 7"
        }
    >
      <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.done} />
      <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.bad} />
      <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.bad} />

      <text x={130} y={112} textAnchor="middle" fontSize=
        9
      >
        </text>

```



```

    {step >= 1 &&
      sorted.map(({v, i}) => {
        <g key={v}>
          <rect x={30 + i * 52} y={62} width={46} height={16}>
          <text x={53 + i * 52} y={77} textAnchor="middle">
            {v}
          </text>
        </g>
      })}

    {step >= 2 &&
      raw.map(({v, i}) => {
        <g key={`c${i}`}>
          <rect x={startX + i * (boxW + 4)} y={98} width={boxW} height={16}>
          <text x={startX + i * (boxW + 4) + boxW / 2} y={113} textAnchor="center">
            {sorted.indexOf(v)}
          </text>
        </g>
      })}
    </Svg>
  );
};

```

/* ----- Splay: три случая поворота ----- */

```
type Pt = [number, number];
```

```

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, string][];
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#2ecc71";

```

```
-----
export const ZigZigDemo: React.FC = () => (
  <SplayFrames
    captions={["Линия g → p → x («бамбук»)", "Поворот В"]}
    frames={[
      { pos: { g: [176, 24], p: [126, 70], x: [76, 112] }
      { pos: { p: [130, 26], x: [78, 78], g: [186, 78] }
      { pos: { x: [102, 24], p: [152, 70], g: [202, 112] }
    ]}
  />
);
```

/** Zig-Zag – x и p с разных сторон, крутим сначала ниж

```
export const ZigZagDemo: React.FC = () => (
  <SplayFrames
    captions={["Зигзаг: g влево, x вправо", "Поворот НИ"]}
    frames={[
      { pos: { g: [176, 24], p: [110, 70], x: [152, 112] }
      { pos: { g: [176, 24], x: [110, 70], p: [64, 112] }
      { pos: { x: [130, 30], p: [80, 100], g: [180, 100] }
    ]}
  />
);
```

ФАЙЛ 43/82: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
-----
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];
```

```
};
```

```
export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["труссы", "штаны"],
    ["носки", "штаны"],
    ["штаны", "боты"]
  ];
  const order = ["труссы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, {x: n.x, y: n.y}]));
  return (
    <Svg caption={`Порядок: ${placed.join(" → ") || "..."} `} >
      {edges.map(([u, v], i) => (
        <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
      ))}
      {nodes.map((n) => {
        const idx = placed.indexOf(n.id);
        return (
          <g key={n.id}>
            <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
            <text x={n.x} y={n.y} textAnchor="middle" dx={-30} dy={0}>{n.id}</text>
            {idx >= 0 && <circle cx={n.x + 30} cy={n.y} r={12} />
            {idx >= 0 && (
              <text x={n.x + 30} y={n.y - 12} textAnchor="middle">
                {placed[idx]}
              </text>
            )}
          </g>
        );
      })}
      <text x={224} y={62} textAnchor="middle" fontSize={16}>Порядок:</text>
      <text x={224} y={74} textAnchor="middle" fontSize={16}>{placed.join(" → ")}</text>
    </Svg>
  );
};
```



```
};
```

```
export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
  };
  const edges: [string, string][] = [["A", "B"], ["A",
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали вершину C → две отдельные"
      {edges.map(([u, v], i) => {cut && (u === "C" || v
      {Object.entries(pos).map(([k, [x, y]]) =>
        cut && k === "C" ? (
          <circle key={k} cx={x} cy={y} r={12} fill="no
        ) : (
          <Node key={k} x={x} y={y} label={k} fill={k =
            } />
        )
      )}
    </Svg>
  );
};
```

```
export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);
  const taken = new Set(chosen.slice(0, step));
  return (
    <Svg caption={`Жадно берём самые лёгкие рёбра без ц
      {edges.map((e, i) => {
```



```

    {p.map((par, i) =>
      par !== i ? {
        <path key={i} d={`M ${pos[i][0]} ${pos[i][1]}
          fill="none" stroke={rootColor[root(i)]} stroke-width={1}
        } : null
      )}
    )}
    {pos.map(([x, y], i) => (<Node key={i} x={x} y={y}
  </Svg>
  ));
};

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

export const TrieDemo: React.FC = () => {
  const step = useLoop(TRIE_NODES.length + 1, 700);
  return (
    <Svg caption="Путь от корня по буквам = слово; общий"
      {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
      })}
      {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x} y={n.y}
    </Svg>
  ));
};

export const TrieBfsDemo: React.FC = () => {
  const levels = [[0], [1, 2], [3, 4, 5]];
  const step = useLoop(levels.length + 1, 950);
  const doneIds = new Set(levels.slice(0, step).flat());
  const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];

```

```
export const SegmentTreeDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const nodes = [
    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
    { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
    { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых"
      {nodes.map((n, i) => (
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={20} fill={n.lvl === step ? C.active : C.idle} stroke={C.dim}/>
          <text x={n.x} y={n.y} textAnchor="middle" dom
            </g>
          )))
    </Svg>
  );
};
```

```
export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;
  return (
    <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] - P[${l}]` : "о префиксные суммы P"}
      <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
      {a.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 39} y={24} width={34} height={10} fill={C.dim} stroke={C.dim}/>
          <text x={20 + i * 39} y={24} fill={C.dim} stroke={C.dim}>{v}</text>
        </g>
      ))}
    </Svg>
  );
};
```

```

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  const avg = step ? (total / step).toFixed(2) : "-";
  return (
    <Svg caption={`Сделано ${step} операций, суммарно $
      {costs.map((c, i) => (
        <rect key={i} x={14 + i * 30} y={104 - c * 10} width={30} height={10}
        fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}
      ))}
      <line x1={8} y1={106} x2={252} y2={106} stroke={C.done}
      <text x={130} y={20} textAnchor="middle" fontSize={16}
    </Svg>
  );
};

export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (
    <Svg caption={step === 0 ? "Найти решение – долго (N^2)" : "Решение найдено"}
      <ellipse cx={130} cy={64} rx={112} ry={48} fill={C.done}
      <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333"
      <text x={92} y={64} textAnchor="middle" dominantBaseline="middle"
      <text x={206} y={34} textAnchor="middle" fontSize={16}
      <circle cx={196} cy={84} r={13} fill={step === 1 ? C.done : C.bad}
      <text x={196} y={84} textAnchor="middle" dominantBaseline="middle"
      ?`}
      <text x={130} y={124} textAnchor="middle" fontSize={16}
    </Svg>
  );
};

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);

```

```

        <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
            fontSize={10} fontWeight="700" fill="black">
            {c}
        </text>
    </g>
);
})
}
<text x={130} y={122} textAnchor="middle" fontSize={12}
    fill="black">
    каждая клетка считается один раз и переиспользуется
</text>
</Svg>
);
};

```

ФАЙЛ 45/82: `src/components/hints/MiniDemo.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
    AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
    SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
    ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
    ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
    | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
    | "suffix-link" | "toposort" | "relax" | "prefix-sum"
    | "segment-tree" | "bridge" | "articulation" | "mst"
    | "prefix-function" | "dp" | "floyd" | "components" |
    | "zig" | "zig-zig" | "zig-zag";
```

```
    <DemoPauseContext.Provider value={paused}>
      <div
        className="bg-slate-950/70 rounded-lg border border-slate-800
        onMouseEnter={() => setPaused(true)}
        onMouseLeave={() => setPaused(false)}
      >
        <Cmp />
      </div>
    </DemoPauseContext.Provider>
  );
};
```

ФАЙЛ 46/82: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";

interface Props {
  vizId: string | null;
  onClose: () => void;
}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
```

```
import React, { useEffect, useLayoutEffect, useRef, useImperativeHandle } from "react";
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../../vizRegistry";
```

```
export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}
```

```
interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
  onCardLeave: () => void;
}
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor, onClose, onOpenSimulator, onCardEnter, onCardLeave }) => {
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: number; }>({ top: 0, left: 0 });
```

```
  useLayoutEffect(() => {
    if (!anchor) {
      setPos(null);
      return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
```



```

        top: pos ? pos.top : -9999,
        left: pos ? pos.left : -9999,
        width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
        visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
  >
  <div className="bg-slate-900 border border-indigo-500" >
    <div className="flex items-start gap-2 px-3 pt-3" >
      <h4 className="text-sm font-bold text-indigo-400" >Подсказка
      <button
        onClick={onClose}
        className="text-slate-500 hover:text-white"
        aria-label="Заккрыть подсказку"
      >
        <X className="w-4 h-4" />
      </button>
    </div>

    <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
      <p className="text-[13px] leading-relaxed text-slate-300" >
        {entry.demo && <MiniDemo kind={entry.demo} />
        {entry.formal && (
          <p className="text-[11.5px] leading-relaxed text-slate-300" >
            {entry.formal}
          </p>
        )}
        {entry.complexity && (
          <div className="text-[11px] font-mono text-slate-300" >
            Сложность: {entry.complexity}
          </div>
        )}

        {viz && simulatorId && (
          <button

```

```

    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0 },
    { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0 },
    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0 }
  ];

const isNodeVisible = (id: string) => {
  if (id === 'S' && (step === 0 || step === 7)) return true;
  return false;
};

const getPotential = (id: string) => {
  if (step >= 2 && step < 7) {
    if (id === 'S') return 0;
    if (id === 'A') return 0;
    if (id === 'B') return -2;
    if (id === 'C') return -1;
  }
  return null;
};

const getEdgeColorClass = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "opacity-0";
    if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
    if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
    return "stroke-slate-700 stroke-dasharray-4";
  }
  if (e.id === 'AB') {
    if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 4) return "stroke-emerald-500";
  }
  if (e.id === 'BC') {
    if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 5) return "stroke-emerald-500";
  }
  if (e.id === 'CA') {
    if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 6) return "stroke-emerald-500";
  }
};

```

```

    }
    if (e.id === 'CA') {
        if (step < 6) return e.weight;
        if (step === 6) return `${e.weight} + (${-1
        return e.newWeight;
    }
    return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
    if (e.id === 'AB' && step === 4) return true;
    if (e.id === 'BC' && step === 5) return true;
    if (e.id === 'CA' && step === 6) return true;
    return false;
};

const isEdgeTextEmerald = (e: any) => {
    if (e.id === 'AB' && step > 4) return true;
    if (e.id === 'BC' && step > 5) return true;
    if (e.id === 'CA' && step > 6) return true;
    return false;
};

const getDesc = () => {
    switch (step) {
        case 0: return (
            <div>
                <b>Исходный граф.</b> Имеется ребро
                Если мы запустим быстрый алгоритм Д
            </div>
        );
        case 1: return (
            <div>
                <b>Шаг 1: Точка отсчета.</b><br/>
                Добавляем фиктивную вершину S. Соед
                ми</span>.
                Это наша база для расчета потенциал
            </div>
        );
    }
};

```

```

        case 7: return (
            <div className="text-emerald-300">
                <b>Готово!</b> Фиктивную вершину S
                Все новые веса ребер в графе стали
                При этом старые кратчайшие пути ост
            </div>
        );
        default: return "";
    }
};

return (
    <div className="bg-slate-800 p-4 rounded-xl border">
        <h4 className="text-sm md:text-base font-bo">
            <span className="w-2.5 h-2.5 bg-amber-400">
                Визуализация: Перевзвешивание алгоритмом
            </h4>

        <div className="flex flex-col md:flex-row gap-4">
            <div className="bg-[#0f172a] flex-1 min-h-100 inter p-4">
                <svg width="400" height="300" class="dark">
                    <defs>
                        <marker id="arrow-slate" viewBox="0 0 10 5"
                            start-reverse">
                            <path d="M 0 0 L 10 5 L 10 0" />
                        </marker>
                        <marker id="arrow-slate-dim" viewBox="0 0 10 5"
                            uto-start-reverse">
                            <path d="M 0 0 L 10 5 L 10 0" />
                        </marker>
                        <marker id="arrow-amber" viewBox="0 0 10 5"
                            start-reverse">
                            <path d="M 0 0 L 10 5 L 10 0" />
                        </marker>
                        <marker id="arrow-pink" viewBox="0 0 10 5"
                            tart-reverse">

```

```

                                isS ? 'fi
                                `}>
                                {getEdgeContent
                                </text>
                                </g>
                                )
                                }}}

    { /* Nodes */
    {nodes.map(n => {
      if (!isNodeVisible(n.id)) r
      const pot = getPotential(n.

      return (
        <g key={n.id} className=
          <circle
            cx={n.x} cy={n.y}
            className={`str
              ${n.id === 'S'
                ? 'fill-[#2
                : 'fill-sla
            />
            <text x={n.x} y={n.y}
- white">
              {n.label}
            </text>

            {pot !== null && (
              <g transform={`
fade-in zoom-in duration-500">
              <rect x="-1
k-500 stroke-1" />
              <text x="0"
font-bold fill-pink-400">
              {pot}
            </text>
          </g>
        )}

```



```

const recordStep = {
  phase: AlgoStep["phase"],
  desc: string,
  currentNode: number | null,
  transposed: boolean,
  currentSccId: number,
} => {
  generatedSteps.push({
    phase,
    desc,
    visited: new Set(visited),
    currentNode,
    order: [...order],
    transposed,
    sccMap: { ...sccMap },
    currentSccId,
    activeEdges: transposed
      ? initialEdges.map((e) => ({ u: e.v, v: e.u }
        : [...initialEdges],
  ));
};

recordStep(
  "init",
  "Инициализация алгоритма Косарайю. Начинаем первый",
  null,
  false,
  -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
  visited.add(u);

```

```

        -1,
    });

    // Transpose adj
    const adjT: number[][] = Array.from({ length: 5 },
    initialEdges.forEach((e) => adjT[e.v].push(e.u));

    // Reset visited for phase 2
    visited.clear();
    let sccId = 0;

    const dfs2 = (u: number, currentScc: number) => {
        visited.add(u);
        sccMap[u] = currentScc;
        recordStep(
            "dfs2",
            `DFS 2: заходим в ${u} на транспонированном графе`,
            u,
            true,
            currentScc,
        );

        for (const to of adjT[u]) {
            if (!visited.has(to)) {
                dfs2(to, currentScc);
            }
        }
    };

    // Run DFS2 using the order (reversed from back to front)
    for (let i = order.length - 1; i >= 0; i--) {
        const u = order[i];
        if (!visited.has(u)) {
            sccId++;
            recordStep(
                "dfs2",
                `DFS 2: берем следующую непосещенную из стека`,
                u,
            );
        }
    }
}

```



```

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1) {
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1500);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getScColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Layers className="w-5 h-5 text-emerald-400"
        Визуализатор Алгоритма Косарайю (Поиск SCC)
      </h3>

    <div className="flex items-center gap-2 bg-slate
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
          d-500 text-white"

```

```

        viewBox="0 0 10 10"
        refX="22"
        refY="5"
        markerWidth="6"
        markerHeight="6"
        orient="auto-start-reverse"
    >
        <path d="M 0 1 L 10 5 L 0 9 z" fill="#1f77b4" stroke="#1f77b4" stroke-width="1" />
    </marker>
    <marker
        id="arrow-transposed"
        viewBox="0 0 10 10"
        refX="22"
        refY="5"
        markerWidth="6"
        markerHeight="6"
        orient="auto-start-reverse"
    >
        <path d="M 0 1 L 10 5 L 0 9 z" fill="#66bb6a" stroke="#66bb6a" stroke-width="1" />
    </marker>
</defs>

```

```

{/* Edges */}
{initialEdges.map((edge, idx) => {
    const u = initialNodes.find((n) => n.id === edge.u);
    const v = initialNodes.find((n) => n.id === edge.v);

    // If transposed, draw reversed coordinates
    const x1 = step.transposed ? v.x : u.x;
    const y1 = step.transposed ? v.y : u.y;
    const x2 = step.transposed ? u.x : v.x;
    const y2 = step.transposed ? u.y : v.y;

    const isSccEdge =
        step.sccMap[edge.u] &&
        step.sccMap[edge.v] &&
        step.sccMap[edge.u] === step.sccMap[edge.v];
    let stroke = "#475569";

```

```

const isCurrent = step.currentNode === node
const isVisited =
  step.visited.has(node.id) ||
  (step.phase === "dfs1" && step.currentNode === node)
const sccId = step.sccMap[node.id];

let classes = "fill-slate-800 stroke-slate-800";
if (sccId) {
  classes = getSccColor(sccId);
} else if (isCurrent) {
  classes = "fill-amber-600 stroke-amber-600";
} else if (isVisited && step.phase === "dfs1") {
  classes = "fill-emerald-700 stroke-emerald-700";
}

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r="20"
      className={`transition-all duration-200`}
      strokeWidth="3"
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="13px"
      fontWeight="bold"
      className="select-none"
    >
      {node.label}
    </text>

    {sccId && (

```

```

    { /* Top-Sort output / Order stack represent
    <div>
      <span className="text-[10px] text-slate-500">
        СТЕК ВЫХОДА (DFS 1 ORDER):
      </span>
      <div className="flex flex-wrap gap-1 bg-slate-500">
        {step.order.length === 0 ? (
          <span className="text-slate-600 text-[10px]">
            Нет элементов
          </span>
        ) : (
          [...step.order].reverse().map(({val, idx}) => {
            <div
              key={idx}
              className="bg-indigo-950 border border-slate-500 p-1"
            >
              {val}
            </div>
          })
        )}
      </div>
    </div>

    { /* List of actions log */}
    <div>
      <span className="text-[10px] text-slate-500">
        ЛОГ ОПЕРАЦИЙ:
      </span>
      <div className="space-y-1.5 max-h-[150px] bg-slate-500">
        {steps
          .slice(0, currentStepIdx + 1)
          .reverse()
          .slice(0, 5)
          .map(({s, idx}) => {
            <div
              key={idx}
              className={`text-[11px] p-1.5 rounded`
            >
              {s.desc}
            </div>
          })
        }
      </div>
    </div>
  )}
)

```

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check
  ide-react';
```

```
interface Vertex {
  id: number;
  label: string;
  x: number;
  y: number;
  color: string; // 'none', 'red', 'blue', 'purple', 'm
}
```

```
interface Edge {
  id: string;
  u: number;
  v: number;
  weight: number;
  status: 'pending' | 'inspecting' | 'included' | 'reje
  color: string;
}
```

```
interface StepLog {
  title: string;
  description: string;
  type: 'info' | 'success' | 'warning' | 'error';
}
```

```
// 9 Vertices layout
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
  { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
  { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
  { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
  { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
```

```
// Step 1: Inspect A-B (2)
() => {
  setEdges(prev => prev.map(e => e.id === '0-1' ? {
  setLogs(prev => [{
    title: 'Шаг 1: Берем самое дешевое ребро A-B (в
    description: 'Оно соединяет две бесцветные верши
    type: 'info'
  }, ...prev]));
},
// Step 2: Include A-B
() => {
  setVertices(prev => prev.map(v => v.id === 0 || v
  setEdges(prev => prev.map(e => e.id === '0-1' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро A-B в осто́ве',
    description: 'Вершины A and B окрашены в красны
    type: 'success'
  }, ...prev]));
},
// Step 3: Inspect D-E (3)
() => {
  setEdges(prev => prev.map(e => e.id === '3-4' ? {
  setLogs(prev => [{
    title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
    description: 'Оно соединяет две другие бесцветн
    type: 'info'
  }, ...prev]));
},
// Step 4: Include D-E
() => {
  setVertices(prev => prev.map(v => v.id === 3 || v
  setEdges(prev => prev.map(e => e.id === '3-4' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро D-E в осто́ве',
    description: 'Вершины D и E окрашены в синий цв
    type: 'success'
  }, ...prev]));
},
```

```

    },
    // Step 9: Inspect B-E (6) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '1-4' ? {
        setLogs(prev => [{
            title: 'Шаг 5: Берем ребро B-E (вес 6)',
            description: 'НО: это ребро соединяет вершины B
            type: 'warning'
        }, ...prev]));
    },
    // Step 10: Reject B-E
    () => {
        setEdges(prev => prev.map(e => e.id === '1-4' ? {
        setLogs(prev => [{
            title: 'Отказ: Цикл обнаружен!',
            description: 'Мы выбрасываем ребро B-E. Иначе п
            type: 'error'
        }, ...prev]));
    },
    // Step 11: Inspect E-F (7)
    () => {
        setEdges(prev => prev.map(e => e.id === '4-5' ? {
        setLogs(prev => [{
            title: 'Шаг 6: Берем ребро E-F (вес 7)',
            description: 'Оно соединяет фиолетовую вершину
            type: 'info'
        }, ...prev]));
    },
    // Step 12: Include E-F
    () => {
        setVertices(prev => prev.map(v => v.id === 5 ? {
        setEdges(prev => prev.map(e => e.id === '4-5' ? {
        setLogs(prev => [{
            title: 'Успех: Ребро E-F в осто́ве',
            description: 'Вершина F окрашена в фиолетовый ц
            type: 'success'
        }, ...prev]));
    },

```

```

() => {
  setEdges(prev => prev.map(e => e.id === '6-7' ? {
  setLogs(prev => [{
    title: 'Шаг 9: Берем ребро G-H (вес 10)',
    description: 'Соединяет фиолетовую G и бесцветную H',
    type: 'info'
  }, ...prev]));
},
// Step 18: Include G-H
() => {
  setVertices(prev => prev.map(v => v.id === 7 ? {
  setEdges(prev => prev.map(e => e.id === '6-7' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро G-H в остове',
    description: 'Вершина H перекрашена в фиолетовую',
    type: 'success'
  }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '4-7' ? {
  setLogs(prev => [{
    title: 'Шаг 10: Берем ребро E-H (вес 11)',
    description: 'Оба конца E и H уже фиолетовые. Соединение создаст цикл!',
    type: 'warning'
  }, ...prev]));
},
// Step 20: Reject E-H
() => {
  setEdges(prev => prev.map(e => e.id === '4-7' ? {
  setLogs(prev => [{
    title: 'Отказ: Цикл обнаружен!',
    description: 'Выбрасываем ребро E-H.',
    type: 'error'
  }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {

```



```

        title: 'Шаг 2: Куча выдает самое дешевое ребро D-E',
        description: 'Плесень тянется по ребру D-E к вершине E',
        type: 'info'
    }, ...prev));
},
// Step 3: Include D-E, add D's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 3 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
        ..e, status: 'in_heap' } : e));
    setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-E (7)']);
    setLogs(prev => [{
        title: 'Успех: Плесень захватила вершину D',
        description: 'В кучу добавлены новые пути из D: D-E (7)',
        type: 'success'
    }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
        description: 'Плесень расширяется в сторону вершины D',
        type: 'info'
    }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D-E (7)']);
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в кучу',
        type: 'success'
    }, ...prev]));
},

```

```

        description: 'Ребро C-F(9) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
        description: 'ВНИМАНИЕ! Плесень проверяет верши
        type: 'warning'
    }, ...prev]));
},
// Step 11: Reject B-E
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
    setLogs(prev => [{
        title: 'Отказ: Игнорируем внутреннее ребро',
        description: 'Ребро B-E отброшено. Плесень не р
        type: 'error'
    }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
        description: 'Плесень из центра Токио (E) тянет
        type: 'info'
    }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
        eap' } : e));
    setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (

```

```

    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
    setLogs(prev => [{
      title: 'Отказ: Игнорируем ребро C-F',
      description: 'Выбрасываем из кучи.',
      type: 'error'
    }, ...prev]));
  },
// Step 18: Pop G-H (10)
() => {
  setEdges(prev => prev.map(e => e.id === '6-7' ? {
  setLogs(prev => [{
    title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
    description: 'Плесень ползет к вершине H.',
    type: 'info'
  }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
  setVertices(prev => prev.map(v => v.id === 7 ? {
  setEdges(prev => prev.map(e => e.id === '6-7' ? {
    eap' } : e));
  setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
  setLogs(prev => [{
    title: 'Успех: Вершина H захвачена',
    description: 'В кучу добавлено H-I(12).',
    type: 'success'
  }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '4-7' ? {
  setLogs(prev => [{
    title: 'Шаг 11: Куча выдает E-H (вес 11)',
    description: 'Обе вершины E и H уже в плесени.',
    type: 'warning'
  }, ...prev]));
},

```

```

// Node 0(A) -> A-B(2)
// Node 1(B) -> A-B(2)
// Node 2(C) -> B-C(4)
// Node 3(D) -> D-E(3)
// Node 4(E) -> D-E(3)
// Node 5(F) -> E-F(7)
// Node 6(G) -> D-G(8)
// Node 7(H) -> G-H(10)
// Node 8(I) -> H-I(12)
setEdges(prev => prev.map(e =>
  ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
setLogs(prev => [{
  title: 'Первая пятилетка: Каждая деревня выбирает
  description: 'Каждая из 9 деревень одновременно
    А выбирает A-B(2), D выбирает D-E(3), а G вы
    !',
  type: 'info'
}, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Components)
() => {
  // Concurrently build chosen roads. Merge into:
  // Component 1: {A, B, C} (colored red)
  // Component 2: {D, E, F, G, H, I} (colored gold/blue)
  setVertices(prev => prev.map(v =>
    [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
  ));
  setEdges(prev => prev.map(e => {
    if (['0-1', '1-2'].includes(e.id)) {
      return { ...e, status: 'included', color: 'red' };
    }
    if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
      return { ...e, status: 'included', color: 'blue' };
    }
    return e;
  }));
  setLogs(prev => [{

```

```

        вес: 51. Благодаря параллельности, мы потрати
        type: 'success'
    }, ...prev]);
    }
};

const currentSteps =
    activeAlgo === 'kruskal' ? kruskalSteps :
    activeAlgo === 'prim' ? primSteps :
    boruvkaSteps;

const handleNextStep = () => {
    if (stepIndex < currentSteps.length) {
        currentSteps[stepIndex]();
        setStepIndex(stepIndex + 1);
    }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
    setActiveAlgo(algo);
    setVertices(initialVertices);
    setEdges(initialEdges);
    setStepIndex(0);
    setHeapQueue([]);
    if (algo === 'kruskal') {
        setLogs([
            {
                title: 'Введение в раскраску (Краскал)',
                description: 'Представь, что все 9 вершин гра
                type: 'info',
            },
        ]));
    } else if (algo === 'prim') {
        setLogs([
            {
                title: 'Введение в Плесень (Прима)',
                description: 'Логика «Плесени»: выбираем стар
                edь (Heap) и захватываем соседей!',
            },
        ]));
    }
};

```

```

    }
    return '#475569';
};

return (
  <div className="space-y-12">

    {/* 3-in-1 MST Simulator Box */}
    <div className="bg-slate-900/90 border border-slate-800" >
      <div className="flex flex-col lg:flex-row justify-between" >
        <div>
          <div className="flex items-center gap-2 mb-2" >
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-bold uppercase tracking-wider">
              Супер-Интерактив 3 в 1
            </span>
            <h3 className="text-2xl font-bold text-white">
              MST
            </h3>
          </div>
          <p className="text-slate-400 text-sm">
            Наблюдай за работой Краскала (Раскраска),
          </p>
        </div>

        {/* Algorithm Tabs */}
        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center bg-slate-800" >
            <button
              onClick={() => handleReset('kruskal')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
                +
                (activeAlgo === 'kruskal'
                  ? 'bg-indigo-600 text-white shadow-md'
                  : 'text-slate-400 hover:text-slate-300')
              }
            >
              <GitGraph className="w-3.5 h-3.5" />
              <span>Краскал (Билет 18)</span>
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>
);

```

```

<button
  onClick={handleNextStep}
  disabled={stepIndex >= currentSteps.length}
  className={
    "flex items-center justify-center gap-2
initial cursor-pointer " +
    (stepIndex >= currentSteps.length
      ? "bg-slate-800 text-slate-500 border
      : activeAlgo === 'kruskal'
      ? "bg-indigo-600 hover:bg-indigo-500
      : activeAlgo === 'prim'
      ? "bg-emerald-600 hover:bg-emerald-500
      : "bg-rose-600 hover:bg-rose-500 text
    )
  }
>
  <SkipForward className="w-4 h-4" />
  <span>{stepIndex === 0 ? 'Начать' : stepIndex}
</button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3"
  /* Graph Canvas */
  <div className="lg:col-span-2 bg-slate-950 rounded-full
    active min-h-[480px] overflow-hidden">

    /* Legend */
    <div className="absolute top-4 left-4 bg-slate-900
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5">
        <div className="w-3 h-3 rounded-full bg-slate-800" />
      </div>
      {activeAlgo === 'kruskal' ? (
        <>
          <div className="flex items-center gap-1.5">
            <div className="w-3 h-3 rounded-full bg-slate-800" />
          </div>
          <div className="flex items-center gap-1.5">

```

```

    {edges.map(edge => {
      const u = vertices[edge.u];
      const v = vertices[edge.v];
      const strokeColor = getEdgeStroke(edge);
      const isInspecting = edge.status === 'inspect';
      const isRejected = edge.status === 'rejected';
      const isInHeap = edge.status === 'in_heap';

      return (
        <g key={edge.id}>
          <line
            x1={u.x + "%"}
            y1={u.y + "%"}
            x2={v.x + "%"}
            y2={v.y + "%"}
            stroke={strokeColor}
            strokeWidth={isInspecting ? 4.5 : 1}
            strokeDasharray={isRejected ? '4,4' : null}
            className={"transition-all duration-300"}
          />
          <text
            x={(u.x + v.x) / 2 + "%"}
            y={(u.y + v.y) / 2 + "%"}
            fill={isInspecting ? '#f59e0b' : '#94a3b8'}
            : '#94a3b8'}
            fontSize="4.5"
            fontFamily="monospace"
            fontWeight="bold"
            textAnchor="middle"
            dominantBaseline="middle"
            className="select-none filter drop-shadow"
            dy="-1.5"
          >
            {edge.weight}
          </text>
        </g>
      );
    })}
  </div>
</pre>

```


Ход выполнения

</h4>

</div>

```
{activeAlgo === 'prim' && {
  <div className="p-3 bg-slate-900/60 border-1" >
    <p className="text-[11px] font-bold text-slate-200">
      ✂ Приоритетная очередь (Min-Heap):
    </p>
    <div className="flex flex-wrap gap-1.5">
      {heapQueue.length === 0 ? (
        <span className="text-xs text-slate-400">
          Пусто
        </span>
      ) : (
        heapQueue.map(({item, idx}) => (
          <span key={idx} className={"px-2 py-1 shadow-md animate-pulse" : "bg-slate-800/50"}>
            {item}
          </span>
        ))
      )}
    </div>
  </div>
)}
```

```
<div className="flex-1 p-4 overflow-y-auto">
  {logs.map((log, idx) => (
    <div
      key={idx}
      className={
        "p-4 rounded-xl border transition-all"
        (log.type === 'success'
          ? 'bg-emerald-950/40 border-emerald-500'
          : log.type === 'warning'
          ? 'bg-amber-950/40 border-amber-500'
          : log.type === 'error'
          ? 'bg-rose-950/40 border-rose-500'
          : 'bg-slate-900/60 border-slate-700')
      >
      {log.message}
    </div>
  ))}
</div>
```

```

        </button>
        <button
          onClick={() => setActiveCheatTab('boruvka')}
          className={"px-3 py-1.5 rounded text-xs font-medium"}
          style={activeCheatTab === 'boruvka' ? "bg-rose-600 text-white" : "bg-white text-rose-600"}
        >
          Борувка (Билет 20)
        </button>
      </div>
    </div>

    {activeCheatTab === 'kruskal' && (
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
        <div className="lg:col-span-1 space-y-4">
          <div className="bg-slate-950 p-4 rounded-lg">
            <p className="text-slate-400 text-xs font-medium">
              <Clock className="w-3.5 h-3.5 text-inherit" />
            </p>
            <p className="text-2xl font-black text-white">Крускал</p>
            <p className="text-slate-400 text-xs font-medium">Минимальное количество ребер, соединяющих все вершины графа, называется минимальным остовным деревом.
            </p>
          </div>
          <div className="bg-slate-950 p-4 rounded-lg">
            <p className="text-slate-400 text-xs font-medium">
              <Award className="w-3.5 h-3.5 text-inherit" />
            </p>
            <p className="text-2xl font-black text-white">Награда</p>
            <p className="text-slate-400 text-xs font-medium">Награда за решение задачи.
            </p>
          </div>
          <div className="bg-slate-950 p-4 rounded-lg">
            <p className="text-slate-400 text-xs font-medium">
              <Award className="w-3.5 h-3.5 text-inherit" />
            </p>
            <p className="text-2xl font-black text-white">Награда</p>
            <p className="text-slate-400 text-xs font-medium">Награда за решение задачи.
            </p>
          </div>
        </div>
        <div className="lg:col-span-2">
          <div className="bg-slate-950 p-4 rounded-lg">
            <p className="text-slate-400 text-xs font-medium">
              <Award className="w-3.5 h-3.5 text-inherit" />
            </p>
            <p className="text-2xl font-black text-white">Награда</p>
            <p className="text-slate-400 text-xs font-medium">Награда за решение задачи.
            </p>
          </div>
          <div className="bg-slate-950 p-4 rounded-lg">
            <p className="text-slate-400 text-xs font-medium">
              <Award className="w-3.5 h-3.5 text-inherit" />
            </p>
            <p className="text-2xl font-black text-white">Награда</p>
            <p className="text-slate-400 text-xs font-medium">Награда за решение задачи.
            </p>
          </div>
        </div>
      </div>
    )}
  </div>

```

```

    </div>
    <div className="bg-slate-950 p-4 rounded-1
      <p className="text-slate-400 text-xs fo
        <Award className="w-3.5 h-3.5 text-em
      </p>
      <p className="text-2xl font-black text-r
      <p className="text-slate-400 text-xs mt
    </div>
    <div className="bg-slate-950 p-4 rounded-1
      <p className="text-slate-400 text-xs fo
ложении:</p>
      <p className="text-xs text-slate-300 le
ие чистые вершины через самое дешевое досту
    </div>
  </div>
  <div className="lg:col-span-2">
    <div className="bg-slate-950 p-4 rounded-1
      <p className="text-slate-400 text-xs fo
        <Code className="w-3.5 h-3.5 text-eme
      </p>
      <pre className="text-xs text-emerald-40
80 flex-1">
`import heapq

def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next w, v, u))

```



```
    },
  ];

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4">
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt}
              className="w-full h-full object-cover group-hover:scale-105 transition duration-300"/>
          </div>
          <div>
            <h3 className="text-lg font-bold mb-1 ${c.isHighlighted ? 'text-white' : 'text-slate-400'}">{c.title}</h3>
            <p className="text-slate-400 text-sm">{c.desc} <span className={`font-bold ${c.isHighlighted ? 'text-white' : 'text-slate-400'}">{c.status}</span>
          </p>
          <div className={`mt-3 text-xs font-mono px-1 ${c.isHighlighted ? 'text-white' : 'text-slate-400'}">{c.code}</div>
        </div>
      ))}
    </div>
  );
}
```

ФАЙЛ 52/82: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";
```

```
interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
}
```

```

        <span className="block text-[10px] uppercase">
          Содержание • {index + 1} из {total}
        </span>
        <span className="block text-[13px] font-bold">
        </span>
      </button>
    </div>

    {open && (
      <div className="lg:hidden fixed inset-0 z-[90]">
        <div className="absolute inset-0 bg-slate-950">
          <div className="absolute inset-y-0 left-0 w-[100%] h-[100%]
            te-[tocIn_.18s_ease-out]">
            <div className="flex items-center justify-between">
              <span className="font-bold text-white text-[14px]">
                <button
                  onClick={() => setOpen(false)}
                  className="p-2 -mr-2 text-slate-400 hover:text-white"
                  aria-label="Заккрыть содержание"
                >
                  <X className="w-5 h-5" />
                </button>
              </div>
            </div>
            <div className="flex-1 min-h-0">
              <Sidebar
                selectedChapterId={selectedChapterId}
                setSelectedChapterId={setSelectedChapterId}
                onNavigate={() => setOpen(false)}
              />
            </div>
          </div>
        </div>
      </div>
    )}
  </>
);
};

```

```
        Универсальное пособие
      </h1>
      <p className="text-xs text-indigo-400 font-mono">
        Подготовка к экзаменам: Алгоритмы, Структуры
      </p>
    </div>
  </div>

  <div className="flex items-center w-full lg:w-auto" >
    >
    <button
      id="tab-guide-btn"
      onClick={() => setActiveTab('guide')}
      className={`flex-1 lg:flex-none flex items-center justify-center
        duration-200 whitespace-nowrap ${
          activeTab === 'guide'
            ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
            : 'text-slate-400 hover:text-white'
        }`}
    >
      <BookOpen className="w-4 h-4" /> Учебное пособие
    </button>
    <button
      id="tab-simulator-btn"
      onClick={() => setActiveTab('simulator')}
      className={`flex-1 lg:flex-none flex items-center justify-center
        duration-200 whitespace-nowrap ${
          activeTab === 'simulator'
            ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
            : 'text-slate-400 hover:text-white'
        }`}
    >
      <Cpu className="w-4 h-4" /> Тренажёры
    </button>
    <a
      id="download-pdf-btn"
      href="/export/full_code_all_pages.pdf"
      download="full code all pages.pdf"
    >
```

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотри
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        { /* K5 */ }
        <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
          <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
          <svg className="w-full h-full absolute inset-0">
            {(() => {
              const pts = [
                {id:0,x:160,y:40},
                {id:1,x:270,y:100},
                {id:2,x:230,y:230},
                {id:3,x:90,y:230},
                {id:4,x:50,y:100}
              ];
              const edges = [
                [0,1],[0,2],[0,3],[0,4],
                [1,2],[1,3],[1,4],
                [2,3],[2,4],
                [3,4]
              ];
              function findPt(id:number) { return pts.find(p => p.id === id); }
              function intersect(a:number,b:number,c:number,d:number) {
                if (a===c||a===d||b===c||b===d) return null;
                const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
                function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
                  return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
                }
                return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y);
              }
            })()}
          </svg>
        </div>
      </div>
    )
  }
```



```

<svg className="w-full h-full absolute inset-0"
  {(() => {
    const pts = [
      {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:160,y:220},
      {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:60,y:60}
    ];
    const edges = [
      [0,3],[0,4],[0,5],
      [1,3],[1,4],[1,5],
      [2,3],[2,4],[2,5]
    ];
    function findPt(id:number) { return pts.find(p => p.id === id); }
    function intersect(a:number,b:number,c:number,d:number) {
      if (a===c||a===d||b===c||b===d) return true;
      const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
      if (!p1||!p2||!p3||!p4) return false;
      function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
        return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
      }
      return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
        && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
    }
    const crosses:number[]=[];
    for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
      if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
        if (!crosses.includes(i)) crosses.push(i);
        if (!crosses.includes(j)) crosses.push(j);
      }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
      const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
      const xing=crosses.includes(i);
      lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
        stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
    }
    const labels = ["\u04141","\u04142","\u04143","\u04144","\u04145"];
    for(const p of pts) {

```

```

-----
  animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
  { name: 'Студент Вася', emoji: '🍌', color: 'from-blue', text: 'голода после матана!' },
  { name: 'Кодер Семён', emoji: '🍜', color: 'from-violet', text: 'порцию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber', text: 'сметаны, плиз.' },
  { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose', text: 'сметаны, плиз.' },
  { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald', text: 'салатик для нашего НЛО!' },
  { name: 'Бизнесмен', emoji: '💼', color: 'from-slate', text: 'сметаны, плиз.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'from-blue', text: 'голода после матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: '🍜', color: 'from-violet', text: 'порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber', text: 'сметаны, плиз.', animState: 'idle' },
  ]));

  const [servedHistory, setServedHistory] = useState<string[]>([]);
  const [isServing, setIsServing] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>([]);

  const addLog = (msg: string) => setLog(prev => [...prev, msg]);

  // ENQUEUE: Встать в конец очереди (хвост / Tail)

```

```

addLog(`  DEQUEUE: Повар наливает суп первому – ${headGuest.name}`);

// Step 1: Head guest gets eating animation
setQueue(prev => prev.map((c, idx) => idx === 0 ? {
  ...c,
  animState: 'in'
} : c), 500);

setTimeout(() => {
  // Step 2: Guest leaves with happy face, queue shifts
  setQueue(prev => prev.map((c, idx) => idx === 0 ? {
    ...c,
    emoji: '😄'
  } : c), 500);

  setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

  setTimeout(() => {
    setQueue(prev => prev.slice(1));
    setIsServing(false);
    addLog(`  ${headGuest.name} поел и ушел сытым.`, 400);
  }, 900);
}, [queue, isServing]);

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: '😞', color: 'red', col: 1, text: 'У меня нет голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: '😞', color: 'blue', col: 2, text: 'Напишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: '😞', color: 'green', col: 3, text: 'Дайте котика без сметаны, плиз.', animState: 'in' },
  ]);
  setServedHistory([]);
  setIsServing(false);
  setLog(['  Столовая сброшена. Снова голодная толпа!']);
  setTimeout(() => {
    setQueue(prev => prev.map(c => ({ ...c, animState: 'idle' })), 500);
  }, 500);
};

```



```
      <div className="text-[10px] font-mono text-slate-500" data-bbox="206 101 1000 121">
        {log.map(({entry, idx} => {
          <div data-bbox="239 151 1000 216">
            key={idx}
            className={`text-xs font-mono flex items-center justify-between`} data-bbox="272 196 1000 216">
              >
                {idx === 0 ? <span className="text-indigo-500 font-weight-bold">
                  <span>{entry}</span>
                </span> : <span>{entry}</span>}
              </div>
            </div>
          </div>
        </div>
      </div>
    );
  };
};
```

ФАЙЛ 56/82: src/components/SalmonAutomatonWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
  x?: number;
  y?: number;
}
```

```

    let curr = 0;
    for (let i = 0; i < word.length; i++) {
      const c = word[i].toUpperCase();
      if (newNodes[curr].children[c] === undefined) {
        count++;
        newNodes[curr].children[c] = count;
        newNodes[count] = {
          id: count,
          char: c,
          parent: curr,
          children: {},
          fail: 0,
          term_link: 0,
          is_terminal: false,
          words: [],
          depth: newNodes[curr].depth + 1
        };
      }
      curr = newNodes[curr].children[c];
    }
    if (!newNodes[curr].words.includes(word)) {
      newNodes[curr].words.push(word);
      newNodes[curr].is_terminal = true;
    }
  });

  let currentX = 0;
  const paddingX = 70;
  const paddingY = 80;

  const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
      newNodes[u].x = 40 + currentX * paddingX;
      currentX++;
    } else {

```

```

    setNewWord('');
    buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем работать');
    return;
  }
  const updated = words.filter((w) => w !== wordToRemove);
  setWords(updated);
  buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];
  const updatedNodes = { ...nodes };
  rootChildren.forEach((childId) => {
    updatedNodes[childId].fail = 0;
  });

  setNodes(updatedNodes);
  setBfsQueue(queue);
  setSimulationStep(1);
  setCurrentNode(null);
  setSpeechText(
    'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
  );
};

// Один шаг BFS
const stepBFS = () => {
  if (bfsQueue.length === 0) {
    setSimulationStep(2);
    setCurrentNode(null);
  }
};

```



```

        setNodes(updatedNodes);
        setBfsQueue(newQueue);
        setCurrentNode(r);
        setSpeechText(logs);
    };

    return (
      <div className="bg-slate-800/90 rounded-2xl p-6 border" >
        {/* Шапка виджета */}
        <div className="flex flex-wrap items-center justify-between" >
          <div className="flex items-center space-x-3" >
            <span className="text-3xl" > </span>
            <div>
              <h4 className="text-2xl font-bold text-white" >
                Интерактивный конструктор: Говорящий Эхо-
              </h4>
              <p className="text-sm text-slate-400" >
                Построение дерева и расчет суффиксных ссы
              </p>
            </div>
          </div>
          <div className="flex items-center gap-2" >
            <button
              onClick={() => buildInitialTrie(words)}
              className="flex items-center gap-1 bg-slate-700 text-white
              transition-colors"
              title="Сбросить автомат"
            >
              <RotateCcw className="w-4 h-4" /> Сбросить
            </button>
          </div>
        </div>
      </div>

      {/* Верхняя панель: Говорящий лосось и диалоговое
      <div className="grid grid-cols-1 md:grid-cols-3 gap-4" >
        {/* Аватар Лосося (Сеня) с анимацией моргания и
        <div className="flex flex-col items-center justify-center" >
          <div className="w-40 h-40 relative flex items

```

```

        { /* Рот (анимация разговора) */ }
        { isTalking ? (
            <path d="M 165 105 Q 155 120 165 125 Q
        ) : (
            <path d="M 165 110 Q 155 115 168 118" s
        ) }

        { /* Улыбка / жабры */ }
        <path d="M 130 80 Q 125 100 130 120" stro
        <path d="M 120 85 Q 115 100 120 115" stro
    </svg>

    { /* Декоративные пузыри */ }
    <div className="absolute -top-1 right-4 tex
    <div className="absolute top-8 right-1 text
    </div>
    <span className="mt-3 bg-rose-600/30 border b
        Эхо-Лосось Сеня
    </span>
    </div>

    { /* Пузырь с текстом (Баббл) */ }
    <div className="md:col-span-2 bg-slate-800 bord
        tween min-h-[140px]">
        { /* Треугольник баббла */ }
        <div className="absolute -left-3 top-1/2 -tra
            slate-800 border-b-[12px] border-b-transpar

    <div className="flex items-center space-x-2 t
        <Volume2 className={`w-4 h-4 ${isTalking ?
        <span>Прямое вещание из аквариума:</span>
    </div>

    <p className="text-slate-200 text-base leading
        {speechText}
    </p>

```

```

    </div>
  </div>

  {/* SVG Canvas for Automaton Tree */}
  <div className="mb-8 bg-slate-900 border border-slate-800" >
    <h5 className="text-white font-bold mb-3 flex items-center" >
      <span> </span> Дерево и Суффиксные (fail) ссылки
    </h5>

    {{{() => {
      let maxX = 0;
      let maxY = 0;
      Object.values(nodes).forEach(n => {
        if (n.x && n.x > maxX) maxX = n.x;
        if (n.y && n.y > maxY) maxY = n.y;
      });
      const svgWidth = Math.max(maxX + 80, 400);
      const svgHeight = Math.max(maxY + 60, 200);

      return (
        <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
          <defs>
            <marker id="arrowhead" markerWidth="6" markerHeight="6" >
              <polygone points="0 0, 6 3, 0 6" fill="white" />
            </marker>
            <marker id="fail-arrow" markerWidth="6" markerHeight="6" >
              <polygone points="0 0, 6 3, 0 6" fill="white" />
            </marker>
          </defs>

          {/* Draw Edges */}
          {Object.values(nodes).map(node => {
            if (node.id === 0) return null;
            const parent = nodes[node.parent];
            if (!parent.x || !parent.y || !node.x || !node.y) return null;

            const isCurrent = currentNode === node.id ? "text-white" : "text-slate-400";
            const strokeDash = isCurrent ? [] : [4, 4];
            const stroke = isCurrent ? "white" : "slategray";
            const arrow = isCurrent ? "arrowhead" : "fail-arrow";
            const arrowSize = isCurrent ? 6 : 8;

            return <g>
              <line x1={parent.x} y1={parent.y} x2={node.x} y2={node.y} stroke={stroke} strokeDash={strokeDash} />
              <marker
                id={arrow}
                x={node.x} y={node.y}
                angle={Math.atan2(node.y - parent.y, node.x - parent.x)}
                style={{width: arrowSize, height: arrowSize}}
              />
            </g>
          )}
        </svg>
      )
    }}
  </div>

```

```

        <path
          key={`fail-${node.id}`}
          d={`M ${node.x} ${node.y - 10} Q ${
- 16}`}
          fill="none"
          stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
          strokeWidth={isCurrent ? 2 : 1.5}
          strokeDasharray="4 4"
          markerEnd="url(#fail-arrow)"
        />
      );
    }
  })

  { /* Draw Nodes */
  Object.values(nodes).map(node => {
    if (!node.x || !node.y) return null;
    const isRoot = node.id === 0;
    const isCurrent = currentNode === node.id;
    const isInQueue = bfsQueue.includes(node.id);

    let fill = '#1e293b';
    let stroke = '#475569';

    if (isCurrent) {
      fill = '#2563eb';
      stroke = '#60a5fa';
    } else if (isInQueue) {
      fill = '#b45309';
      stroke = '#fbbf24';
    } else if (node.is_terminal) {
      fill = '#059669';
      stroke = '#34d399';
    }

    return (
      <g key={`node-${node.id}`}>
        <circle
          cx={node.x}

```

```

    <h5 className="text-white font-bold mb-3 flex items-center">
      <span> </span> Словарь (Паттерны)
    </h5>
    <div className="flex flex-wrap gap-2 mb-4">
      {words.map((w) => {
        <span
          key={w}
          className="bg-slate-800 border border-slate-700 padding-2
            text-white font-medium rounded-md gap-1 group"
        >
          {w}
          <button
            onClick={() => removeWord(w)}
            className="text-slate-500 hover:text-white font-size-0.8
              transition-colors"
            title="Удалить"
          >
            ✕
          </button>
        </span>
      )}
    </div>
  </div>

  <form onSubmit={handleAddWord} className="flex items-center gap-2">
    <input
      type="text"
      value={newWord}
      onChange={(e) => setNewWord(e.target.value)}
      placeholder="НОВОЕ СЛОВО..."
      maxLength={8}
      className="bg-slate-800 border border-slate-700 padding-2 rounded-md
        outline-none focus:border-blue-500"
    />
    <button
      type="submit"
      className="bg-slate-700 hover:bg-slate-600 text-white font-medium
        transition-colors"
    >
      Добавить
    </button>
  </form>

```

ФАЙЛ 57/82: src/components/SegmentTreeVisualizer.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
// ===== TYPES =====
```

```
interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>; // node index to value
  highlightNodes: number[]; // current highlighted nodes
  mergeChildren?: [number, number]; // pair being merged
  arrayHighlight?: [number, number]; // [L, R] range in array
  result?: number; // partial result
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
function buildTreeFull(arr: number[]): Record<number, number> {
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  }
  go(1, 0, n - 1);
  return tree;
}
```

```
// ===== STEP GENERATORS =====
```

```

        steps.push({ id: id++, desc: `Объединяем: tree[${v}]
            state: { ...tree }, highlightNodes: [v], mergeChildren:
        }
    go(1, 0, n - 1);
    steps.push({ id: id++, desc: `    Дерево построено! Корень:
        Highlight: [0, n - 1] });
    return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
    const n = arr.length;
    const tree = buildTreeFull(arr);
    const steps: Step[] = [];
    let id = 0;

    function go(v: number, l: number, r: number, ql: number, qr: number) {
        if (ql > r || qr < l) {
            steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
                codeLine: 2, treeState: tree, highlightNodes: [v],
            return 0;
        }
        if (ql <= l && r <= qr) {
            steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
                state: { tree: tree, highlightNodes: [v] }, codeLine: 4, treeState: tree,
            return tree[v];
        }
        const m = (l + r) >> 1;
        steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
            codeLine: 6, treeState: tree, highlightNodes: [v],
        const left = go(2 * v, l, m, ql, qr);
        const right = go(2 * v + 1, m + 1, r, ql, qr);
        const res = left + right;
        steps.push({ id: id++, desc: `Возврат в узел ${v}:
            mergeChildren: [2 * v, 2 * v + 1], arrayHighlight:
        return res;
    }
    const ans = go(1, 0, n - 1, qL, qR);
}

```



```

        l >>= 1;
        r >>= 1;
        if (l < r) {
            steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, r = ${r}`
                , arrayHighlight: [qL, qR], result: res });
        }
    }
    steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`
        , highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
    return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; left: VNode; right: VNode; }
```

```

function buildVisualTree(treeState: Record<number, number>) {
    if (l === r) return { v, l, r, val: treeState[v] ?? null, left: null, right: null };
    const m = (l + r) >> 1;
    return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m),
        right: buildVisualTree(treeState, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
    "build(v, l, r) {",
    "    if (l === r) { tree[v] = A[l]; return; }",
    "    // -----",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    build(2*v, l, mid);",
    "    build(2*v+1, mid+1, r);",
    "    tree[v] = tree[2*v] + tree[2*v+1];",
    "}"
];

const QUERY_TD_CODE = [
    "query(v, l, r, qL, qR) {",
    "    if (qL > r || qR < l) return 0;",
    "    // -----",

```

```

border-indigo-500 bg-indigo-600', childRing: 'ring-
-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
30 border-emerald-500 bg-emerald-600', childRing: '
-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
der-amber-500 bg-amber-600', childRing: 'ring-rose-
[color];

return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
| 'emerald' | 'amber'; arr: number[] }) {
  const p = Player({ steps, color });
  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed
  const n = arr.length;

  const vTree = useMemo(() => buildVisualTree(step.tree

  const renderNode = useCallback((nd: VNode): React.Rea
    const isHigh = step.highlightNodes.includes(nd.v);
    const isChild = step.mergeChildren?.includes(nd.v);
    const has = nd.val !== null;

    let cls = 'bg-slate-800 border-slate-700 text-slate
    if (isHigh) cls = `${clr.ring} text-white ring-2 sc
    else if (isChild) cls = `${clr.childRing} text-ambe
    else if (has) cls = `${clr.filled} text-slate-200`;

    return (
      <div key={nd.v} className="flex flex-col items-ce
        <div className={`flex flex-col items-center jus
          ${cls}`}>
          <span className="text-[7px] opacity-60 font-m
          <span className="text-[9px] font-bold">[{nd.l
          <span className="text-xs font-extrabold font-r
        </div>

```

```

    { /* Tree */ }
    <div className="flex-1 p-2 overflow-x-auto bg-slate-900">
      <div className="scale-[0.85] origin-top min-w-max">
        </div>

    { /* Code */ }
    <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
      {code.map((line, i) => {
        const ln = i + 1;
        const active = step.codeLine === ln;
        return (
          <div key={ln} className={`px-2 py-0.5 rounded-2 border-indigo-500 text-indigo-200 ${active ? 'bg-amber-600/20 border-l-2 border-amber-600' : ''}`>
            <span className="inline-block w-4 text-slate-400">{ln}</span>
            {line}
          </div>
        );
      })}
    </pre>

    { /* Description + result */ }
    <div className="px-3 py-2.5 bg-slate-900/60 border-t border-slate-800">
      <span className={`mt-0.5 inline-block w-2 h-2 rounded-2 border-indigo-500 text-indigo-200 ${active ? 'bg-amber-600/20 border-l-2 border-amber-600' : ''}`></span>
      <span className="leading-relaxed">{step.desc}</span>
      {step.result !== undefined && (
        <span className={`ml-auto shrink-0 font-mono text-slate-400 ${active ? 'text-emerald-300' : ''}`>
          {step.result}
        </span>
      )}
    </div>

    { /* Controls */ }
    <div className="px-3 py-2.5 bg-slate-950 border-t border-slate-800">
      <div className="flex items-center bg-slate-900/60 border-t border-slate-800">
        <button onClick={() => { setPlaying(false); setStep(0); }}>

```



```

    <div className="flex items-center gap-1">
      <span className="text-xs text-slate-400">
        <input
          type="number"
          min={0}
          max={arr.length - 1}
          value={qR}
          onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
          className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
        />
      </div>
    </div>
    <div className="text-[10px] text-slate-500">
      Запрос:  $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.\text{slice}(qL, qR + 1).reduce((acc, val) => acc + val, 0)}\}$ 
    </div>
  </div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1">
  <button onClick={() => setView('build')} className="text-sm font-medium border border-indigo-500 rounded-md px-2 py-1.5 text-slate-200">
    Построить
  </button>
  <button onClick={() => setView('queries')} className="text-sm font-medium border border-emerald-500 rounded-md px-2 py-1.5 text-slate-200">
    Запрос: св
  </button>
  <button onClick={() => setView('theory')} className="text-sm font-medium border border-blue-500 rounded-md px-2 py-1.5 text-slate-200">
    Почему / Сра
  </button>
</div>

```

```

<div className="space-y-5">
  {/* Why it splits this way */}
  <div className="bg-slate-950 p-5 rounded-xl b
    <h4 className="text-base font-bold text-whi
    во разбилось именно так для N={arr.length}?
    <p className="text-sm text-slate-300 leadin
      Каждый узел берёт <code className="bg-sla
      к получает <code className="bg-slate-900 px
      -slate-900 px-1 rounded font-mono text-emerg
      евая часть забирает на 1 больше (округление
    </p>
    <div className="bg-slate-900 p-4 rounded-lg
      <div className="text-slate-400">Корень: L
      <div className="text-slate-300">mid = |(0
      <div className="text-indigo-300">→ Левый:
      <div className="text-emerald-300">→ Правы
      h - 1) >> 1)) эл.)</div>
      <div className="text-slate-500 mt-2">Всег
      } внутренних.</div>
    </div>
  </div>

```

```

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols
  <div className="bg-slate-950 p-5 rounded-xl
    <div className="absolute -top-3 left-4 bg
    e border border-emerald-500 shadow-md">
      1 Запрос Сверху-вниз (Рекурсия)
    </div>
    <p className="text-xs text-slate-300 mb-3
      Начинаем с корня. Если отрезок узла полн
      наче спускаемся в обоих детей.
    </p>
    <div className="space-y-1.5 text-xs">
      <div className="flex justify-between bo
      span className="text-rose-400 font-mono">0(
      <div className="flex justify-between bo
      ><span className="text-rose-400 font-mono">

```

ФАЙЛ 58/82: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";
```

```
interface SidebarProps {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Мобильный режим: список живёт в выдвижной панели */
  onNavigate?: () => void;
}
```

```
export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId }) => {
  const [query, setQuery] = useState("");
```

```
  const groups = useMemo(() => {
    const q = query.trim().toLowerCase();
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
    const map = new Map<string, typeof chapters>();
    for (const c of filtered) {
      const key = c.category?.trim() || "Прочие темы";
      if (!map.has(key)) map.set(key, []);
      (map.get(key) as typeof chapters).push(c);
    }
    return Array.from(map.entries());
  }, [query]);
```

```
  return (
    <aside className="w-full bg-slate-900/80 lg:bg-transparent" >
      <div className="p-4 pb-3 shrink-0">
        <h3 className="text-xs font-bold text-slate-400">
```

```

        onNavigate?.();
        window.scrollTo({ top: 0, behavior: 'smooth' });
    }}
    className={`w-full text-left px-3 py-2 border-1
        isSelected
            ? "bg-indigo-600/15 border-indigo-600/15"
            : "bg-slate-800/40 border-transparent"
    }`}
>
    {hasViz && {
        <span
            className="w-1.5 h-1.5 rounded-full border-1"
            title="Есть интерактивная визуализация"
        />
    }}
    <span className="font-semibold text-white">
    <ChevronRight
        className={`w-4 h-4 shrink-0 mt-0.5
            isSelected ? "text-indigo-400" : "text-white"
        }`>
    />
    </button>
    );
    }
    </div>
</div>
    )}
</div>

```

```

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-600/15 to-indigo-600/0"
    ] text-slate-300 space-y-1.5 hidden lg:block">
    <div className="font-bold text-indigo-400 flex items-center">
        <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
    </div>
    <p className="leading-relaxed">
        Подчёркнутые термины в тексте раскрываются по клику на иконку.
    </p>

```



```

    { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
    { id: '3', name: 'Блюдец от торта', icon: ' ' },
  ]);
const [popMessage, setPopMessage] = useState<string | null>('');

// Queue state
const [queue, setQueue] = useState<Person[]>([
  { id: '1', name: 'Студент Вася', icon: ' ' },
  { id: '2', name: 'Голодный кодер', icon: ' ' },
  { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<string | null>('');

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', icon: ' ' },
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"', icon: ' ' },
  { id: '3', text: 'Кандидат: "Привет, я испанский интеллект"', icon: ' ' },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.5);
const [heapMessage, setHeapMessage] = useState<string | null>('');

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: ' ' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setQueue([newPlate, ...queue]);
  setPopMessage('');
  setDequeueMessage('');
  setHeapMessage('');
};
const dequeuePlate = () => {
  if (queue.length > 0) {
    setQueue(queue.slice(1));
    setDequeueMessage('');
  }
};
const popPlate = () => {
  if (queue.length > 0) {
    const poppedPlate = queue[0];
    setQueue(queue.slice(1));
    setPopMessage(poppedPlate.name);
  }
};
const addHypothesis = () => {
  const newHypothesis = {
    id: Date.now().toString(),
    text: newCustomText,
    icon: ' '
  };
  setHeap([newHypothesis, ...heap]);
  setHeapMessage('');
};
const removeHypothesis = () => {
  if (heap.length > 0) {
    setHeap(heap.slice(1));
    setHeapMessage('');
  }
};
const popHypothesis = () => {
  if (heap.length > 0) {
    const poppedHypothesis = heap[0];
    setHeap(heap.slice(1));
    setHeapMessage(poppedHypothesis.text);
  }
};

```

```

    setQueue(prev => [...prev, newPerson]);
    setDequeueMessage(` Встал в конец очереди: ${newPe
});

const servePerson = () => {
  if (queue.length === 0) {
    setDequeueMessage("⚠ Очередь пуста! Суп ждет нов
    return;
  }
  const firstPerson = queue[0];
  setQueue(prev => prev.slice(1));
  setDequeueMessage(` Выдан суп первому в очереди (F
});

const resetQueue = () => {
  setQueue([
    { id: '1', name: 'Студент Вася', icon: ' ' },
    { id: '2', name: 'Голодный кодер', icon: ' ' },
    { id: '3', name: 'Кот Борис', icon: ' ' },
  ]);
  setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {
    id: Date.now().toString(),
    text: `Кандидат: "${newCustomText}"`,
    probability: Number(newCustomProb)
  };
  setHeap(prev => [...prev, item]);
  setNewCustomText('');
  setHeapMessage(` Добавлена гипотеза с вероятностью
});

const popHeap = () => {

```

```
</div>
```

```
{/* Tabs */}
```

```
<div className="grid grid-cols-1 md:grid-cols-3 gap-3">
```

```
  <button
```

```
    onClick={() => setActiveTab('stack')}>
```

```
    className={`flex items-center justify-between border-1px border-solid border-gray-200
      ${activeTab === 'stack' ? 'bg-blue-600/20 border-blue-500 text-blue-600' : 'bg-slate-800/60 border-slate-700/60 text-white'}`}
```

```
  >
    <div className="flex items-center gap-3">
      <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'bg-blue-600/20 border-blue-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
      <div className="text-left">
        <div className="font-bold text-sm text-white">LIFO
        <div className="text-xs opacity-80">LIFO
      </div>
    </div>
  </div>
  <span className="text-xs bg-blue-500/20 text-white">DFS
  </span>
</button>
```

```

  <button
    onClick={() => setActiveTab('queue')}>
    className={`flex items-center justify-between border-1px border-solid border-gray-200
      ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-600' : 'bg-slate-800/60 border-slate-700/60 text-white'}`}
```

```
  >
    <div className="flex items-center gap-3">
      <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
      <div className="text-left">
        <div className="font-bold text-sm text-white">FIFO
        <div className="text-xs opacity-80">FIFO
      </div>
    </div>
  </div>
```

```

  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">FIFO
      <div className="text-xs opacity-80">FIFO
    </div>
  </div>
  </div>
```

```

  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">FIFO
      <div className="text-xs opacity-80">FIFO
    </div>
  </div>
  </div>
```

```

  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">FIFO
      <div className="text-xs opacity-80">FIFO
    </div>
  </div>
  </div>
```

```

  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">FIFO
      <div className="text-xs opacity-80">FIFO
    </div>
  </div>
  </div>
```

```

  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">FIFO
      <div className="text-xs opacity-80">FIFO
    </div>
  </div>
  </div>
```

```

  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">FIFO
      <div className="text-xs opacity-80">FIFO
    </div>
  </div>
  </div>
```

```

  <div className="flex items-center gap-3">
    <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500' : 'bg-slate-800/60 border-slate-700/60'}`} />
    <div className="text-left">
      <div className="font-bold text-sm text-white">FIFO
      <div className="text-xs opacity-80">FIFO
    </div>
  </div>
  </div>
```

```

        Новые тарелки ставятся наверх. Мыть мож
    </p>
</div>
<div className="flex flex-wrap items-center" >
    <button
        onClick={addPlate}
        className="flex-1 md:flex-none flex item
        rounded-xl text-sm font-bold shadow-lg tra
    >
        <Plus className="w-4 h-4" /> Поставить
    </button>
    <button
        onClick={popPlate}
        className="flex-1 md:flex-none flex item
        border-blue-500/40 px-4 py-2.5 rounded-xl
    >
        Помыть верхнюю
    </button>
    <button
        onClick={resetStack}
        title="Сбросить"
        className="p-2.5 bg-slate-800 hover:bg-
        tion-colors cursor-pointer"
    >
        <RotateCcw className="w-5 h-5" />
    </button>
</div>
</div>

{popMessage && (
    <div className="bg-blue-950/60 border borde
    imate-fade-in">
        <span className="inline-block w-2 h-2 roun
        {popMessage}
    </div>
)}

{/* Visualization */}

```

```

        {isTop ? 'Готово к помывке' : `
      </span>
    </div>
  );
  }}}
</div>
})
<div className="mt-6 text-xs text-slate-500
  ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
)}

{/* QUEUE TAB */}
{activeTab === 'queue' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-i
        /span>
      </h3>
      <p className="text-slate-400 text-xs mt-1
        Кто первым пришел, тот первым получил с
      </p>
    </div>
    <div className="flex flex-wrap items-center
      <button
        onClick={addPerson}
        className="flex-1 md:flex-none flex ite
      -2.5 rounded-xl text-sm font-bold shadow-lg
    >
      <Plus className="w-4 h-4" /> Встать в о
    </button>
    <button

```

```

        <span className="text-3xl"> </span>
        <span className="text-xs font-bold font-mono">
        </div>

<div className="text-indigo-500 flex items-center">
  <ArrowRight className="w-6 h-6 animate-pulse">
</div>

{queue.map((person, index) => {
  const isFirst = index === 0;
  return (
    <div
      key={person.id}
      className={`flex flex-col items-center justify-center
        ${isFirst
          ? 'bg-gradient-to-b from-indigo-500 to-white
relative'
          : 'bg-slate-900/90 border-slate-800'}
        `}
    >
      {isFirst && (
        <span className="absolute -top-10 left-1/2 -translate-x-1/2
se tracking-wider shadow">
          Первый (Head)
        </span>
      )}
      <span className="text-4xl mb-2">{person.name}</span>
      <span className={`font-bold text-2xl font-mono
        ${isFirst ? 'bg-indigo-500/30 text-white' : ''}
        `}>
        {isFirst ? 'Получает сун' : `Входит ${person.name}
        </span>
      </div>
    );
  });
})}

```

```

        <button
          onClick={resetHeap}
          title="Сбросить"
          className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
      </div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
  <div className="md:col-span-6">
    <label className="block text-xs font-bold">
      <input
        type="text"
        value={newCustomText}
        onChange={e => setNewCustomText(e.target.value)}
        placeholder='например: "Привет, я лучший"'
        className="w-full bg-slate-900 border border-
rder-emerald-500 transition-colors"
      />
    </div>
    <div className="md:col-span-3">
      <label className="block text-xs font-bold">
        Вероятность: <span className="text-emerald-500">
      </label>
      <input
        type="range"
        min="0.01"
        max="0.99"
        step="0.01"
        value={newCustomProb}
        onChange={e => setNewCustomProb(Number(e.target.value))}
        className="w-full accent-emerald-500 cu
      />
    </div>
  </div>
</form>

```

```

{sortedHeap.map(({item, index}) => {
  const isTop = index === 0;
  const percent = (item.probability * 100).toFixed(
  return {
    <div
      key={item.id}
      className={`flex items-center justify-center
        isTop
          ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]'
          : 'bg-slate-900/80 border-slate-800'}
      `}
    >
      <div className="flex items-center justify-center"
        <span className={`flex items-center justify-center
          isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900/80 text-slate-300'
        `}>
          #{index + 1}
        </span>
        <div>
          <div className={`font-bold text-white`}
            {item.text}
          </div>
          {isTop && (
            <span className="text-[10px] font-normal text-white"
              -0.5 rounded inline-block mt-1">
                Вершина Кучи (Root) • Будущее
              </span>
            )}
          </div>
        </div>

      <div className="flex items-center justify-center"
        {/* Custom progress bar */}
        <div className="hidden sm:block"
          <div
            className={`h-full rounded-full`}
            style={{ width: `${percent}%`

```



```

onGoChapter: (chapterId: string) => void;
}

/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно поты
 */
export const SimulatorHub: React.FC<Props> = ({ onOpen,
  const [query, setQuery] = useState("");

  const items = useMemo(() => {
    const titleById = new Map(chapters.map((c) => [c.id,
    const all = Object.entries(VIZ_REGISTRY).map(([id,
      id,
      entry,
      chapterTitle: titleById.get(id),
    ]));
    const q = query.trim().toLowerCase();
    if (!q) return all;
    return all.filter(
      (i) =>
        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q)
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
  }, [query]);

  return (
    <div>
      <div className="mb-6">
        <h2 className="text-xl sm:text-2xl font-extrabold">
        <p className="text-sm text-slate-400 leading-relaxed">
          Все интерактивные демонстрации в одном месте.
          здесь их можно открыть напрямую.
        </p>
      </div>
    </div>
  );
}

```

```

        >
        <Play className="w-3.5 h-3.5" /> Запусти
    </button>
    {chapterTitle && {
        <button
            onClick={() => onGoChapter(id)}
            title={chapterTitle}
            className="flex items-center gap-1.5
0 hover:text-white transition-colors min-w-1
        >
            <BookOpen className="w-3.5 h-3.5 shrink-0" />
            <span className="truncate max-w-[9rem]" />
        </button>
    }}
</div>
</div>
    )})
</div>
</div>
);
};

```

ФАЙЛ 61/82: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";

```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерево
```

```
 * поднять отмеченный узел в корень. Клик по узлу = один шаг
```

```
 * Порядок поворотов выбираешь сам; правильный ли он по
```

```
 * только после того, как узел окажется наверху (или по
```

```

    name: "Один уровень",
    hintCase: "Zig",
    target: 2,
    build: () => ({ key: 4, left: { key: 2, left: leaf(
},
{
    name: "Смешанный",
    hintCase: "Zig-Zag, затем Zig-Zig",
    target: 5,
    build: () => {
        const n5 = leaf(5);
        const n6: TNode = { key: 6, left: n5, right: leaf(
        const n4: TNode = { key: 4, left: leaf(3), right:
        return { key: 8, left: n4, right: leaf(9) };
    },
},
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
    const path: TNode[] = [];
    let cur = root;
    while (cur) {
        path.push(cur);
        if (key === cur.key) return path;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return root;

    const x = path[path.length - 1];
    const p = path[path.length - 2];

```

```

    depth: number;
  }

function layout(root: TNode | null): { nodes: Placed[];
  const nodes: Placed[] = [];
  const edges: [number, number][] = [];
  let order = 0;

  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
    if (n.left) edges.push([n.key, n.left.key]);
    if (n.right) edges.push([n.key, n.right.key]);
    walk(n.right, depth + 1);
  };
  walk(root, 0);

  const cols = Math.max(1, order);
  const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
  const colW = 46;
  const rowH = 62;
  return {
    nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
    edges,
    width: cols * colW,
    height: rows * rowH + 20,
  };
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.root);
  const [history, setHistory] = useState<TNode[]>([]);

```

```

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build()))

return (
  <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
    <div className="flex items-start justify-between p-3">
      <div>
        <h4 className="text-sm sm:text-base font-bold text-white">
          <p className="text-[12px] text-slate-400 leading-4">
            Клик по узлу = один поворот с его родителем
            <span className="font-mono font-bold text-indigo-400">
              поворотов выбираешь сам, разбор покажем в к
            </p>
          </div>
          <button
            onClick={() => reset((presetIdx + 1) % PRESETS.length)}
            className="flex items-center gap-1.5 px-3 py-1.5 text-white font-medium text-xs font-bold transition-colors shrink-0"
          >
            <Dices className="w-3.5 h-3.5" /> Другое дерево
          </button>
        </div>

        <div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
          {PRESETS.map((p, i) => {
            <button
              key={p.name}
              onClick={() => reset(i)}
              className={`px-3 py-1.5 rounded-lg text-[11px] font-medium
                ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-400"}
              `}
            >
              {p.name}
            </button>
          ))}
        </div>
      </div>
    </div>
  )

```

```

        >
          {isTarget && {
            <circle r={22} fill="none" stroke="#6366f1"
          }}
          <circle
            r={16}
            fill={isTarget ? "#6366f1" : path.has
            stroke={isTarget ? "#a5b4fc" : "#475568"}
            strokeWidth={2}
          />
          <text textAnchor="middle" dominantBaseline="middle"
            {n.key}
          </text>
        </g>
      );
    }}}}
  </svg>
</div>

<div className="flex flex-wrap items-center gap-2" >
  <button
    onClick={undo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold"
  >
    <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
  </button>
  <button
    onClick={() => reset()}
    className="flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" /> ЗАНОВО
  </button>
  <button
    onClick={() => setShowAnswer((s) => !s)}
    className={`flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors`}
  >
    <RotateCcw className="w-3.5 h-3.5" /> ЗАНОВО
  </button>
  <button
    onClick={() => setShowAnswer((s) => !s)}
    className={`flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors`}
  >
    <RotateCcw className="w-3.5 h-3.5" /> ЗАНОВО
  </button>

```

```

        (m, i) =>
          !m.correct && (
            <li key={i}>
              ход {i + 1}: крутили <span className=
                {m.expected}>
              </li>
            )
          )
        )
      </ul>
    )
    {wrong === 0 && moves.length > minimal && (
      <div className="opacity-80">Порядок верный,
    )
  </div>
)}
</div>
);
};

```

ФАЙЛ 62/82: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";

```

/**

* Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag

*

* Что сделано ради понятности:

* • у узлов настоящие числа (20 / 40 / 60), а не абстрактные

* сразу видно, что дерево остаётся деревом поиска;

* • под деревом печатается обход слева направо: он ОДНОЗНАЧЕН

* это и есть доказательство, что поворот ничего не меняет

* • кадр «прицел» ничего не двигает, только подсвечивает

* • на кадре «результат» остаются призраки – пунктирные

```

-----
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
  pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
  edges: [{"p", "x"}, {"p", "C"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
  key: "zig",
  label: "Zig",
  when: "родитель x — уже корень, деда нет",
  rotations: "1 поворот",
  keys: { x: 20, p: 40 },
  inorder: ["A", "20", "B", "40", "C"],
  ranges: "A < 20 · B между 20 и 40 · C > 40",
  frames: [
    {
      ...ZIG_BEFORE,
      kind: "start",
      title: "Было: 20 висит под корнем 40",
      text: "Нам нужен ключ 20, а он на глубине 1. Деду
      ms: RESULT_MS,
    },
    {
      ...ZIG_BEFORE,
      kind: "aim",
      title: "Прицел: крутим ребро 40 — 20",
      text: "Ничего пока не двигается. Смотри на подсве
      focus: ["p", "x"],
      ms: AIM_MS,
    },
    {
      pos: { x: [160, 50], A: [80, 130], p: [245, 130],
      edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
      kind: "result",
      title: "Стало: 20 — корень",
      text: "40 стало правым сыном двадцатки. Поддерев

```



```

        title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
        text: "Главная хитрость этого случая: первым крути
        focus: ["g", "p"],
        ms: AIM_MS,
    },
    {
        ...ZZ_S1,
        kind: "result",
        title: "Поворот 1: 40 поднялось над 60",
        text: "40 встало в корень, 60 опустилось к нему п
            убине 1, а не 2.",
        moved: ["p", "g", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 – 20",
        text: "А теперь обычный одиночный поворот – тот ж
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [100, 45], A: [45, 118], p: [188, 118],
        edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
        kind: "result",
        title: "Стало: 20 в корне, бамбук стал кустом",
        text: "Сравни с первым кадром: А и В были на глуб
            дерево, а не только достаёт ключ.",
        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    },
    ],
};

```

/* ----- Zig-Zag -----

```

const ZG_S0 = {
    pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:

```

```

        moved: ["x", "p", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZG_S1,
        kind: "aim",
        title: "Прицел 2: ребро 60 – 40",
        text: "Остался одиночный поворот вокруг деда – и",
        focus: ["g", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [180, 50], p: [90, 130], g: [275, 130],
        edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
        kind: "result",
        title: "Стало: 20 и 60 – два сына сорока",
        text: "Дерево стало идеально симметричным: все че
            очку». ",
        moved: ["x", "p", "g", "C"],
        ms: RESULT_MS,
    },
    ],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: string }> = {
    x: { fill: "#6366f1", stroke: "#a5b4fc" },
    p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
    g: { fill: "#f59e0b", stroke: "#fcd34d" },
    sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
    x: "x – ищем его",
    p: "p – родитель",
    g: "g – дед",
};

```

```

        />
      })
    </g>
  );
})))

{edges.map(([a, b], i) => {
  const pa = pos[a];
  const pb = pos[b];
  if (!pa || !pb) return null;
  const hot = Boolean(focus && focus.includes(a));
  return (
    <line
      key={`${a}-${b}-${i}`}
      x1={pa[0]}
      y1={pa[1]}
      x2={pb[0]}
      y2={pb[1]}
      stroke={hot ? "#fbbf24" : "#475569"}
      strokeWidth={hot ? 5 : 2}
      opacity={focus && !hot ? 0.25 : 1}
      style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
    />
  );
})))

(({Object.keys(pos) as NodeId[]).map((id) => {
  const p = pos[id];
  if (!p) return null;
  const sub = isSubtree(id);
  const c = COLOR[sub ? "sub" : id];
  const r = sub ? 15 : 20;
  const hot = Boolean(focus && focus.includes(id));
  return (
    <g
      key={id}
      style={{
        transform: `translate(${p[0]}px, ${p[1]}px)`,

```

```

const duration = useMemo(() => Math.round(current.ms
const last = idx === total - 1;

useEffect(() => {
  setFrame(0);
}, [caseIdx]);

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => setFrame((f) => (f + 1)
  return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
  <div className="w-full bg-slate-950 rounded-2xl border
    <div className="flex gap-2 mb-4 overflow-x-auto no
      {CASES.map((c, i) => {
        <button
          key={c.key}
          onClick={() => setCaseIdx(i)}
          className={`px-3 sm:px-4 py-2 rounded-lg text
            i === caseIdx ? "bg-indigo-600 text-white
          }`}
        >
          {c.label}
        </button>
      })}
    </div>

    <div className="mb-3 text-xs sm:text-[13px] text-
      <span className="font-bold text-indigo-300">Kor
      <span className="text-slate-500">({active.rotat
    </div>

    /* шаг + пояснение стоят НАД картинкой: сначала
    <div className="mb-3 rounded-xl border border-sla
      <div className="flex items-center gap-2 mb-1 fl
        <span

```

```
</div>
```

```
{/* индикатор автопрокрутки */}
```

```
<div className="h-1 w-full bg-slate-800 rounded-f
```

```
  <div
```

```
    key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `
```

```
    className="h-full bg-indigo-500"
```

```
    style={
```

```
      playing
```

```
        ? { animation: `demoProgress ${duration}m
```

```
          : { width: "100%", background: "#334155"
```

```
    }
```

```
  />
```

```
</div>
```

```
<div className="flex flex-wrap items-center gap-2
```

```
  <button
```

```
    onClick={() => {
```

```
      setPlaying(false);
```

```
      setFrame((f) => (f - 1 + total) % total);
```

```
    }}
```

```
    disabled={idx === 0}
```

```
    className="px-3 py-2 rounded-lg bg-slate-800
```

```
      d:hover:text-slate-300 text-xs font-bold tr
```

```
  >
```

```
    ← Назад
```

```
</button>
```

```
<button
```

```
    onClick={() => {
```

```
      setPlaying(false);
```

```
      setFrame((f) => (f + 1) % total);
```

```
    }}
```

```
    className="px-4 py-2 rounded-lg bg-indigo-600
```

```
      w-indigo-900/40"
```

```
  >
```

```
    {last ? "⏮ Сначала" : "Дальше →"}
```

```
</button>
```

```
<button
```

```
        </div>

        <p className="mt-2 text-[11px] text-slate-500">
          Листай кнопкой «Дальше» в своём темпе – кадры н
        </p>
      </div>
    );
  };
};
```

ФАЙЛ 63/82: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
import {
  RotateCcw,
  HelpCircle,
  BookOpen,
} from "lucide-react";

interface SplayNode {
  key: number;
  left: SplayNode | null;
  right: SplayNode | null;
  x?: number;
  y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
  if (!root) return { key, left: null, right: null };
  if (key < root.key) {
    root.left = insertBST(root.left, key);
  } else if (key > root.key) {
    root.right = insertBST(root.right, key);
  }
}
```

```

    const initialKeys = [40, 20, 50, 10, 30, 60];
    initialKeys.forEach((key) => {
      r = insertBST(r, key);
    });
    return r!;
  };

const [tree, setTree] = useState<SplayNode>(createInitialTree);
const [inputValue, setInputValue] = useState<string>('');
const [log, setLog] = useState<string[]>([
  "Дерево инициализировано. Попробуйте кликнуть на узел."
]);
const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

// Splay operation that tracks steps!
const splayStepByStep = (
  root: SplayNode | null,
  key: number,
): { newRoot: SplayNode | null; steps: string[] } => {
  const steps: string[] = [];
  if (!root) return { newRoot: null, steps: ["Дерево пусто."] };

  // We implement the classic top-down or bottom-up splay.
  // For visualization logs, we do a bottom-up explanation.
  let path: { node: SplayNode; dir: "left" | "right" | null }[] = [];
  let current: SplayNode | null = root;

  // Find the element or the element where search failed.
  let lastNode: SplayNode = root;
  while (current) {
    lastNode = current;
    if (key === current.key) {
      path.push({ node: current, dir: null });
      break;
    } else if (key < current.key) {
      path.push({ node: current, dir: "left" });
      current = current.left;
    } else {
      path.push({ node: current, dir: "right" });
      current = current.right;
    }
  }

  // Now we splay the lastNode.
  // If the lastNode is null, it means the tree was empty.
  if (lastNode === null) return { newRoot: null, steps: ["Дерево пусто."] };

  // Perform the splay operation.
  // This is a simplified version of the splay operation.
  // In a real implementation, you would need to handle rotations.
  // For simplicity, we will just move the lastNode to the root.
  const newRoot = lastNode;
  newRoot.left = null;
  newRoot.right = null;

  return { newRoot, steps: [...path, "Сплей завершён."] };
};

```

```

        node.left.right = splayAction(node.left.right);
        if (node.left.right) {
            node.left = leftRotate(node.left);
            steps.push(`Компонент Zig-Zag: левый поворот`);
        }
    }

    if (!node.left) return node;
    steps.push(
        `Финальный поворот Zig (Правое вращение корня)`);
    return rightRotate(node);
} else {
    if (!node.right) return node;

    if (k < node.right.key) {
        // Zag-Zig (Right-Left)
        node.right.left = splayAction(node.right.left);
        if (node.right.left) {
            node.right = rightRotate(node.right);
            steps.push(`Компонент Zig-Zag: правый поворот`);
        }
    } else if (k > node.right.key) {
        // Zag-Zag (Right-Right)
        node.right.right = splayAction(node.right.right);
        node = leftRotate(node);
        steps.push(
            `Вращение Zag-Zag (Левый поворот родителя д...`);
    }
}

if (!node.right) return node;
steps.push(
    `Финальный поворот Zag (Левое вращение корня)`);
return leftRotate(node);
}
};

```



```

    });

    const resetTree = () => {
      setTree(createInitialTree());
      setHighlightedNode(null);
      setLog(["Дерево возвращено в исходное состояние."]);
    };

    // Assign coordinate positions using a simple recursion
    const computeCoordinates = (
      node: SplayNode | null,
      x: number,
      y: number,
      levelWidth: number,
    ): void => {
      if (!node) return;
      node.x = x;
      node.y = y;
      if (node.left) {
        computeCoordinates(node.left, x - levelWidth, y + 1);
      }
      if (node.right) {
        computeCoordinates(node.right, x + levelWidth, y + 1);
      }
    };

    const drawTree = cloneTree(tree);
    computeCoordinates(drawTree, 300, 40, 140);

    // Render lines and nodes
    const renderLines = (node: SplayNode | null): React.ReactNode => {
      if (!node) return [];
      let lines: React.ReactNode[] = [];
      if (node.left && node.left.x && node.left.y) {
        lines.push(
          <line
            key={`l-${node.key}-${node.left.key}`}
            x1={node.x}

```

```

        <circle
          cx={node.x}
          cy={node.y}
          r="18"
          fill={isHighlighted ? "#4f46e5" : "#1e293b"}
          stroke={isHighlighted ? "#818cf8" : "#64748b"}
          strokeWidth={isHighlighted ? "3" : "2"}
          className="transition-all duration-300 group-1
        />
        <text
          x={node.x}
          y={node.y}
          dy=".3em"
          textAnchor="middle"
          fill="white"
          fontSize="12px"
          fontWeight="bold"
          className="select-none pointer-events-none"
        >
          {node.key}
        </text>
      </g>,
    );
    circles = circles.concat(renderNodes(node.left));
    circles = circles.concat(renderNodes(node.right));
    return circles;
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-
      { /* Header */ }
      <div className="bg-slate-800 p-4 border-b border-
        <h3 className="font-bold text-white flex items-
          <BookOpen className="w-5 h-5 text-indigo-400"
            Интерактивное Splay-дерево
          </h3>
          <p className="text-xs text-slate-400">
            Кликните на вершины дерева, чтобы «вытащить»

```

```

        if (!isNaN(val)) {
          let newTree = cloneTree(tree);
          newTree = insertBST(newTree, val);
          const { newRoot, steps } = splayStep(newTree, val);
          if (newRoot) setTree(newRoot);
          setLog([`Вставили [${val}].`, ...steps]);
          setInputValue("");
          setHighlightedNode(val);
        }
      }}
      className="bg-indigo-950 hover:bg-slate-900"
    >
      >
      Вставить
    </button>
  </form>

  <button
    onClick={resetTree}
    className="flex items-center gap-1.5 px-3"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Сброс
  </button>
</div>
</div>

{/* Logs */}
<div className="lg:col-span-5 bg-slate-950 border to overflow-y-auto custom-scrollbar">
  <h4 className="text-xs font-bold text-slate-400">
    Ход вращения (Splay-логи)
  </h4>
  <div className="flex-1 space-y-2 mb-4 overflow">
    {log.map((step, idx) => (
      <div
        key={idx}
        className={`text-xs p-2.5 rounded trans
          idx === log.length - 1

```

```

    </p>
    <p className="text-slate-400 leading-relaxed">
      Допустим, мы ищем{" "}
      <code className="text-indigo-400 font-sans">
        его нет. Алгоритм спустится к листу, на
        неудачей (например,{" "})
      <code className="text-indigo-400">[20]</code>
      <code className="text-indigo-400">[30]</code>
      поднимет в корень{" "}
      <strong className="text-white">
        последний успешно просмотренный узел
      </strong>
      , на котором он споткнулся! Это оставля
      сверху.
    </p>
  </div>
  <div className="mt-3 text-[10px] text-indigo-400">
    Поиск настраивает локальный кэш под реали
  </div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
  <div>
    <p className="font-bold text-slate-300 mb-2">
      2. Эффект качелей (Swing)
    </p>
    <p className="text-slate-400 leading-relaxed">
      Если агент запрашивает по очереди{" "}
      <code className="text-rose-400">[10]</code>
      <code className="text-rose-400">[60]</code>
      углы), дерево будет превращаться в палку
      сторону. Первая операция Splay(60) стои
      <strong className="text-white">O(n)</strong>
      глубину n. Вторая Splay(10) заставляет
      Постоянный свинг убивает производительн
      сложности <strong className="text-white">
    </p>
  </div>
</div>

```

```
-----
import { ArrowUp, Sparkles } from 'lucide-react';

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: ' ', color: 'from-pa'},
  { name: 'Тарелка из-под пиццы', emoji: ' ', color: 'from-pa'},
  { name: 'Блюдец от торта', emoji: ' ', color: 'from-pa'},
  { name: 'Тарелка от тако', emoji: ' ', color: 'from-y'},
  { name: 'Тарелка из-под суши', emoji: ' ', color: 'from-y'},
  { name: 'Сковорода от глазуньи', emoji: ' ', color: 'from-y'}
];

let counter = 0;

export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>(
    [
      { id: 'p1', name: 'Тарелка из-под спагетти', emoji: ' ', color: 'from-pa', animState: 'idle' },
      { id: 'p2', name: 'Тарелка из-под пиццы', emoji: ' ', color: 'from-pa', animState: 'idle' },
      { id: 'p3', name: 'Блюдец от торта', emoji: ' ', color: 'from-pa', animState: 'idle' },
    ]
  );

  const [cleanRack, setCleanRack] = useState<Plate[]>(
    []
  );
  const [isWashing, setIsWashing] = useState(false);
  const [showSoap, setShowSoap] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>(
    []
  );
}
```

```

    }

    setIsWashing(true);
    const topPlate = dirtyStack[dirtyStack.length - 1];

    addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.name} 🍴`);

    // Step 1: Start washing animation (sponge and soap)
    setDirtyStack(prev => prev.map(p => p.id !== topPlate.id));
    setShowSoap(true);

    setTimeout(() => {
      // Step 2: Wash complete. Plate becomes clean and goes to clean rack
      const cleanPlate: Plate = {
        ...topPlate,
        isDirty: false,
        animState: 'clean',
      };

      setCleanRack(prev => [cleanPlate, ...prev].slice(0, 5));
      setDirtyStack(prev => prev.slice(0, prev.length - 1));
      setShowSoap(false);
      setIsWashing(false);
      addLog(`  Готово! Чистая тарелка ${topPlate.emoji} 🍴`);
    }, 850);
  }, [dirtyStack, isWashing]);

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', animState: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', animState: 'in' },
    { id: 'r3', name: 'Блюдце от торта', emoji: '🍰', animState: 'in' },
  ]);
  setCleanRack([]);
};

```

```
        opacity-40 disabled:cursor-not-allowed text-
        ve:scale-95 transition-all cursor-pointer f
    >
    <span> Помыть верхнюю (POP)</span>
</button>

<button
  onClick={reset}
  className="px-5 py-3.5 rounded-2xl bg-slate-800
    r-pointer border border-slate-700"
>
  Сброс
</button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12">

  {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
  <div className="md:col-span-7 bg-slate-900 rounded-2xl
    [380px]">
    <div className="absolute top-4 left-4 text-[10px] font-mono">
      Грязная стопка (Стек вызовов / LIFO)
    </div>

    {/* Визуализация раковины */}
    <div className="absolute top-4 right-4 flex items-center
      r-slate-800/80 text-[10px] font-mono">
      <span className="w-1.5 h-1.5 rounded-full bg-slate-700">
      <span>РАКОВИНА </span>
    </div>

    {/* Мыльные пузыри при мытье */}
    {showSoap && {
      <div className="absolute inset-0 pointer-events-none">
        <div className="absolute w-6 h-6 rounded-full bg-white
          "></div>
        <div className="absolute w-4 h-4 rounded-full bg-white
          "></div>
```

```

        shadow-md select-none transition-all duration-300
        ${plate.color}
        ${plate.animState === 'in' ? 'animate-pulse'}
        ${plate.animState === 'washing' ? 'animate-spin'}
        ${isTop ? 'ring-4 ring-blue-500/50' : ''}
      `}
    >
      <span className="text-lg">{plate.emerald}</span>
      <span className="text-slate-800 text-sm">{plate.slate}</span>
      {isTop && (
        <span className="text-[9px] bg-blue-500/50 rounded-full px-1">
          LIFO
        </span>
      )}
    </div>
  );
  }}}
</div>
))

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800">
</div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 rounded-2xl p-4">
  <div className="absolute top-4 left-4 text-[10px] font-mono">
    Сушилка для чистой посуды
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-end gap-2">
    <div className="order-emerald-800/80 text-[9px] font-mono">
      <Sparkles className="w-3.5 h-3.5 animate-pulse">
        <span>ЧИСТО </span>
      </div>
    </div>

    <div className="flex-1 flex flex-col justify-end">
      {cleanRack.length === 0 ? (

```



```
        </div>
      </div>
    );
  };
};
```

ФАЙЛ 65/82: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

function generateKmpSteps(pattern: string, text: string): Step[] {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];

  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];

    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });

    while (j > 0 && s[i] !== s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Не совпадение` });
      j = pi[j - 1];
    }

    if (s[i] === s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
      j++;
    }
  }
}
```

```

    while (i + z[i] < n && s[z[i]] === s[i + z[i]]) {
      steps.push({ i, l, r, zTarget: z[i], zSource:
        падают! Окно растёт.` });
      z[i]++;
    }

    if (i + z[i] < n) {
      steps.push({ i, l, r, zTarget: z[i], zSource:
        [i]}` разные. Стоп.` });
    } else {
      steps.push({ i, l, r, z: [...z], desc: `Упёр
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксирывае

    if (i + z[i] - 1 > r) {
      l = i;
      r = i + z[i] - 1;
      steps.push({ i, l, r, z: [...z], desc: ` 0
    }

    if (z[i] === pattern.length) {
      steps.push({ i, l, r, z: [...z], found: true
    }
  }
  return { s, steps, patternLength: pattern.length };
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp"
  const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
  const [pattern, setPattern] = useState("aba");
  const [text, setText] = useState("abacaba");

```

```

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

```

```

const { s, steps, patternLength } = useMemo(() => {

```

```

        <button onClick={() => setMode("kmp")}
          className={`px-3 py-1.5 text-slate-400
            hover:text-slate-300`}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")}
          className={`px-3 py-1.5 text-slate-400
            hover:text-slate-300`}
          Z-ФУНКЦИЯ
        </button>
      </div>

```

```

    <div className="flex items-center gap-2"
      <div className="bg-slate-800 p-1 flex items-center gap-2"
        <span className="text-[10px] text-white font-bold font-mono px-2 py-1">
          <input value={pattern} onChange={e}
        </div>
        <div className="bg-slate-800 p-1 flex items-center gap-2"
          <span className="text-[10px] text-white font-bold font-mono px-2 py-1">
            <input value={text} onChange={e}
          </div>
        </div>
      </div>
    </div>

```

```

    <div className="bg-slate-950 rounded-xl border border-slate-900
      px] flex items-center justify-start">
      <div className="flex gap-1.5 px-4 h-full"
        {s.split("").map((char, index) => {
          const isPattern = index < pattern.length;
          const isSeparator = index === pattern.length;

          let borderColor = "border-slate-900";
          let bgColor = "bg-slate-900";
          let textColor = isSeparator ? "text-slate-400" :

```

```
// KMP Logic
```

```

        </div>

        { /* Value array box */ }
        <div className="text-[1
e slate-700/50 font-mono font-bold">
            {mode == "kmp" ? s
        </div>

        { /* KMP fingers */ }
        {mode == "kmp" && step
            <div className="abs
                <span classNam
            </div>
        }}
        {mode == "kmp" && step
            <div className="abs
                <span classNam
            </div>
        }}
        {mode == "kmp" && step
            <div className="abs
                <span classNam
            </div>
        }}

        { /* Z fingers */ }
        {mode == "z" && step.i
            <div className="abs
                <span classNam
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span classNam
            </div>
        }}
    e z-20">

```

```
border-slate-800">
    <RefreshCw strokeWidth={3} size={24} />
  </button>
</div>
```

```
  {/* Description Log */}
  <div className="flex-1 bg-slate-900/50 border border-slate-800
    overflow-hidden">
    <div className="absolute top-0 right-0 p-2" >
      <span className="font-mono text-sm text-slate-400">
        {steps[steps.length - 1].desc}
      </span>
    </div>
    <div className="text-[10px] text-slate-400">
      {steps.length}
    </div>
    <div className="text-sm text-slate-400">
      {step.desc}
    </div>
  </div>
</div>
```

```
{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl">
  <span className="text-2xl mt-1"> </span>
  <p className="text-xs text-slate-400 leading-relaxed">
    Обрати внимание на хитрость с решёткой.
    Мы пишем <b>Шаблон + # + Текст</b>.
    Когда значени {mode === 'kmp' ? "проблема не
    о было найдено!
    Пройди шаги вручную, посмотри, как работает алгоритм
    ля пропуска работы (в Z).
  </p>
</div>
```

```
{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800 p-4">
  <div className="bg-slate-800 px-4 py-2 border border-slate-700">
    <span>Код C++ {mode === 'kmp' ? 'КМП' : 'З'}</span>
  </div>
```

```

    x: number;
    y: number;
    label: string;
    name: string;
}

interface TEdge {
    u: number;
    v: number;
}

const dagNodes: TNode[] = [
    { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
    { id: 1, x: 280, y: 70, label: "1", name: "Дискретка" },
    { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра" },
    { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы" },
    { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение" },
];

const dagEdges: TEdge[] = [
    { u: 0, v: 1 },
    { u: 0, v: 2 },
    { u: 1, v: 3 },
    { u: 2, v: 3 },
    { u: 3, v: 4 },
];

interface TStep {
    desc: string;
    visited: Set<number>;
    fullyProcessed: Set<number>;
    currentNode: number | null;
    topoList: number[];
    dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
    const [steps, setSteps] = useState<TStep[]>([]);

```

```

    for (const to of adj[u]) {
      if (!visited.has(to)) {
        dfs(to);
      } else if (!fullyProcessed.has(to)) {
        // Explaining potential cycle detected (not in DAG)
        recordStep(
          `⚠ Внимание: Рёбра в серую вершину ${to} о
            u,
          );
      }
    }
  }

  fullyProcessed.add(u);
  dfsStack.pop();
  topoList.push(u); // prepend behavior: we write in reverse
  recordStep(
    `DFS: закончили обработку '${dagNodes[u].name}'
      ,
      u,
    );
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    recordStep(
      `Запускаем новый обход DFS от нетронутой верши
        null,
      );
    dfs(i);
  }
}

recordStep(
  "Топологическая сортировка завершена! Полученный
    null,
  );
setSteps(listSteps);

```

```

<div className="bg-slate-900 rounded-xl border border-
  <div className="bg-slate-800 p-4 border-b border-
    <h3 className="font-bold text-white flex items-
      <Clock className="w-5 h-5 text-indigo-400" />
      Топологическая сортировка (Зависимости обучен
    </h3>

```

```

<div className="flex items-center gap-2 bg-slate-
  <button
    onClick={togglePlay}
    className="flex items-center gap-2 px-3 py-
    500 text-white"
  >
    {isPlaying ? (
      <Pause className="w-4 h-4 ml-1" />
    ) : (
      <Play className="w-4 h-4 ml-1" />
    )}
    {isPlaying ? "Пауза " : "Старт "}
  </button>

```

```

  <button
    onClick={reset}
    className="flex items-center justify-center
    ors"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
</div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-12
  {/* Playfield SVG */}
  <div className="lg:col-span-8 bg-[#0B1120] rela
    <svg className="w-full h-full max-w-[650px]"
      <defs>
        <marker
          id="dag-arrow"

```



```

        stroke={isActive ? "#818cf8" : "#334155"}
        strokeWidth={isActive ? "3" : "2"}
        markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow-processed"})`}
        className="transition-all duration-300"
      />
    );
  }
}

```

```

{ /* Nodes */ }
{dagNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited = step.visited.has(node.id);
  const isProcessed = step.fullyProcessed.has(node.id);

```

```

    let fill = "#1e293b";
    let stroke = "#334155";
    let textColor = "text-slate-400";
    void textColor;

```

```

    if (isCurrent) {
      fill = "#312e81";
      stroke = "#818cf8";
      textColor = "text-white font-bold";
    } else if (isProcessed) {
      fill = "#022c22";
      stroke = "#059669";
      textColor = "text-emerald-300";
    } else if (isVisited) {
      fill = "#3730a3";
      stroke = "#6366f1";
      textColor = "text-indigo-200";
    }

```

```

    return (
      <g key={node.id}>
        <rect
          x={node.x - 55}
          y={node.y - 22}

```

```
<h4 className="text-xs font-bold text-slate-400">
  Статус сортировки
</h4>
```

```
<div className="bg-slate-900 border border-slate-800 p-2">
  <p className="text-xs text-indigo-300 min-h-[100px]">
    {step.desc}
  </p>
</div>
```

```
<div className="flex-1 space-y-4 overflow-y-auto">
  {/* Topological List output */}
  <div>
    <span className="text-[10px] text-slate-500">
      РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
    </span>
    <div className="bg-slate-900/60 p-2.5 rounded border border-slate-800 pr-1 text-slate-300">
      {step.topoList.length === 0 ? (
        <span className="text-slate-600 text-sm">
          ожидаем обработку узлов...
        </span>
      ) : (
        getTopoListString()
      )}
    </div>
  </div>
```

```
{/* Color key */}
<div className="space-y-1">
  <span className="text-[10px] text-slate-500">
    ВЕРШИНЫ В DFS:
  </span>
  <div className="flex items-center gap-2">
    <span className="w-3.5 h-3.5 rounded bg-slate-500">
    <span className="text-slate-400 text-[10px]">
      Серые (зашли)
    </span>
  </div>
</div>
```

ФАЙЛ 67/82: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useMemo, useState } from "react";
```

```
/**
```

```
 * Интерактивный Treap — шесть режимов:
```

```
 *   build   — сборка: ВИДИМАЯ сортировка точек (и чем с
```

```
 *           координатная плоскость НАВЕРХУ (ассоциаци
```

```
 *           чек-лист инвариантов (автопроверка, как у
```

```
 *   split   — разрез по x с двумя корзинами (грамотная
```

```
 *   merge   — склейка двух деревьев по y.
```

```
 *   erase   — удаление ПРОИЗВОЛЬНОЙ точки: найти по x,
```

```
 *   layers   — миф «дети в массиве по  $2k/2k+1$ »: работает
```

```
 *   search   — «отсортирую бинарным поиском»: поиск не с
```

```
 *           сортировка платит  $\geq n \cdot \log n$  сравнений, со
```

```
 *
```

```
 * Канонический набор: все x и все y различны  $\Rightarrow$  дерево
```

```
 * Узел идентифицируется парой  $id = "x:y"$ .
```

```
 */
```

```
export interface Pair {
```

```
  key: number;
```

```
  grade: number;
```

```
}
```

```
type NodeId = string;
```

```
const pairId = (key: number, grade: number): NodeId =>
```

```
interface TNode {
```

```
  id: NodeId;
```

```
  key: number;
```

```
  grade: number;
```

```
  left: TNode | null;
```

```
  right: TNode | null;
```

```
export function buildTree(ps: Pair[]): TNode | null {
  let root: TNode | null = null;
  for (const p of sortedByY(ps)) root = insert(root, p);
  return root;
}
```

```
export interface SplitResult {
  l: TNode | null;
  r: TNode | null;
  trace: { atId: NodeId; goLeft: boolean; note: string }
}
export function splitTree(root: TNode | null, x: number): SplitResult {
  const trace: SplitResult["trace"] = [];
  const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
    if (!t) return { l: null, r: null };
    if (t.key <= x) {
      const sub = go(t.right);
      trace.push({ atId: t.id, goLeft: true, note: `(${t.key} >=${x})  
о правое поддерево.` });
      t.right = sub.l;
      return { l: t, r: sub.r };
    }
    const sub = go(t.left);
    trace.push({ atId: t.id, goLeft: false, note: `(${t.key} < ${x})  
о левое поддерево.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
  };
  const res = go(root);
  return { l: res.l, r: res.r, trace };
}
```

```
export interface MergeResult {
  root: TNode | null;
  trace: { aId: NodeId | null; bId: NodeId | null; chosen: boolean }
}
export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
```

```

        return m.root;
    };
    return { root: go(root), trace };
}

const cloneTree = (t: TNode | null): TNode | null =>
    t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

/* ----- валидация treap (для чек-листа и стресс-теста) ----- */

export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean } {
    let bst = true;
    let heap = true;
    let prev: number | null = null;
    const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null): void => {
        if (!n) return;
        if (n.key <= min || n.key > max) bst = false;
        if (parentGrade !== null && n.grade > parentGrade) heap = false;
        walk(n.left, min, n.key, n.grade);
        if (prev !== null && n.key < prev) bst = false;
        prev = n.key;
        walk(n.right, n.key, max, n.grade);
    };
    walk(root, -Infinity, Infinity, null);
    return { bst, heap };
}

/* ----- раскладка сцены дерева ----- */

const layoutTree = (root: TNode | null) => {
    const pos = new Map<NodeId, { px: number; py: number }>();
    let slot = 0;
    const walk = (n: TNode | null, depth: number) => {
        if (!n) return;
        walk(n.left, depth + 1);
        pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 20 });
        slot += 1;
        walk(n.right, depth + 1);
    };
    walk(root, 0);
    return pos;
}

```

```

const h = height ?? autoH;
const byId = (id: NodeId) => nodes.find((n) => n.id === id);
return (
  <div>
    <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full" >
      {edges.map((e) => {
        const a = byId(e.from);
        const b = byId(e.to);
        if (!a || !b) return null;
        return (
          <line
            key={`te-${e.from}-${e.to}`}
            x1={a.px} y1={a.py} x2={b.px} y2={b.py}
            stroke={edgeColor ? edgeColor(e.from, e.to) : 'black'}
            strokeWidth={2.5}
            strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
            className="transition-all duration-500"
          />
        );
      })}
      {nodes.map((n) => {
        const p = pairParts(n.id);
        const pulse = pulseId === n.id;
        return (
          <g key={`tn-${n.id}`} transform={`translate(${n.px} ${n.py})`} >
            <circle r={pulse ? 24 : 20} fill={nodeColor(n)} stroke="black"
              strokeWidth={pulse ? 4 : 2} className="transition-all duration-500" />
            <text y={-2} fill="#fff" fontSize="13" fontWeight="bold">{n.id}</text>
            <text y={13} fill="#cbd5e1" fontSize="10">{p}</text>
          </g>
        );
      })}
    </svg>
    {subtitle && <p className="text-xs text-slate-400">{subtitle}</p>}
  </div>
);
};

```

```

    {drawnEdges.map((e, i) => {
      const a = byPair(e.a);
      const b = byPair(e.b);
      return <line key={`pe-${i}`} x1={a.x} y1={a.y}
        ration-500" />;
    })}

    {points.map((p) => {
      const done = drawnEdges.some((e) => (e.a.key === p.key))
      return (
        <g key={`pp-${p.key}:${p.grade}`} className="point"
          <circle cx={PX(p.key)} cy={PY(p.grade)} r={
            b8" } strokeWidth={1.6} />
          <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
            ">{p.key}</text>
        </g>
      );
    })}

    {pending && (
      <g className="transition-all duration-500">
        <circle cx={PX(pending.key)} cy={PY(pending.g
          >
        <text x={PX(pending.key)} y={PY(pending.grade
          ly="monospace">{pending.key}</text>
        </g>
      )}
    </svg>
  );
};

```

/* ===== РЕЖИМ BUILD ===== */

```

type BuildPhase = "sort" | "connect";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

```

```

const n = userPairs.length;

const userTree = useMemo(() => (userPairs.length >= 1
const canonEdges = useMemo(() => {
  const es: { a: Pair; b: Pair }[] = [];
  const walk = (node: TNode | null) => {
    if (!node) return;
    if (node.left) { es.push({ a: { key: node.key, gr
    if (node.right) { es.push({ a: { key: node.key, g
    walk(node.left);
    walk(node.right);
  };
  walk(userTree);
  return es;
}, [userTree]));
const leftRightEdges = useMemo(() => [...canonEdges].

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => {
    if (phase === "sort") {
      if (algo === "none") { setPlaying(false); return
      if (sortIdx < 3) setSortIdx(sortIdx + 1);
      else setPlaying(false);
    } else {
      const total = order === "y-desc" ? canonEdges.l
      if (connIdx < total) setConnIdx(connIdx + 1);
      else setPlaying(false);
    }
  }, 900);
  return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, ca

const startConnect = (ord: ConnectOrder) => {
  setOrder(ord);
  setPhase("connect");
  setConnIdx(0);
  setPlaying(true);

```



```

    }}
  </div>

  { /* Координатная плоскость – НАВЕРХУ */ }
  <div className="bg-emerald-950/20 rounded-xl p-4" >
    <div className="flex flex-wrap items-center justify-center" >
      <h4 className="text-base font-bold text-emerald-400" >
        <div className="flex items-center gap-2" >
          {phase === "sort" ? (
            <>
              <span className="text-xs text-slate-400" >
                {[ "quicksort", "counting" ] as const }.map((a) => {
                  <button
                    key={a}
                    onClick={() => { setAlgo(a); setSort(a); }}
                    disabled={n < 2}
                    className={`px-3 py-1.5 rounded-lg text-sm
                      bg-emerald-600 text-white" : "bg-slate-700 text-slate-300`}
                  >
                    {a === "quicksort" ? "quicksort (counting)" : "counting"}
                  </button>
                ))}
            </>
          ) : (
            <>
              <span className="text-xs text-slate-400" >
                <button
                  onClick={() => startConnect("y-desc")}
                  className={`px-3 py-1.5 rounded-lg text-sm
                    bg-emerald-600 text-white" : "bg-slate-700 text-slate-300`}
                  >
                    по y ↓ (алгоритм)
                  </button>
                  <button
                    onClick={() => startConnect("left-right")}
                    className={`px-3 py-1.5 rounded-lg text-sm
                      hite" : "bg-slate-700 text-slate-300 hover:bg-emerald-600`}
                    >

```

```

        <li>{connDone ? " " : " "} каждое ребро ведёт
        <li>{connDone ? " " : " "} обход слева-направ
        <li>{connDone ? " " : " "} линий ровно <b>n -
    </ul>
</div>

    {/* Дерево */}
    {order === "y-desc" && userTree && {
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <h4 className="text-sm font-bold text-slate-3
        <TreeView root={userTree} subtitle="Слева-нап
      </div>
    }}
  </div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SplitMode: React.FC = () => {
  const [x0, setX0] = useState(7);
  const [stepIdx, setStepIdx] = useState(0);
  const fullTree = useMemo(() => buildTree(POINTS), [])
  const split = useMemo(() => splitTree(buildTree(POINT
  const steps = split.trace;
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

  const inL = new Set<NodeId>();
  const inR = new Set<NodeId>();
  steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? in

  const lTree = split.l;
  const rTree = split.r;

  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center gap-3

```

```

    })

    <div className="text-xs text-slate-400 bg-slate-900 p-3 rounded-xl">
      <b>Как это работает на самом деле:</b> идём с
      деревом), отдаётся <b>вместе с веткой</b> – д
    </div>
  </div>
);
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
  const [stepIdx, setStepIdx] = useState(0);
  const { l, r } = useMemo(() => splitTree(buildTree(P0
  const merged = useMemo(() => mergeTree(cloneTree(l),
  const steps = merged.trace;
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;
  const cur = steps[idx];

  return (
    <div className="space-y-4">
      <div className="flex items-center gap-3">
        <button onClick={() => setStepIdx(Math.max(0, i
          bg-slate-700 text-white disabled:opacity-30">
        <button onClick={() => setStepIdx(Math.min(step
          font-bold bg-indigo-600 text-white disabled:op
        <span className="text-xs text-slate-400">mar {i
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 g
        <div className="bg-sky-950/20 rounded-xl p-3 bo
          <h5 className="text-sm font-bold text-sky-300
          <TreeView root={l} width={480} height={220} n
        </div>
        <div className="bg-rose-950/20 rounded-xl p-3 b
          <h5 className="text-sm font-bold text-rose-300

```

```

const pathIds = useMemo(() => {
  const ids: NodeId[] = [];
  for (const s of steps.slice(0, idx + 1)) if (s.atId)
    return ids;
}, [steps, idx]);
const pathSet = new Set(pathIds);

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm text-slate-300">Удалить</span>
      <select value={key} onChange={(e) => { setKey(N
        te-700 rounded-lg px-3 py-1.5 text-sm font-mon
        {POINTS.map((p) => <option key={p.key} value=
      </select>
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:op
      <span className="text-xs text-slate-400">шаг {i
    </div>

    {!done ? (
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <TreeView
          root={fullTree}
          nodeColor={(id) => (pathSet.has(id) ? "#783
          nodeStroke={(id) => (pathSet.has(id) ? "#f5
          subtitle="Куча сама умеет удалять только ве
        />
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <h4 className="text-sm font-bold text-slate-3
        <TreeView root={res.root} subtitle="Валидность
        <ul className="mt-2 space-y-1 text-sm">
          <li> обход слева-направо по-прежнему сорти

```

```

return (
  <div className="space-y-4">
    <div className="flex gap-2">
      <button onClick={() => setPick("heap")} className="text-white" : "bg-slate-700 text-slate-300" `>
      <button onClick={() => setPick("treap")} className="text-white" : "bg-slate-700 text-slate-300" `>
    </div>

    {pick === "heap" ? (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-slate-300">
        </div>
        <div className="flex gap-2 overflow-x-auto mb-4">
          {heapArr.slice(1).map((v, i) => (
            <div key={i} className="px-3 py-2 rounded">
              <div className="text-slate-500 text-[10px]>
              <div className="text-white font-bold">{v}</div>
              <div className="text-emerald-400 text-[10px]>
            </div>
          ))}
        </div>
        <p className="text-xs text-slate-400">Полное  $2i/2i+1$  попадает в узел.</p>
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-rose-300">
        </div>
        <TreeView root={full} subtitle="Попробуй «посмотреть»" />
        <div className="flex gap-2 overflow-x-auto mt-4">
          {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((v, i) => (
            const id = slots[i];
            return (
              <div key={i} className={`px-2.5 py-2 rounded border-slate-700`} : "bg-rose-950/40 border-rose-950">
                <div className="text-slate-500 text-[10px]>

```

```
// шаги бинарного поиска на отсортированном массиве д
const bsSteps = useMemo(() => {
  const steps: { lo: number; hi: number; mid: number;
  let lo = 0, hi = sortedByKey.length - 1;
  while (lo <= hi) {
    const mid = Math.floor((lo + hi) / 2);
    if (sortedByKey[mid].key === target) { steps.push(
      нашли за ${steps.length + 1} сравнение(й).` `));
    if (sortedByKey[mid].key < target) { steps.push({
      } — вся левая половина выброшена.` `)); lo = mid
    else { steps.push({ lo, hi, mid, found: false, no
      ` `)); hi = mid - 1; }
  }
  return steps;
}, [sortedByKey]);
```

```
const wrongMid = Math.floor((shuffled.length - 1) / 2
const comparisons = stage === 2 ? bidx + 1 : 0;
```

```
return (
  <div className="space-y-4">
    <div className="flex gap-2">
      {stage < 2 && (
        <button onClick={() => { setStage(stage === 0
          digo-600 text-white">
            {stage === 0 ? "Найти y для x=7 бинарным по
          </button>
        )}
      {stage === 2 && (
        <button onClick={() => setBidx(Math.min(bsSte
          -1.5 rounded-lg text-xs font-bold bg-indigo
          Шаг поиска →
        </button>
      )}
      <button onClick={() => { setStage(0); setBidx(0
        Сброс</button>
    </div>
```

```
};
```

```
/* ===== КОРПУС ===== */
```

```
type Tab = "build" | "split" | "merge" | "erase" | "layers";
```

```
const TABS: { id: Tab; label: string }[] = [
  { id: "build", label: "Собрать" },
  { id: "split", label: "Split ≈ " },
  { id: "merge", label: "Merge  " },
  { id: "erase", label: "Erase   " },
  { id: "layers", label: "Миф: 2k/2k+1" },
  { id: "search", label: "Поиск ≠ сортировка" },
];
```

```
export const TreapBuildViz = () => {
  const [tab, setTab] = useState<Tab>("build");
```

```
  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border" >
      <div className="flex flex-wrap items-center justify-center" >
        <div className="flex items-center gap-3" >
          <div className="p-2.5 bg-emerald-600 text-white" >
            <span className="text-2xl" > </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white" >
              <p className="text-sm text-emerald-300" >Параметры
            </p>
          </div>
        </div>
      </div>
    </div>
```

```
    <div className="flex flex-wrap gap-2 mb-5" >
      {TABS.map((t) => (
        <button
          key={t.id}
          onClick={() => setTab(t.id)}
          className={`px-4 py-2 rounded-lg text-xs font-medium`}

```

```

import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton";
import { SegmentTreeVisualizer } from "./SegmentTreeViz";
import { SplayTreeViz } from "./SplayTreeViz";
import { SplayRotationsViz } from "./SplayRotationsViz";
import { SplayRotationSandbox } from "./SplayRotationSandbox";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimation";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

```

```

/** Разбор поворота и песочница всегда идут парой: посмотри
**

```

```

* Анимация и песочница идут друг под другом, а не в две
* колонке дерево сжималось до нечитаемого размера, а не
* компактности. Между ними – подпись, объясняющая перемену
*/

```

```

const SplayRotationsPair: React.FC = () => (
  <div className="space-y-4">
    <SplayRotationsViz />
    <p className="flex items-start gap-2 text-[12px] leading-3 py-2">
      <span aria-hidden="true"> </span>
      <span>
        Разобрался – проверь себя: ниже то же самое дерево
        не решишь.
      </span>
    </p>
    <SplayRotationSandbox />
  </div>
);

```



```

    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветится",
    Component: SparseTableViz,
  },
  treap: {
    title: "Treap: плоскость → дерево → Split/Merge/Eraser",
    hint: "6 вкладок: Собрать (сортировка + соединение), Разобрать (разрыв),
      , миф про 2k/2k+1, поиск ≠ сортировка.",
    Component: TreapBuildViz,
  },
  "prefix-sums-2d": {
    title: "Префиксные суммы (1D и 2D)",
    hint: "Наведите на число – увидите, из чего оно сложилось",
    Component: PrefixSumViz,
  },
  "dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": {
    title: "Splay-дерево",
    hint: "Покадровый разбор трёх поворотов, песочница",
    Component: () => (
      <div className="space-y-10">
        <SplayRotationsPair />
        <SplayTreeViz />
      </div>
    ),
  },
  "splay-rotations": {
    title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
    hint: "Сверху – покадровый разбор поворота, снизу –",
    Component: () => <SplayRotationsPair />,
  },

```

```
"mst-boruvka": { title: "Борувка", Component: Kruskal },
"string-kmp": { title: "Префикс-функция (КМП)", Component: Kmp },
"string-z-func": { title: "Z-функция", Component: () => {
  title: "Ахо-Корасик",
  hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
  Component: () => (
    <div className="space-y-10">
      <WaterfallAnimationWidget />
      <SalmonAutomatonWidget />
    </div>
  ),
},
intro: { title: "Мнемокарточки", Component: MnemonicCards },
"everyday-basics": {
  title: "Бытовой тренажёр: стек, очередь, куча",
  hint: "Тарелки, очередь в столовой и мешок гипотез",
  Component: Simulator,
},
};
```

```
export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 69/82: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
```

```

};

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');

  // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
  const [textToScan, setTextToScan] = useState<string>('');
  const [currentIndex, setCurrentIndex] = useState<number>(0);
  const [currentNode, setCurrentNode] = useState<number>(0);
  const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
  const [searchLog, setSearchLog] = useState<string>('');
  'Режим поиска: Введите текст и жмите «Следующий шаг»';
  const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 });
  const [foundMatches, setFoundMatches] = useState<{ index: number; match: string }>([]);
  const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

  // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
  // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
  const [trainStep, setTrainStep] = useState<number>(0);

  const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const clean = e.target.value.toUpperCase().replace(/[^a-zA-Z0-9 ]/g, '');
    setTextToScan(clean);
    resetSearchSimulation();
  };

  const resetSearchSimulation = () => {
    setCurrentIndex(0);
    setCurrentNode(0);
    setIsSearchDone(false);
    setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
    setJumpPath(null);
    setFoundMatches([]);
    setActiveSearchCodeLine(1);
  };
};

```



```

        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 5,
        title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
        desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
            "H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'I',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 8,
        title: 'Шаг 5 (Depth 2): Вершина #8 (SH) – КАК СТАТЬ
            ВАЖНЫМ ВОПРОС! Обрабатываем #8 (SH) на D
            ). Рыба плывет в ROOT и ищет ветку "H". У корня
            суффикс "H"!',
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'H',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 3,
        title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
        desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
            е подходят. fail[HER] = ROOT.',
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'R',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 6,

```

```

<div className="bg-slate-800/95 rounded-2xl p-6 border"
  {/* Шапка и переключатель режимов */}
  <div className="flex flex-wrap items-center justify-between"
    <div className="flex items-center space-x-3">
      <span className="text-3xl"> </span>
      <div>
        <h4 className="text-2xl font-bold text-white">
          Анимация водопада: Полное обучение автомата
        </h4>
        <p className="text-sm text-slate-400">
          Визуализация пошагового построения всех функций
        </p>
      </div>
    </div>
  </div>

  {/* Переключатель режимов */}
  <div className="bg-slate-900 p-1.5 rounded-xl border"
    <button
      onClick={() => setActiveMode('training')}
      className={`flex items-center space-x-1.5 p-2`}
      activeMode === 'training'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    >
      <GitBranch className="w-4 h-4" />
      <span>Режим 1: Полное обучение (BFS от корня)</span>
    </button>
    <button
      onClick={() => setActiveMode('search')}
      className={`flex items-center space-x-1.5 p-2`}
      activeMode === 'search'
        ? 'bg-blue-600 text-white shadow-lg shadow-blue-500'
        : 'text-slate-400 hover:text-white'
    >
      <Play className="w-4 h-4" />
      <span>Режим 2: Поиск в тексте</span>
    </button>
  </div>

```

```

        <button
          onClick={() => setTrainStep((prev) => M
          disabled={trainStep === 0}
          className="flex-1 bg-slate-700 hover:bg
ion-all"
        >
          Назад
        </button>
        <button
          onClick={() => setTrainStep((prev) => M
          disabled={trainStep === TRAINING_STEPS_
          className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
        >
          {trainStep === TRAINING_STEPS_INFO.leng
        </button>
      </div>
    </div>

    /* Блок с кодом построения fail-ссылки и опи
    <div className="lg:col-span-2 bg-slate-900/90
      >
        <div>
          <h5 className="text-white font-bold mb-3
            <span className="flex items-center gap-
              <span>❌</span> Код BFS построения fail
            </span>
            <span className="text-xs bg-indigo-950
              Вершина: #{targetTrainingNode.id} ({t
            </span>
          </h5>
          <div className="bg-slate-950 p-4 rounded-
            <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-500 text-indigo-300 font
              <span>1. let u = trie[r][char]; // Те
              {currentTrainInfo.codeLine === 1 && <
            тут</span>}
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

        </div>
      </div>
    ) : (
      /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
      <div className="grid grid-cols-1 lg:grid-cols-3" >
        <div className="bg-slate-900/60 p-5 rounded-x" >
          <div>
            <h5 className="text-white font-bold mb-2" >
              <span>» </span> Текст для сканирования
            </h5>
            <p className="text-xs text-slate-400 mb-4" >
              Введи текст, чтобы лосось просканировал
            </p>
            <input
              type="text"
              value={textToScan}
              onChange={handleTextChange}
              maxLength={15}
              placeholder="ВВЕДИТЕ ТЕКСТ..."
              className="w-full bg-slate-800 border border-b
            cus:outline-none focus:border-blue-500"
            />
          </div>

          <div>
            <div className="mb-4 flex flex-wrap gap-1" >
              <span className="text-xs text-slate-400" >
                {textToScan.split('').map((char, idx) =>
                  <span
                    key={idx}
                    className={`w-7 h-8 flex items-cent
                      idx === currentIndex
                        ? 'bg-blue-600 text-white shadow
                        : idx < currentIndex
                        ? 'bg-emerald-950/60 text-emera
                        : 'bg-slate-800 text-slate-500'
                    }`}
                  >

```



```

        <div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
        <span>2. while (curr != root && !trie
        {activeSearchCodeLine === 2 && <span>
рыжок по fail!</span>}
        </div>
        <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold' :
        <span>3. curr = trie[curr].get(char)
        {activeSearchCodeLine === 3 && <span>
ем по течению</span>}
        </div>
        <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
        <span>4. if (is_terminal[curr]) report
        {activeSearchCodeLine === 4 && <span>
овпадение!</span>}
        </div>
        </div>
</div>

<div>
    <h5 className="text-slate-300 font-bold ml
    <AlertCircle className="w-3.5 h-3.5 tex
    </h5>
    <div className="bg-slate-800 p-3.5 rounded
tems-center">
        {searchLog}
    </div>
</div>
</div>
</div>
))}

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
    {/* Водопадные фоновые эффекты */}

```



```

// В режиме обучения показываем fail-ссыл
// Найдем индекс шага, на котором обработ
const stepIdxForNode = TRAINING_STEPS_INF
if (activeMode === 'training' && trainSte

    let isHighlighted = activeMode === 'search
e === 'fail';
    let isJustEstablished = activeMode === 't
-ссылку

// Вычисляем изогнутую кривую для красоты
const dx = targetNode.x - node.x;
const dy = targetNode.y - node.y;
const cx = node.x + dx / 2 + (node.id % 2
const cy = node.y + dy / 2 - 30;

return (
    <g key={`fail-${node.id}`}>
        <path
            d={`M ${node.x} ${node.y} Q ${cx} $
            fill="none"
            stroke={isHighlighted ? '#f43f5e' :
            strokeWidth={isHighlighted || isJus
            strokeDasharray="6,6"
            className={isHighlighted || isJustE
            opacity={isHighlighted || isJustEst
        />
        {isJustEstablished && (
            <text x={cx} y={cy - 10} fill="#10b
        >
            fail[{node.label}] = {targetNode.
        </text>
        )}
    </g>
    );
    )}
    )}
    )}

    {/* Рендер вершин (бассейнов водопада) */}

```

```

        { /* Метка искомой вершины в режиме об
        {isTargetChild && {
            <g transform={`translate(${node.x},
                <rect x="-35" y="-12" width="70" h
                <text x="0" y="2" fontSize="10" f
                    ИЩЕМ FAIL
                </text>
            </g>
        }}

        { /* Индикатор словарных слов */}
        {hasWords && {
            <g transform={`translate(${node.x +
                <circle cx="0" cy="0" r="9" fill=
                <text x="0" y="3" fontSize="10" f
                    *
                </text>
            </g>
        }}
    </g>
    );
    }}}

    { /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
    <g>
        { /* Круг-подложка под рыбу */}
        <circle
            cx={activeFishPosNode.x + 15}
            cy={activeFishPosNode.y - 25}
            r="18"
            fill={activeMode === 'training' ? '#8b5
            stroke="#ffffff"
            strokeWidth="2"
            style={{ transition: 'cx 0.8s cubic-bez
            className="shadow-lg"
        />
        { /* Сама рыба */}
        <text

```

```
        </div>
      )})
    </div>
  })
</div>
})
</div>
};
};
```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 70/82: src/components/visualizers/PrefixSumViz.tsx
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ: 1000

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка — наведение на число:
 * * в 1D наводим на P[i] — подсвечивается кусок массива
 * * в 1D наводим на a[i] — видно, в какие префиксы он входит
 * * в 2D наводим на ячейку — подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения-исключения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
];
```



```

        }` }
        title={`P[${i}] = сумма a[0..${i} - 1]
    >
        {v}
    </div>
    );
    }}}
</div>
</div>
</div>

```

```

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border
    {hover?.kind === "pref" ? {
        hover.i === 0 ? {
            <p className="text-slate-300">
                <b className="text-emerald-400">P[0] = 0<
                отрезка, начинающегося с нуля.
            </p>
        ) : {
            <p className="text-slate-300">
                <b className="text-emerald-400">
                    P[{hover.i}] = {pref[hover.i]}
                </b>{" "}
                – это сумма всего, что набежало от начала
            <span className="font-mono text-indigo-300">
                a[0..{hover.i} - 1] = {A1.slice(0, hove
            </span>
        </p>
    )
    ) : hover?.kind === "a" ? {
        <p className="text-slate-300">
            <b className="text-indigo-400">
                a[{hover.i}] = {A1[hover.i]}
            </b>{" "}
            входит во все префиксы, начиная с{" "}
            <span className="font-mono text-emerald-300">
                подвинуть границу запроса.

```

```

    }
  }
  return p;
}, [n, m]));

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">Исходная матрица
        <div className="inline-block">
          {A2.map((row, i) => {
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                return (
                  <div
                    key={j}
                    onClick={() => {
                      setRect((r) => {
                        r.r1 = i <= r.r1 ? i : r.r1;
                        r.r2 = i >= r.r2 ? i : r.r2;
                        r.c1 = j <= r.c1 ? j : r.c1;
                        r.c2 = j >= r.c2 ? j : r.c2;
                      });
                    }}
                    className={inRect ? "bg-slate-200" : "bg-white"}
                  >{v}</div>
                );
              })}
            </div>
          })}
        </div>
      </div>
    </div>
  </div>
);

```



```

        className={` ${cellBase} cursor-he
        isHover ? "bg-amber-500 text-wh
        }` }
    >
    {v}
    {corner && !isHover && {
        <span className="absolute -top-
ht">
        {corner}
        </span>
    }}
    </div>
    );
    }}}
    </div>
    )))
    </div>
    </div>
    </div>

<div className="min-h-[54px] bg-slate-900 border
    {hover ? {
        <p className="text-slate-300">
            <b className="text-amber-400">
                P[{hover.i}][{hover.j}] = {P[hover.i][hov
            </b>{" "}
            – сумма всего прямоугольника от левого верх
        </p>
    } : {
        <p className="text-slate-500">Наведите курсор
    }}
    </div>

<div className="bg-slate-900 border border-indigo
    <p className="font-mono text-sm sm:text-base te
        сумма = <span className="text-emerald-400">A<
        <span className="text-rose-400">C</span> + <s
        <span className="text-indigo-300 font-bold">{

```

```
// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
  Array.from({length: 8}, () =>
    Array.from({length: 4}, () =>
      Array(4).fill(0)
    )
  )
);

for (let r = 0; r < 8; r++) {
  for (let c = 0; c < 8; c++) {
    ST2D[r][c][0][0] = val2D[r][c];
  }
}

for (let kx = 0; kx <= 3; kx++) {
  for (let ky = 0; ky <= 3; ky++) {
    if (kx === 0 && ky === 0) continue;
    for (let r = 0; r + (1 << kx) <= 8; r++) {
      for (let c = 0; c + (1 << ky) <= 8; c++) {
        if (kx > 0) {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx-1][ky],
            ST2D[r + (1 << (kx-1))][c][kx-1][ky]
          );
        } else {

```

```

    <AnimatePresence mode="wait">
      {tab === "1d" ? (
        <Viz1D key="1d" />
      ) : tab === "2d_build" ? (
        <Viz2DBuild key="2d_build" />
      ) : (
        <Viz2DQuery key="2d_query" />
      )}
    </AnimatePresence>
  </div>
);
};

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  const covered = hover ? { from: hover.i, to: hover.i + (1 << j) } : null;

  return (
    <div className="flex flex-col gap-5">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="text-lg font-bold text-indigo-600">
            Построение Sparse Table (Min)
          </h3>
          <p className="text-sm text-slate-400">
            Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$ 

```

```

        >
        {v}
    </div>
    <span className="text-[9px] text-slate-100" data-bbox="305 171 1000 191">
</div>
    );
  }}}}
</div>
<div className="mt-2 text-xs min-h-[36px]" data-bbox="205 290 928 310">
  {hover && covered ? (
    <p className="text-slate-300" data-bbox="273 338 776 358">
      <b className="text-sky-400" data-bbox="305 361 776 381">
        ST[{hover.i}][{hover.j}] = {stID[hover.i][{hover.j}]}
      </b>{" "}
      – это минимум на отрезке{" "}
      <span className="font-mono text-sky-300" data-bbox="305 456 1000 476">
        [{covered.from}, {covered.to}]
      </span>{" "}
      длины  $2^{\{hover.j\}}$  = {1 << hover.j}:{" "}
      <span className="font-mono text-slate-400" data-bbox="305 551 1000 571">
        min({arrID.slice(covered.from, covered.to)})
      </span>
    </p>
  ) : (
    <p className="text-slate-500" data-bbox="273 671 776 691">
      Наведите курсор (или тапните) на любое число
    </p>
  )}
</div>
</div>

<div className="overflow-x-auto pb-2" data-bbox="171 836 811 856">
  <div className="inline-block min-w-full bg-slate-100" data-bbox="205 859 1000 879">
    <div className="grid grid-cols-[auto_repeat(4,1fr)]" data-bbox="239 882 1000 902">
      {/* Headers */}
      <div className="text-slate-500 font-mono text-sm" data-bbox="273 931 1000 951">
        i \ j
      </div>
    </div>
  </div>
</div>

```

```

        if (currentStep.i === i && currentStep.isActive) {
            isActive = true;
        }
        if (currentStep.j === j + 1 && currentStep.isActive) {
            isSrc1 = true;
        }
        if (
            currentStep.j === j + 1 &&
            currentStep.i === i - (1 << (currentStep.i - i))
        ) {
            isSrc2 = true; // wait, no. Src2 is not the source.
            // Let's re-eval exact source:
            // We are asking if THIS cell [i][j] is the source.
            if (currentStep.j - 1 === j) {
                if (currentStep.i === i) isSrc1 = true;
                if (currentStep.i + (1 << (currentStep.i - i)) === i) {
                    isSrc2 = true;
                }
            }
        }
    }
}

return (
    <div key={j} className="relative flex flex-col items-center justify-center"
        {!isValid ? (
            <div className="w-[clamp(30px,9vw,11px)] h-[clamp(11px,3vw,17px)]"
        ) : (
            <motion.div
                onMouseEnter={() => setIsVisible(true)}
                onMouseLeave={() => setIsVisible(false)}
                onClick={() => setIsVisible(true) && setStep(j)}
                initial={{ opacity: 0, scale: 0.8 }}
                animate={{
                    opacity: isVisible ? 1 : 0,
                    scale: isVisible ? 1 : 0.8,
                }}
                className={`w-[clamp(30px,9vw,11px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-200
                    hover && hover.i === i && hover.j === j
                    ? "bg-sky-500 text-white"
                    : isActive
                    ? "bg-indigo-500 text-white"
                    : "bg-gray-500 text-gray-500"
                }
            />
        )}
    />
)

```

```

        </React.Fragment>
      )}
    </div>
  </div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-800" >
  {step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-l" >
              ST[{c.i}][{c.j - 1}]
            </span>
            ,
            <span className="text-amber-400 font-bo" >
              ST[{c.i + half}][{c.j - 1}]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{stlD[c.i][c.j]}
            <span className="text-amber-400">
              {stlD[c.i + half][c.j - 1]}
            </span>
          ) ={" "}
            <span className="text-indigo-400 font-b" >
              {stlD[c.i][c.j]}
            </span>
          </p>
        );
      })()
    ) : (

```

```

    }}}
  </div>

  <div className="flex flex-col items-center gap-1
    scrollbar">
    {[...Array(k + 1)].map((_, i) => k - i).map((
      const stepL = 1 << step;
      const stepSize = 8 - stepL + 1;
      const isCurr = step === k;

      return (
        <React.Fragment key={step}>
          {idx > 0 && <div className="text-slate
            <div className={`flex flex-col items-c
              <h4 className={`font-bold mb-3 flex
            }>
              <span>Матрица блоков {stepL}x{step
                <span className={`text-[10px] font
                  rder-indigo-800/40 text-indigo-300' : 'bg-s
                </h4>
                <div className="max-w-full overflow-
                  <div className={`grid gap-[2px] p-
                    go-900/40 shadow-[0_0_25px_rgba(99,102,241,
                    eColumns: `repeat(${stepSize}, max-content)
                  {Array.from({length: stepSize * st
                    const r = Math.floor(idx / step
                    const c = idx % stepSize;

                    let highlightClass = isCurr ? '
                  /50 cursor-pointer' : 'bg-slate-800/50 text
                    let zIndex = 'z-0';

                    const isActive = !!activeCell &
                    ll.c + L) && ((r - activeCell.r) % (1 << st

                    if (isActive) {
                      if (isCurr) {
                        highlightClass = `bg-indigo

```

```

        >
        {ST2D[r][c][step][step]}
      </div>
    );
  }}}
</div>
</div>
</div>
</React.Fragment>
)
}}}  

</div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-4">
  <div className="bg-slate-900 p-4 sm:p-6 rounded-2xl">
    <h3 className="text-lg font-bold text-slate-300">Шаг {k + 1}
    {k === 0 ? (
      <div className="text-slate-400 text-sm leading-6">
        <p className="mb-4">При <span className="font-italic">ST</span>
        и имеют размер <span className="font-italic">ST</span>
        <div className="bg-[#0a0f1e] rounded-xl p-4">
          <span className="text-slate-500">// База данных</span>
          <span className="text-indigo-400">ST</span>
        </div>
        <p className="mt-6 italic text-indigo-400">
          <Play size={14} className="fill-indigo-400"></Play>
        </p>
      </div>
    ) : (
      <div className="flex flex-col gap-4 animate-pulse">
        <div className="bg-[#0a0f1e] rounded-xl p-4 text-slate-300 shadow-inner overflow-x-auto">
          <span className="text-slate-500">// Текущее состояние</span>
          <span className="text-purple-400">int</span>
          <span className="text-amber-300">1</span></div>
          <span className="text-slate-500">// Квантование</span>
          <span className="text-indigo-400">ST</span>
        </div>
      </div>
    )}
  </div>
</div>

```



```

        </div>
        <div className="flex item
            <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
            <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
        </div>
        <div className="flex item
            <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
            <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
        </div>
        <div className="flex item
            <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{activ
            <span className="text
[activeCell.r + prevL][activeCell.c + prevL
        </div>
    </>
    )
    }}())}
</div>

```

```

        <div className="pt-3 mt-3 border-t
            ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
        </div>
        <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
        </div>
    ) : (
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
            Наведите курсор на матрицу {L}x{L
        </div>
    )}
</div>

```

```

        return (
            <div key={idx} className={`
y-center rounded text-[clamp(9px,2.4vw,12px)
0 border-rose-500/50 border scale-105 z-10'
            {val2D[r][c]}
            </div>
        )
    }}}}

```

```

<div
    className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
    style={{
        top: `${(r1 / 8) * 100}%`,
        left: `${(c1 / 8) * 100}%`,
        marginTop: '8px',
        marginLeft: '8px',
        height: 'calc(' + (H/8*100) +
        width: 'calc(' + (W/8*100) +
    }}
/>

```

```

    {/* Показываем 4 угла в самой матрице */}
    <div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
        <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px)`,
            <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
                <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
            </div>
        </div>
    }

```

```

<div className="bg-slate-950 p-4 roun

```

```

<div className="absolute right-3 top-3
  <span className="text-slate-500">// 1.
  <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
  <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
  <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
  <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

  <span className="text-slate-500">// 2.
  <span className="text-indigo-400">int<
  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">В запроса)</span><br/>
  &nbsp;&nbsp;&nbsp;<span className="text-rose-400">r1</span>][<span className="text-white bg-r

  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">во, чтобы правый край уперся в c2)</span><b
  &nbsp;&nbsp;&nbsp;<span className="text-blue-400">r1</span>][<span className="text-white bg-b

  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">рх, чтобы нижний край уперся в r2)</span><b
  &nbsp;&nbsp;&nbsp;<span className="text-emerald-400">nded">r2 - Lx + 1</span>][<span className="

  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">и влево от c2)</span><br/>
  &nbsp;&nbsp;&nbsp;<span className="text-amber-400">">r2 - Lx + 1</span>][<span className="text
    {'}}'}});
</div>

<div className="w-full flex justify-cent
  <div className="flex flex-col items-ce
    <h4 className="text-slate-400 font-bo

```



```

// Just some visual feedback on buttons
document.querySelectorAll('[id^="theme-"]')
    .forEach(btn => btn.classList.remove('outline', 'outline-1'));

if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');

document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors');

}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
    const mobileMenuBtn = document.getElementById('mobile-menu-btn');
    const mobileDrawer = document.getElementById('mobile-drawer');
    const closeDrawerBtn = document.getElementById('close-drawer');
    const drawerOverlay = document.getElementById('drawer-overlay');

    function toggleDrawer() {
        const isClosed = mobileDrawer.classList.contains('closed');
        if (isClosed) {
            mobileDrawer.classList.remove('closed');
            if (drawerOverlay) {
                drawerOverlay.classList.remove('hidden');
            }
        } else {
            mobileDrawer.classList.add('closed');
            if (drawerOverlay) {
                drawerOverlay.classList.add('hidden');
            }
        }
    }

    mobileMenuBtn.addEventListener('click', toggleDrawer);
    closeDrawerBtn.addEventListener('click', toggleDrawer);
    drawerOverlay.addEventListener('click', toggleDrawer);
});

```

```
        const observer = new IntersectionObserver((  
            entries.forEach(entry => {  
                if (entry.isIntersecting && !document.  
                    // Highlight active TOC link lo  
                    const id = entry.target.id;  
                    document.querySelectorAll('aside  
                        if (link.getAttribute('href'  
                            link.classList.add('bg-  
                        } else {  
                            link.classList.remove('l  
                        }  
                    }));  
                }  
            }));  
        }, observerOptions);  
  
        document.querySelectorAll('section[id^="que  
            observer.observe(section);  
        }));  
    });  
</script>  
</body>  
</html>
```

ФАЙЛ 73/82: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР

```
<!DOCTYPE html>  
<html lang="ru" class="scroll-smooth">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, .  
    <title>Пособие по алгебре: От пиццы к предикатам</t  
    <script src="https://cdn.tailwindcss.com"></script>
```

```
}
.formula-scroll::-webkit-scrollbar {
  display: none;
}

mjx-container {
  max-width: 100% !important;
}
mjx-container[display="true"] {
  overflow-x: auto;
  overflow-y: hidden;
  max-width: 100%;
  scrollbar-width: none;
  -ms-overflow-style: none;
  -webkit-overflow-scrolling: touch;
}
mjx-container[display="true"]::-webkit-scrollbar
  display: none;
}

@media (max-width: 640px) {
  html { font-size: 13px; }

  .uno-card { width: 40px; height: 60px; font
  .uno-card::before, .uno-card::after { font-
  .uno-card::before { top: 1px; left: 2px; }
  .uno-card::after { bottom: 1px; right: 2px;

  .uno-card.mini { width: 28px !important; he
  us: 4px; }
  .uno-card.mini::before, .uno-card.mini::aft

  .uno-equation .text-2xl { font-size: 1.2rem
  .uno-equation.gap-4 { gap: 0.5rem !important

  /* Allow horizontal scroll for equations in
  .formula-scroll { flex-wrap: nowrap !importa
  px; }
```

```
body.theme-light #top-controls { background-color: #f0f0f0; }

body.theme-notebook {
  background-color: #fcf8eb !important; /* light beige */
  color: #3b2818 !important; /* brown ink */
  background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
  background-attachment: local !important;
  font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
  background-color: rgba(255, 255, 255, 0.7);
  border-color: #cbd5e1 !important;
  box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1 !important; }
body.theme-notebook .text-slate-400 { color: #546e7a !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace", "Courier New", "Serif"; }
body.theme-notebook #top-controls { background-color: #f0f0f0; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
  0%, 95%, 98%, 100% { transform: scaleY(1); }
  96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
  transform-origin: center;
  transform-box: fill-box;
  animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
  transform-origin: center;
  transform-box: fill-box;
  animation: creepy-blink 5.2s infinite;
}
```



```

<marker id="arrow-yellow" markerWidth="6" m
    <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
</marker>
<marker id="arrow-orange" markerWidth="6" m
    <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d"
</marker>
<marker id="arrow-pink" markerWidth="6" mar
    <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1"
</marker>
<marker id="arrow-cyan" markerWidth="6" mar
    <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
</marker>
<marker id="arrow-indigo" markerWidth="6" m
    <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
</marker>
<marker id="arrow-lime" markerWidth="6" mar
    <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d"
</marker>
<marker id="arrow-green" markerWidth="6" ma
    <path d="M0,0 L6,3 L0,6 Z" fill="#4ade88"
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
    <rect x="0" y="0" width="40" height="10"
    <rect x="6" y="-5" width="8" height="6"

```

```
<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-
```

```
<!-- Левое меню (Оглавление) -->
```

```
<aside id="mobile-drawer" class="fixed inset-y-0  
  nslate-x-full transition-transform duration-300
```

```
  w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
```

```
    <div class="h-full flex flex-col p-6 overflow-y-auto
```

```
      lg:rounded-2xl lg:border lg:border-slate-700
```

```
        <div class="flex justify-between items-center
```

```
          <h3 class="font-black text-transparent bg-clip-text  
            ider text-sm">
```

```
              Содержание
```

```
            </h3>
```

```
          <button id="close-drawer-btn" class="text-slate-500
```

```
            <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6
```

```
              <path stroke-linecap="round" stroke-width="2" d="M6 9 9 6
```

```
            </svg>
```

```
          </button>
```

```
        </div>
```

```
      <nav class="space-y-3 text-sm font-medium">
```

```
        <details class="group" open>
```

```
          <summary class="list-none cursor-pointer">
```

```
            <a href="#question-1" class="text-blue-500 pl-3 font-bold text-blue-300">
```

```
              1. Алгебраические структуры
```

```
            </a>
```

```
          </summary>
```

```
          <div class="pl-6 space-y-2 text-sm">
```

```
            <a href="#section-1-1" class="block">
```

```
            <a href="#section-1-2" class="block">
```

```
            <a href="#section-1-3" class="block">
```

```
            <a href="#section-1-4" class="block">
```

```
          </div>
```

```
        </details>
```

```
      <details class="group">
```

```

        <a href="#q4-root" class="block
        <a href="#q4-count" class="bloc
</div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-5" class:
er-fuchsia-500 pl-3 font-bold text-fuchsia-
    5. Группа корней из единицы
</a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q5-def" class="block
        <a href="#q5-group" class="bloc
        <a href="#q5-prim" class="block
        <a href="#q5-crit" class="block
        <a href="#q5-cyclic" class="blo
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-6" class:
er-lime-500 pl-3 font-bold text-lime-300">
    6. Делимость, НОД и Евклид
</a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q6-1" class="block ho
        <a href="#q6-2" class="block ho
        <a href="#q6-3" class="block ho
        <a href="#q6-4" class="block ho
        <a href="#q6-5" class="block ho
        <a href="#q6-6" class="block ho
    </div>
</details>
```

er-green-500 pl-3 font-bold text-green-400">

9. Кольцо многочленов

</summary>

<div class="pl-6 space-y-2 text

<a href="#q9-1" class="bloc

<a href="#q9-2" class="bloc

<a href="#q9-3" class="bloc

<a href="#q9-4" class="bloc

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-10" clas

der-rose-500 pl-3 font-bold text-rose-400">

10. Делимость и Схема Г

</summary>

<div class="pl-6 space-y-2 text

<a href="#q10-1" class="blo

<a href="#q10-2" class="blo

<a href="#q10-3" class="blo

a>

<a href="#q10-4" class="blo

<a href="#q10-5" class="blo

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-11" clas

der-cyan-500 pl-3 font-bold text-cyan-400">

11. НОД и НОК многочлен

</summary>

<div class="pl-6 space-y-2 text

<a href="#q11-1" class="blo

```
<details class="group">
  <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-400">
    14. Кратность корня. Корень
  </a>
</summary>
<div class="pl-6 mt-2 space-y-2">
  <a href="#q14-1" class="block text-fuchsia-400">
  <a href="#q14-2" class="block text-fuchsia-400">
  <a href="#q14-3" class="block text-fuchsia-400">
    deg)</a>
    <a href="#q14-4" class="block text-fuchsia-400 border border-rose-500/30"> К
  </div>
</details>
```

```
<details class="group">
  <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
    15. Производная. Критерий
  </a>
</summary>
<div class="pl-6 mt-2 space-y-2">
  <a href="#q15-1" class="block text-cyan-400">
  <a href="#q15-2" class="block text-cyan-400">
  <a href="#q15-3" class="block text-cyan-400">
  <a href="#q15-4" class="block text-cyan-400">
  </div>
</details>
```

```
<details class="group">
  <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
    16. Неприводимые многочлены
  </a>
```

```

</nav>
<script>
    document.querySelectorAll('details.'
        detail.addEventListener('toggle
            if (detail.open) {
                document.querySelectorA
                    if (other !== detail
                        other.removeAtt
                    }
            }
        });
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('clie
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersectin
                visibleId = entry.t

// Highlight active

```

```
modal.style.width = '100vw';
modal.style.height = '100vh';
modal.style.backgroundColor = 'white';
modal.style.zIndex = '99999';
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a202c';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей браузеров, для сохранения веб-страницы в PDF необходимо использовать стороннюю веб-студию в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';
```

```

        box.appendChild(textarea);
        btnContainer.appendChild(closeB);
    }
    btnContainer.appendChild(closeB);
    box.appendChild(btnContainer);
    modal.appendChild(box);
    document.body.appendChild(modal);
}

```

```

function downloadMD() {
    let clone = document.querySelector('#clone');

    // Smart markdown parsing based on DOM
    // Smart markdown parsing based on DOM
    clone.querySelectorAll('h2').forEach(el => {
    clone.querySelectorAll('h3').forEach(el => {
    clone.querySelectorAll('.analog').forEach(el => {
    clone.querySelectorAll('li').forEach(el => {
    clone.querySelectorAll('svg').forEach(el => {
        let caption = '';
        if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
            caption = el.nextElementSibling.alt;
        }
        el.textContent = '\n\n[ Изображение: ' + caption + ' ]';
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        let text = clone.innerText;
        text = text.replace(/\n{3,}/g, '\n\n');

        if (window !== window.top) {
            fallbackModal(text, 'Markdown');
            return;
        }
        const blob = new Blob([text], { type: 'text/markdown' });
        const url = URL.createObjectURL(blob);
        const link = document.createElement('a');
        link.href = url;
        link.textContent = 'Скачать Markdown файл';
        clone.appendChild(link);
    });
}

```



```

/* Specific backgrounds
body.bg-white aside > d
body.bg-white [class*="f
nt; }
body.bg-white [class*="f
nt; }
body.bg-white [class*="f
body.bg-white .analogy-l
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-l
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya

```

```

<div class="flex flex-col gap-2">
  <div class="text-xs text-slate-500">
    <button onclick="downloadWord()"
font-bold flex items-center justify-center gap-2>
      <span> </span> Word
    </button>
    <a href="algebra.pdf" target="_blank"
-sm font-bold flex items-center justify-center gap-2>
      <span> </span> PDF
    </a>
    <button onclick="downloadMD()"
-bold flex items-center justify-center gap-2>
      <span> </span> Markdown
    </button>
    <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center gap-2>
      <span> </span> HTML
    </button>
  </div>

  <!-- Смена темы -->
  <div>
    <div class="text-xs text-slate-500">
      <div class="flex items-center gap-2">
        <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white">
          <span> </span> Темная тема
        </button>
        <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white">
          <span> </span> Светлая тема
        </button>
        <button id="theme-terminal"
-center justify-center text-green-400 hover:text-white">
          <span> </span> Терминал
        </button>
      </div>
    </div>
  </div>
</div>

```

```

        </ul>
    </div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
    <h2 class="text-3xl font-black text-cen

        <div class="overflow-x-auto w-full pb-4
            <div class="grid grid-cols-3 gap-4
                <!-- Column headers -->
                <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
                <div class="bg-orange-900/40 bo
ольцо полиномов</div>
                <div class="bg-cyan-900/40 bord
сные числа</div>

                <!-- Row 1: База -->
                <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
                    <div class="text-xs text-yellow
                    <div class="font-bold text-slat
                </a>
                <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
                    <div class="text-xs text-orange
                    <div class="font-bold text-slat
                </a>
                <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
                    <div class="text-xs text-cyan-5
                    <div class="font-bold text-slat
                </a>

                <!-- Row 2: Базовые операции / Евкли
                <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra

```

```

        </a>
        <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-500"
        <div class="font-bold text-slate-700"
        </a>

<!-- Row 5: Корни / Уравнения -->
<div class="hidden md:block"></div>
<a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-500"
        <div class="font-bold text-slate-700"
        </a>

<!-- Row 6: Кратность -->
<div class="hidden md:block"></div>
<a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <div class="hidden md:block"></div>

```



```
        <ul class="list-disc pl-5 sp-1">
          <li><span class="text-w
валось.</li>
          <li><span class="text-w
Просто перепиши формулу с буквой $k$.</li>
          <li><span class="text-w
усок $k$, замени его на жареного карася из
        </ul>
      </div>
```

```
    <!-- Двойная делимость -->
    <div class="bg-slate-800/80 p-6">
      <h3 class="text-xl text-ora
      <p class="text-sm text-slate-400">
        <ul class="list-disc pl-5 sp-1">
          <li><span class="text-w
вверх по уравнениям).</li>
          <li><span class="text-w
рху вниз, показывая, что  $\Delta \vdots r_i$ 
          <li><span class="text-w
х. Твой  $d$  больше.</li>
        </ul>
      </div>
```

```
    <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
    <div class="grid grid-cols-1 md:grid-cols-2">
      <div class="bg-slate-800 p-6">
        <h4 class="text-red-400">
        <p class="text-sm text-
        <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
      </div>
```

```
    <div class="bg-slate-800 p-6">
      <h4 class="text-red-400">
```



```
/** Собирает CSS страницы: сначала пробуем правила, при  
async function collectCss(): Promise<string> {  
  const parts: string[] = [];  
  for (const sheet of Array.from(document.styleSheets))  
    try {  
      const rules = (sheet as CSSStyleSheet).cssRules;  
      parts.push(Array.from(rules).map((r) => r.cssText));  
    } catch {  
      const href = (sheet as CSSStyleSheet).href;  
      if (!href) continue;  
      try {  
        const res = await fetch(href);  
        if (res.ok) parts.push(await res.text());  
      } catch {  
        /* чужой домен – пропускаем */  
      }  
    }  
  }  
  return parts.join("\n");  
}
```

```
const EXTRA_CSS = `  
/* – правки для автономного файла – */  
html { color-scheme: dark; }  
body { background: #020617; color: #e2e8f0; font-family:  
a.term-hint { text-decoration: none; }  
.export-shell { max-width: 62rem; margin: 0 auto; padding:  
.export-head { border-bottom: 1px solid #1e293b; padding:  
.export-note { background: #0f172a; border: 1px solid #  
  border-radius: .5rem; padding: .85rem 1rem; font-size:  
.export-note a { color: #a5b4fc; }  
.export-gloss { margin-top: 3rem; border-top: 1px solid  
.export-gloss h2 { color: #fff; font-size: 1.25rem; fon  
.export-term { background: #0f172a; border: 1px solid #  
.export-term h3 { color: #c7d2fe; font-size: .95rem; fo  
.export-term p { margin: 0 0 .3rem; font-size: .85rem;  
.export-term .formal { color: #94a3b8; font-size: .8rem
```



```

        el.replaceWith(link);
    });

    // <details> в файле лучше раскрыть: иначе код и разб
    host.querySelectorAll("details").forEach((d) => d.set

    return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
    if (!termIds.length) return "";
    const items = termIds
        .map((id) => {
            const e = GLOSSARY_BY_ID.get(id);
            if (!e) return "";
            return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
        })
        .join("\n");

    return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
    ны сюда.</p>
    ${items}
</section>`;
}

function wrap(opts: { title: string; subtitle: string;
    const stamp = new Date().toLocaleString("ru-RU");
    return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">

```

```

    a.click();
    a.remove();
    setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
    const num = title.match(/^s*(\d+)/)?[1];
    const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
    return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter): Promise<string> {
    const css = await collectCss();
    const { body, termIds } = prepareBody(chapter.content);
    const html = wrap({
        title: chapter.title,
        subtitle: chapter.category || "Универсальное пособие",
        css,
        body,
        gloss: glossarySection(termIds),
        origin: window.location.origin,
    });
    triggerDownload(html, safeName(chapter.title, chapter.id));
}

/** Выгрузить всё пособие одним файлом со сквозным оглавлением. */
export async function downloadBookHtml(chapters: Chapter[]): Promise<string> {
    const css = await collectCss();
    const allTerms: string[] = [];
    const bodies: string[] = [];

    const toc = chapters
        .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a>`)
        .join("\n");

    chapters.forEach((c) => {

```



```
-----
  dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
  dump("НЕТ 2D (аналогия есть)", gapViz);
  console.log(`Пропущенные номера билетов 1-24: ${missi
  console.log(`Визуализаторы без главы: ${orphanViz.joi
}
```

ФАЙЛ 77/82: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
-----
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossa
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m
  external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.\*\+?\^\$\{\}\(\)|[\]\[\]]/g, "\\
const re = new RegExp(
  `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
  "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label
```

Универсальное пособие • полный исходный код

```
-----
console.log("\nГлав без единой подсказки:", bad.length,
console.log("Терминов в глоссарии:", GLOSSARY.length, "
console.log("Не встретились:", GLOSSARY.filter((g) => !
```

ФАЙЛ 78/82: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

```
-----
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
  if (!fs.existsSync(PAGES_DIR)) {
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
  }

  const pages = fs
    .readdirSync(PAGES_DIR)
    .filter((f) => f.endsWith(".png"))
    .sort();

  if (pages.length === 0) throw new Error("Не найдено ни

  ensureDir(OUT_DIR);
```

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть)
 * с фоллбэком на обход файловой системы, если git недоступен
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir, ensureFile } from "../utils";

/** Расширения, которые считаем «кодом» и включаем в дайджест */
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".yml"
]);

/** Явные исключения: генерируемое, бинарное и просто служебные файлы */
const EXCLUDE_FILES = new Set(["package-lock.json", "build", "dist"]);
const EXCLUDE_DIRS = ["node_modules", "dist", "build"];

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (вспомогательные)"],
  [/^src\/components\/$/, "Интерактивные компоненты и симуляторы"],
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
];
```

```

        else files.push(rel);
    }
};
walk(ROOT);
}

return files
    .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
    .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
    .filter((rel) => CODE_EXT.has(path.extname(rel)))
    .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
    .sort((a, b) => {
        const ia = ORDER.indexOf(categoryOf(a));
        const ib = ORDER.indexOf(categoryOf(b));
        return ia !== ib ? ia - ib : a.localeCompare(b);
    });
}

/** Достаём заголовки глав из данных пособия (без запуска)
function extractChapters(files) {
    const chapters = [];
    const seen = new Set();
    for (const rel of files.filter((f) => f.startsWith("src/"))) {
        const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
        const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
        let m;
        while ((m = re.exec(src)) !== null) {
            const [, id, title] = m;
            if (seen.has(id)) continue;
            seen.add(id);
            chapters.push({ id, title, file: rel });
        }
    }
    return chapters.sort((a, b) => {
        const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
        const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
        return na - nb;
    });
}

```

```

    if (cat !== current) {
      current = cat;
      push();
      push(`  # ${cat}`);
    }
    push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`);
  });
  push();
  push();

  current = null;
  files.forEach((rel, i) => {
    const cat = categoryOf(rel);
    if (cat !== current) {
      current = cat;
      push(HR);
      push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
      push(HR);
      push();
    }
    const content = fs.readFileSync(path.join(ROOT, rel));
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
  });

  push(HR);
  push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
  push(HR);

  const txt = out.join("\n") + "\n";
  fs.writeFileSync(TXT_PATH, txt, "utf8");

```



```
-----  
/** Параметры вёрстки страницы (подобраны под A4 @150dpi)  
export const PAGE = {  
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=  
  density: Number(process.env.EXPORT_DENSITY ?? 150),  
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_  
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",  
  size: "A4",  
  font: "DejaVu-Sans-Mono",  
  fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan  
  pointsize: 8,  
  /** Максимум символов в строке (далее – жёсткий пере  
  cols: 138,  
  /** Строк содержимого на странице (+2 служебные: шапка  
  rows: 76,  
};  
  
export const HR = "=".repeat(PAGE.cols);  
export const SUBHR = "-".repeat(PAGE.cols);  
  
export function ensureDir(dir) {  
  fs.mkdirSync(dir, { recursive: true });  
}  
  
export function humanSize(bytes) {  
  if (bytes < 1024) return `${bytes} B`;  
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toF  
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;  
}  
  
/** ImageMagick 7 (`magick`) или 6 (`convert`). */  
export function imageMagickCmd() {  
  for (const cmd of ["magick", "convert"]) {  
    try {  
      execFileSync(cmd, ["-version"], { stdio: "ignore"  
      return cmd;  
    } catch {  
      /* пробуем следующий */  
    }  
  }  
}
```

```

    let first = true;
    while (rest.length > 0) {
      const width = first ? PAGE.cols : PAGE.cols - indent;
      parts.push((first ? "" : indent) + rest.slice(0, width));
      rest = rest.slice(width);
      first = false;
    }
    return parts;
  }

function paginate(txt) {
  const source = txt.replace(/\r\n/g, "\n").split("\n");
  const pages = [];
  for (let i = 0; i < source.length; i += PAGE.rows) {
    pages.push(source.slice(i, i + PAGE.rows));
  }
  return pages;
}

async function main() {
  if (!fs.existsSync(TXT_PATH)) {
    throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
  }

  const im = imageMagickCmd();
  const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

  fs.rmSync(PAGES_DIR, { recursive: true, force: true });
  ensureDir(PAGES_DIR);

  const total = pages.length;
  const jobs = pages.map((lines, idx) => {
    const num = String(idx + 1).padStart(4, "0");
    const header = `Универсальное пособие • полный исходный код
    Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`)
    `}стр. ${idx + 1} / ${total}`;
    const body = [header, "-".repeat(PAGE.cols), ...lines];
    const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
    fs.writeFileSync(txtFile, body.join("\n"));
  });
}

```

```
const missing = jobs.filter((j) => !fs.existsSync(j.p
if (missing.length) throw new Error(`Не отрендерены с

const bytes = jobs.reduce((s, j) => s + fs.statSync(j
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAG
}

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message)
  process.exit(1);
});
```

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

ФАЙЛ 82/82: [.github/workflows/rebuild-pdf.yml](#)
КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 128 | РАЗМЕР: 4.0

name: Rebuild code PDF

```
# Пайплайн пересборки PDF при каждом коммите:
#
#   исходный код  → full_code_all_pages.txt   (script
#
#               |
#               |→ page-0001.png ... page-NNNN.png (
#
#                       |
#                       |→ full_code_all_page
#
# Готовые TXT и PDF коммитаются обратно в ветку (их отда
# промежуточные PNG сохраняются как artifacts запуска.
```


Универсальное пособие • полный исходный код

=====

КОНЕЦ ДОКУМЕНТА • 82 файлов • сгенерировано 11.09.2026,

=====